Machine Learning for Future Engineers



Rohit Gupta

C#Corner

Machine Learning for Future Engineers

Rohit Gupta

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law. Although the author/co-author and publisher have made every effort to ensure that the information in this book was correct at press time, the author/co-author and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause. The resources in this book are provided for informational purposes only and should not be used to replace the specialized training and professional judgment of a health care or mental health care professional. Neither the author/co-author nor the publisher can be held responsible for the use of the information provided within this book. Please always consult a trained professional before making any decision regarding the treatment of yourself or others.

Author – Rohit Gupta
Publisher – C# Corner
Editorial Team – Deepak Tewatia, Baibhav Kumar
Publishing Team – Praveen Kumar
Promotional & Media – Rohit Tomar

Background and Expertise

Rohit Gupta is a Technical Trainer with a Master's Degree in Informatics and a Bachelor's degree in Electronic Science, both from the University of Delhi. As a C# Corner MVP, author, and speaker, Rohit is dedicated to advancing the fields of Artificial Intelligence, Machine Learning, and Python. His vision is to create the world's first human-like humanoid, driven by his belief that the bond between machines and humans can make the world a better place.

— *Rohit Gupta*

Table of Contents:

Introduction 5
Machine Learning 9
Data Science 16
Linear Regression 31
Logistic Regression 49
Multiple Linear Regression 66
Decision Tree 79
Naïve Base 97
K Nearest Neighbor 112
K-Means Clustering 126

1

Introduction

Overview

In this chapter, we identify the target audience, including students and professionals eager to learn Machine Learning with Python. Prepare yourself for a journey tailored to a diverse readership.

Who is this Book for?

This is a book written for students, professionals, enthusiasts, learners who wish to learn about Machine Learning and its implementation using Python.

Pre-Requisites

- To get the best out of this book, it is recommended to have prior knowledge of object-oriented programming using Python.
- The current version of Python is 3.8. Since Python 2 will be getting obsolete soon and TensorFlow 2.1 is the last stable release to support Python 2, hence we will be using Python 3 throughout the book.
- You can check your version using python3 --version

To learn about programming in Python 3, you can read my book on Python 3.

- It is recommended to have prior experience of using Google Colab as we will be using Google Colab as our python environment.

To do a hand-on, please feel free to visit.

It is recommended to have prior experience of using the following Python Libraries

- NumPy
- Pandas
- SciKit-Learn
- Matplotlib
- Seaborn
- TensorFlow

To fulfill this requirement, you are advised to read my book “Python for Machine Learning”.

What Is in This Book?

In this book, we will learn the following machine learning algorithms:

- Linear Regression
- Logistic Regression
- Multiple Linear Regression
- Decision Tree
- Naïve Bayes
- K Nearest Neighbors
- K-Means

In this book, we will discuss various machine learning algorithms, implement each of the algorithm using Python 3 with the help of

- Functions
- SciKit-Learn
- TensorFlow

After completing the book, you should be able to use and implement each of the listed Machine Learning algorithms to your benefit.

Steps to install python 3

Windows

Python latest version can be download from https://www.python.org/downloads/.

- At the time of writing this, the latest version was 3.8.1.
- Run the installer file with the Administrator.
- Make sure to tick the following
- Add Python 3.x to PATH

Adding python to Path will make python available to you over the whole system

- Click on Install Now
- Wait till the process is over.

Ubuntu

- For Ubuntu 17.10+, python3 comes installed already
- For Ubuntu 16.10 and 17.03

sudo apt-get update && sudo apt-get install python3
For Ubuntu 14.04 and 16.04

sudo add-apt-repository ppa:deadsnakes/ppa

- sudo apt-get update
- sudo apt-get install python3

MAC OS/ MAC OS X

- Install Homebrew, type in the following command and hit enter
- /usr/bin/ruby -e "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install)"
- Open Terminal and type in the following command and hit enter brew install python3

Key terms

What batches size?

A batch size is the subset of the training dataset that is utilized in one iteration. It has been observed in practice that when using a larger batch there is a significant degradation in the quality of the model, as measured by its ability to generalize. The lack of generalization ability is since large-batch methods tend to converge to sharp minimizers of the training function.

In general, batch size of 32 is a good starting point, and you should also try with 64, 128, and 256. Batch size below 32 might get too slow because of significantly lower computational speed because of not exploiting vectorization to the full extent.

What Learning Rate to Use?

The Learning Rate is the size of the steps we take to reach a (local) minimum. A lower learning rate means more steps and therefore trains for a longer time while higher learning rate means less steps which therefore trains for shorter time. Also, too low a learning rate never progresses,

and too high a learning rate causes instability and never converges. In between, there is a band of “just right” learning rates that successfully train.

There is no single learning rate that works for all optimizers. Learning rate can affect training time by an order of magnitude. Learning rate performance did not depend on model size. The same rates that performed best for 1x size performed best for 10x size.

Why And What Optimizer to Use?

Gradient Descent is one of the most popular algorithms to perform optimization and by far the most common way to optimize neural networks. Stochastic Gradient Descent (SGD) is a variant of Gradient Descent in which only a few of the samples (selected by batch_size) are used to compute gradient in every iteration. SGD can be optimized with parameter 'Momentum' which is a method that helps accelerate SGD in the relevant direction and dampens oscillations.

One of the challenges of SGD is that the same learning rate applies to all parameter updates. If the dataset is sparse and the features have very different frequencies, it will not be necessary to update all of them to the same extent, but rather a larger update for rarely occurring features. This problem is addressed by Adaptive Learning-Rate optimizers (Adagrad, Adadelta, RMSprop and Adam) that adapt the learning rate to the parameters, performing larger updates for infrequent and smaller updates for frequent parameters.

The main downside of the Adaptive Learning Rate optimizers is that they require more computation to be performed for each parameter in each training step and more state to be retained for each parameter. While a simple SGD Optimizer could equally be used with less computational requirements, it would require more hyperparameter tuning (learning rate) before it would converge as quickly.

In all, most of the optimizers manage to converge in a reasonable time.

2

Machine Learning

Overview

In this chapter, we will be studying Machine Learning. One thing that I believe is that if we can correlate anything with us or our life, there are greater chances of understanding the concept. So, I will try to explain everything by relating it to humans.

What is Machine Learning?

According to Wikipedia, Machine learning is the scientific study of algorithms and statistical models that computer systems use to perform a specific task without using explicit instructions, relying on patterns and inference instead. It is seen as a subset of artificial intelligence.

Machine learning is an application of artificial intelligence (AI) that provides systems the ability to automatically learn and improve from experience without being explicitly programmed. Machine learning focuses on the development of computer programs that can access data and use it to learn for themselves.

The following statement was said by Tom Mitchell:

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

The meaning of the above statement can be understood as:

In the case of Spam Detection, Task T is detecting a mail as spam or not, Performance measure P is how much mail is correctly detected as spam and Experience E is the number of mails detected as spam and not-a spam. So, if the number of false-positive results decreases continuously on each iteration, then it can be said that the system is learning to classify emails as spam and not-a spam.

Another example could be, Alexa, amazon's NLP service, Task T is to execute the given command, Performance Measure P is the extent to which Alexa is able to "understand" our commands and Experience E is the number of times the given command is executed rightly. So, if Alexa's command execution capabilities improve with time and it can develop a better "understanding" of what "humans" means.

How do we do machine learning?

I would explain machine learning in layman's terms in the following way,
I don’t know how many of you remember the way we were taught A-Z in our kindergarten. For those who don’t remember, we were told to, again and again, write A to Z, till the time we were able to recall the name by seeing the diagram of that. In the same way, we train machines, where the machines are given x amount of data points, and the machine tries to form a relationship among those data points, this what I did in my school life, I had a problem learning chemical periodic table, so my teacher gave me an algorithm to learn i.e. "LiBeB CNOFNe NaMgAl SiPSClAr", with this I was able to learn 1st eighteen elements in chronological order.

ML algorithms tend "overfit" data i.e. they do not allow any other value to fit into the relationship equation, this is what we do, for example, I had an argument with someone, so its human nature to think that the other person does not understand us or do not want to understand us.

Some basic terms

Data Science

According to Wikipedia, Data science is a multi-disciplinary field that uses scientific methods, processes, algorithms, and systems to extract knowledge and insights from structured and unstructured data. Data science is the same concept as data mining and big data: "use the most powerful hardware, the most powerful programming systems, and the most efficient algorithms to solve problems".

Data Mining

According to Wikipedia, Data mining is the process of discovering patterns in large data sets involving methods at the intersection of machine learning, statistics, and database systems. Data mining is an interdisciplinary subfield of computer science and statistics with an overall goal to extract information (with intelligent methods) from a data set and transform the information into a comprehensible structure for further use. Data mining is the analysis step of the "knowledge discovery in databases" process or KDD. Aside from the raw analysis step, it also involves database and data management aspects, data pre-processing, model and inference considerations, interestingness metrics, complexity considerations, post-processing of discovered structures, visualization, and online updating

Statistical Learning

According to Wikipedia, Statistical learning theory is a framework for machine learning drawing from the fields of statistics and functional analysis. Statistical learning theory deals with the problem of finding a predictive function based on data. Statistical learning theory has led to successful applications in fields such as computer vision, speech recognition, bioinformatics, and baseball.

Knowledge discovery in databases (kdd)

According to Wikipedia, Knowledge discovery describes the process of automatically searching large volumes of data for patterns that can be considered knowledge about the data. It is often described as deriving knowledge from the input data. Knowledge discovery developed out of the data mining domain and is closely related to it both in terms of methodology and terminology.

The most well-known branch of data mining is knowledge discovery, also known as knowledge discovery in databases (KDD). Just as many other forms of knowledge discovery it creates abstractions of the input data.

The knowledge discovery in databases (KDD) process is commonly defined with the stages:

- Selection
- Pre-processing
- Transformation
- Data mining
- Interpretation/evaluation

What can be called data? How data can be used?

Everything and anything can be called data. Data is an abstract term, which can be used for each and every type of data. The amount of data that is generated on a daily basis can be estimated by seeing the following figures:

- Google: processed 24 Peta Bytes of data every data per day
- Facebook: 10 million photos uploaded every hour
- YouTube: 1 hour of video uploaded every second
- Twitter: 400 million tweets per day
- Astronomy: Satellite Data is in hundreds of petabytes
- Note: 1 petabyte is 1 million megabytes or 1 quadrillion byte or 1000 trillion bytes or 1e-15 bytes

So, from the above values, you can very easily understand that daily we produce around 1 GB or Gigabyte of Data. Even the above-written figures are also a type of data.

Which Data Is Converted into Information?

The theoretical definition of Information is facts provided or learned about something or someone or we can say data can be said to be information if it is "useful to me". Information is an abstract term, i.e. for example, if I am a manager and I must review the work of my team, so the data of work done will become information for me, while on the other hand, this same data may have no value for a person who is not an employee of the same company.

Another example of the same can be you are currently reading this chapter, the data written in this chapter is "information" to you, but for a person who doesn't intend to learn about ML, this is mear data.

So, I would conclude by saying "Everything is information and not everything is information".

How can information be converted into knowledge?

The theoretical definition of knowledge is facts, information, and skills acquired through experience or education; the theoretical or practical understanding of a subject or we can say is knowledge is the "information" that we use to transform a "theoretical concept" into "practically feasible product".

For Example, you are currently reading this chapter, and if you use this "information" to implement it and produce an observable and notable product than this information is said to be converted into knowledge.

For instance, you study linear regression, and you used the algorithm to make a classifier that can differentiate between an animal and a human, then that means that you converted your "linear regression" information into "knowledge".

Advantages/ Features of MI

- It is used in a variety of applications such as banking and financial sector, healthcare, retail, publishing and social media, robot locomotion, game playing, etc.
- It is used by Google and Facebook to push relevant advertisements based on the user's past search behavior.
- It has capabilities to handle multi-dimensional and multi-variety data in dynamic or uncertain environments.
- It allows time cycle reduction and efficient utilization of resources.
- Due to machine learning, there are tools available to provide continuous quality improvement in large and complex process environments.
- Source programs such as RapidMiner helps in the increased usability of algorithms for various applications.

Disadvantages/Shortcomings of MI

- The acquisition of relevant data is a major challenge. Based on different algorithms data need to be processed before providing as input to respective algorithms. This has a significant impact on results to be achieved or obtained.
- Interpretation of results is also a major challenge to determine the effectiveness of machine learning algorithms.
- Based on which action to be taken and when to be taken, various machine learning techniques need to be tried.
- The next level of machine learning technology is being researched.

The Data Science process

Data Collection

this process involves collection of data using various tools like interviews, questionnaires, one-to-one interactions, group discussions, online surveys, and surfing the web. Our motive during this process is that the data so collected can directly be converted into information.

Data Preparation

This process involves, cleaning of data i.e. removing or replacing empty values and null values with placeholder value, re-arranging the data in the specified format, and feature/variable engineering i.e. selecting the required features/variables, interpolation, and scaling of features/variables.

EDA

Exploratory Data Analysis refers to the critical process of performing initial investigations on data so as to discover patterns, spot anomalies, to test hypotheses and to check assumptions with the help of summary statistics and graphical representations.

Machine Learning

Here, we tried to generate a model that can be used to represent a relationship between the given data. It also involves training the machine to be able to understand data and perform required operations.

Data Visualization

Data visualization refers to the techniques used to communicate data or information by encoding it as visual objects (e.g., points, lines or bars) contained in graphics. The goal is to communicate information clearly and efficiently to users. It is one of the steps in data analysis or data science. This visualization can be of a mathematical, graphical or an inference.

Types of Machine Learning

Supervised Learning

It is a type of algorithm where features as well as targets are given, i.e. here we are provided with the domain of values under which the algorithm has to predict, or in other words, in this type of algorithm we have a partially defined algorithm to generate the fit,

This is the algorithm we employ, we say “speaking from experience” as maximum time we know what the outcome would be. We see humans employing this type of algorithm, we a court has to take a decision, as the result is documented in the country’s law book and based on that only a criminal is sentenced.

Unsupervised Learning

It is a type of learning, where unlike supervised learning, no labels are provided, the task of forming the “fit” is solely rested upon the AI. They are quite useful in providing humans with insights into the meaning of data and new useful inputs to supervised machine learning algorithms.

This algorithm resembles the method we employ to classify an object that we are seeing for the first time, or the logic we use to react in each situation.

Reinforcement Learning

It is a type of learning, where the labels are not given but the training is done based on positive and negative feedback i.e. for each, false-positive outcome system gets negative feedback and for a true positive, the system gets positive feedback.

Errors help you learn because they have a penalty added (cost, loss of time, regret, pain, and so on), teaching you that a certain course of action is less likely to succeed than others.

Example of this type can be Google Photos, where we are shown two pictures and we are asked to decide whether they are similar or not, each time we press the tick sign, the AI gets one positive point. And it gets a negative point each time we press cross sign.

Types of Reinforcement Learning

There are two types of Reinforcement:

Positive

Positive Reinforcement is defined as when an event occurs due to a particular behavior, increases the strength and the frequency of the behavior. In other words, it has a positive effect on behavior.

Advantages of reinforcement learning are:

- Maximizes Performance
- Sustain Change for a long period of time

Disadvantages of reinforcement learning:

- Too much Reinforcement can lead to an overload of states which can diminish the results

Negative

Negative Reinforcement is defined as the strengthening of behavior because a negative condition is stopped or avoided.

Advantages of reinforcement learning:

- Increases Behavior
- Provide defiance to a minimum standard of performance

Disadvantages of reinforcement learning:

- It Only provides enough to meet up the minimum behavior

Semi-Supervised Learning

According to Wikipedia, Semi-supervised learning is a class of machine learning tasks and techniques that also make use of unlabeled data for training – typically a small amount of labeled data with a large amount of unlabeled data. Semi-supervised learning falls between unsupervised learning and supervised learning.

Practical Applications of Semi-Supervised Learning

- **Protein Sequence Classification:** Inferring the function of proteins typically requires active human intervention.

- **Web Content Classification:** Organizing the knowledge available in billions of web pages will advance different segments of AI. Unfortunately, that task typically requires human intervention to classify the content.

Application Of Machine Learning

- Augmentation
- Automation
- Finance Industry
- Government organization
- Healthcare industry
- Marketing

Transfer Learning

Transfer learning is a machine learning method where we employ the OOPs concept of re-usability. Here, we take a pre-developed pre-tested model and changes its use cases to fit out application. For Example, MobileNet SSD finds its major implementation in the field of image processing and computer vision, so the MobileNet SSD based Human-Poster Detection model can be used for detecting whether a driver is drunk or not, the same model can be applied in detecting if a person is sad or happy.

Two common approaches are as follows:

- Develop a Model Approach
- Pre-trained Model Approach

Types of machine learning algorithms

- Regression
- Clustering
- Decision Tree
- Association Learning Rule
- Artificial Neural Network
- Induction Logic Programming
- Support Vector Machine
- Similarity and Metric Learning
- Bayesian Networks
- Representation Learning
- Sparse Matrix Learning

Conclusion

In this chapter, we studied machine learning, how do we do machine learning, basic terms, what can be called data? how to convert data into information? how to convert information into knowledge? advantages/features of ML, disadvantages/shortcomings of ML, data science process, types of machine learning, applications of ML, transfer learning and types of ML algorithms. Hope you were able to understand each and everything.

3

Data Science

Overview

In this chapter, we delve into the foundations of data science, exploring key concepts like data types, data science processes, and practical Python implementations. Prepare for a comprehensive journey through the essentials of data science and its applications.

What is data?

According to Wikipedia, Data are individual units of information. A datum describes a single quality or quantity of some object or phenomenon. In analytical processes, data are represented by variables. Although the terms "data", "information" and "knowledge" are often used interchangeably, each of these terms has a distinct meaning.

Data is measured, collected and reported, and analyzed, whereupon it can be visualized using graphs, images or other analysis tools. Data as a general concept refers to the fact that some existing information or knowledge is represented or coded in some form suitable for better usage or processing.

Everything and anything can be called data. Data is an abstract term, which can be used for every type of data. The amount of data that is generated daily can be estimated by seeing the following figures:

- **Google:** processed 24 Peta Bytes of data every data per day
- **Facebook:** 10 million photos uploaded every hour
- **YouTube:** 1 hour of video uploaded every second
- **Twitter:** 400 million tweets per day
- **Astronomy:** Satellite Data is in hundreds of petabytes

Note: 1 petabyte is 1 million megabytes or 1 quadrillion byte or 1000 trillion bytes or 1e-15 bytes

So, from the above values, you can very easily understand that daily we produce around 1 GB or Gigabyte of Data. Even the above-written figures are also a type of data.

What is data science?

According to Wikipedia, Data science is a "concept to unify statistics, data analysis, machine learning, and their related methods" to "understand and analyse actual phenomena" with data. It employs techniques and theories drawn from many fields within the context of mathematics, statistics, computer science, and information science.

Turing award winner Jim Gray imagined data science as a "fourth paradigm" of science (empirical, theoretical, computational and now data-driven) and asserted that "everything about science is changing because of the impact of information technology" and the data deluge. In 2015, the American Statistical Association identified database management, statistics, and machine learning, and distributed and parallel systems as the three emerging foundational professional communities.

What is data munging?

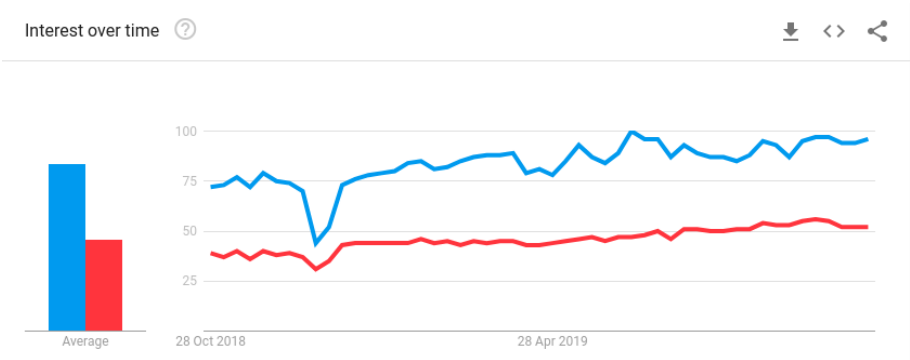
Data wrangling sometimes referred to as data munging, is the process of transforming and mapping data from one "raw" data form into another format with the intent of making it more appropriate and valuable for a variety of downstream purposes such as analytics. A data wrangler is a person who performs these transformation operations.

Difference between Business Intelligence and Data Science

Features	Business Intelligence (BI)	Data Science
Data Sources	Structured (Usually SQL, often Data Warehouse)	Both Structured and Unstructured (logs, cloud data, SQL, NoSQL, text)
Approach	Statistics and Visualization	Statistics, Machine Learning, Graph Analysis, Neuro-linguistic Programming (NLP)
Focus	Past and Present	Present and Future
Tools	Pentaho, Microsoft BI, QlikView, R	RapidMiner, BigML, Weka, R
Coining of Term	Business Analytics has been used since the late 19th Century when it was put in place by Frederick Winslow Taylor	DJ Patil and Jeff Hammerbacher who were working in LinkedIn and Facebook respectively, first coined the term in 2008
Concept	Use of statistical concepts to extract insights from business data	The interdisciplinary field of data inference, algorithm building, and systems to gain insights from data
Top 5 Industries	1. Financial 2. Technology 3. Mix of fields 4. CRM/Marketing 5. Retail	1. Technology 2. Financial 3. Mix of fields 4. Internet-based 5. Academic
Coding	It does not involve much coding. More statistics oriented	Coding is used widely. The field is a combination of traditional analytics practices with sound knowledge of computer science
Language Recommendations	C/C++/C#, JAVA, MATLAB, PYTHON, R, SAS, SCALA, SQL	C/C++/C#, HASKELL, JAVA, MATLAB, PYTHON, R, SAS, SCALA, SQL, JULIA, STATA
Statistics	The whole analysis is based on statistical concepts	Statistics is used at the end of analysis following algorithm building and coding
Data Needed	Predominantly structured data	Both structured and unstructured data

The following image shows a comparison between the popularity of Business Intelligence and Data Science.

Legend:
Blue: Business Intelligence
Red: Data Science



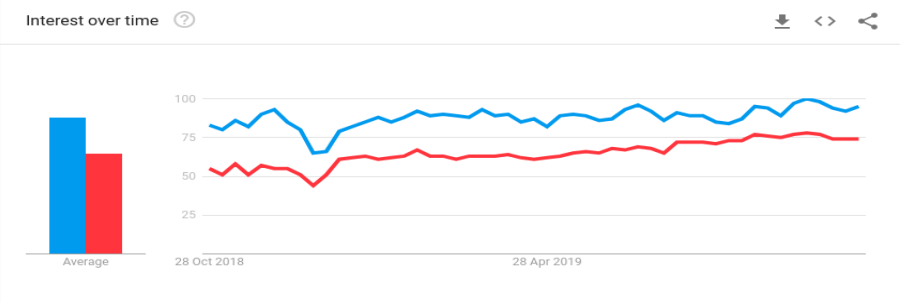
Source: Google Trends

Difference between Machine Learning and Data Science

Features	Machine Learning	Data Science
Scope	Accurately classify or predict the outcome for new data point by learning patterns from historical data using mathematical models	Create insights from data dealing with all real-world complexities. This includes tasks like understand requirements extracting data, etc.
Input Data	Input data for ML will be transformed specifically for algorithms used. Feature scaling, word embedding or adding polynomial features are some examples	Most of the input data is generated as human consumable data which is to be read or analyzed by humans like tabular data or images
System Complexity	1. Major complexity is with algorithms and mathematical concepts behind that 2. Ensemble models will have more than one ML model, and each will have a weighted contribution on the final output	1. Components for handling unstructured raw data coming 2. A lot of moving components typically scheduled by an orchestration layer to synchronize independent jobs
Preferred Skill Set	1. Strong Mathematics understanding 2. Python/R programming 3. Data wrangling with SQL Model specific visualization	1. Domain expertise 2. ETL and data profiling 3. String SQL 4. NoSQL system standard reporting/visualization
Hardware Specification	1. GPUs are preferred for intensive vector operations 2. More powerful versions like TPUs are on the way	1. Horizontally scalable systems preferred to handle massive data 2. High RAM and SSDs used to overcome I/O bottleneck

The following image shows a comparison between the popularity of Machine Learning and Data Science.

Legend:
Blue: Machine Learning
Red: Data Science



Source: Google Trends

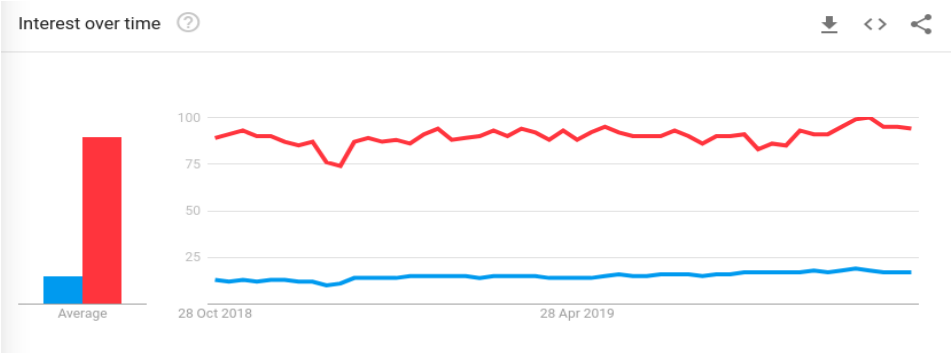
Difference between Data Science and Artificial Intelligence

Factors	Data Science	Artificial Intelligence
Scope	Involves various underlying data operations	Limited to the implementation of ML algorithms
Type of Data	Structured and Unstructured	Standardized in the form of embeddings and vectors

Tools	R, Python, SAS, SPSS, TensorFlow, Keras, Scikit- Learn	Scikit-Learn, Kaffe, PyTorch, TensorFlow, Shogun, Mahout
Applications	Advertising, Marketing, Internet Search Engines	Manufacturing, Automation, Robotics, Transport, Healthcare

The following image shows a comparison between the popularity of Artificial Intelligence and Data Science.

Legend:
Blue: Data Science
Red: Artificial Intelligence



Source: Google Trends

Advantages/Features of Data Science

- It detects and corrects the errors from data sets with the help of data cleansing. This helps in improving the quality of data and consecutively benefits both customers and institutions such as banks, insurance, and finance companies.
- It removes duplicate information from data sets and hence saves a large amount of memory space. This decreases the costs to the company.
- It helps in displaying relevant advertisements on the online shopping websites based on historic data and purchase behavior of the users. Machine learning algorithms are applied to the same. This helps in increasing the revenue and productivity of the companies.
- It reduces banking risks by identifying probable fraudulent customers based on historic data analysis. This helps institutes in deciding whether to issue loans or credit cards to the applicants or not.
- It is used by security agencies for surveillance and monitoring purposes based on information collected by a huge number of sensors. This helps in preventing any wrongdoings and/or calamities.

Disadvantages/Shortcomings of Data Science

- This may breach the privacy of the customers as their information such as purchases, online transactions, subscriptions are visible to their parent companies. The companies may exchange these useful customer databases for their mutual benefit.
- The cost of data analytics tools varies based on applications and features supported. Moreover, some of the data analytics tools are complex to use and require training. These increases cost the company willing to adopt data analytics tools or software.
- The information obtained using data analytics can also be misused against a group of people in certain countries or communities or castes.

- It is very difficult to select the right data analytics tools. This is due to the fact that it requires knowledge of the tools and their accuracy in analyzing the relevant data as per applications. This increases the time and cost to the company.

What are the skills required to be a Data Scientist?

A Data Scientist is a professional with the capabilities to gather large amounts of data to analyze and synthesize the information into actionable plans for companies and other organizations. Following are some requirements that need to be met to become a good Data Scientist:

- Mathematics Expertise
- Technology and Hacking
- Strong Business Acumen

In short, the skills required to be a data scientist are:

- Statistics
- Programming skills
- Critical thinking
- Knowledge of AI, ML, and Deep Learning
- Comfort with math
- Good Knowledge of Python, R, SAS, and Scala
- Communication
- Data Wrangling
- Data Visualization
- Ability to understand analytical functions
- Experience with SQL
- Ability to work with unstructured data

Data Science Components

Now, in this 'What is Data Science?' blog, we will discuss some of the key components of Data Science, which are

Data (And its Various Types)

The raw dataset is the foundation of Data Science, and it can be of various types like structured data (mostly in a tabular form) and unstructured data (images, videos, emails, PDF files, etc.)

Programming (Python And R)

Data management and analysis are done by computer programming. In Data Science, two programming languages are most popular: Python and R.

Statistics And Probability

Data is manipulated to extract information out of it. The mathematical foundation of Data Science is statistics and probability. Without having a clear knowledge of statistics and probability, there is a high possibility of misinterpreting data and reaching incorrect conclusions. That's the reason why statistics and probability play a crucial role in Data Science.

Machine Learning

As a Data Scientist, every day, you will be using Machine Learning algorithms such as regression and classification methods. It is very important for a Data Scientist to know Machine learning as a part of their job so that they can predict valuable insights from available data.

Big Data

In the current world, raw data is compared with crude oil, and the way we extract refined oil from the crude oil, by applying Data Science, we can extract different kinds of information from raw data. Different tools used by Data Scientists to process big data are Java, Hadoop, R, Pig, Apache Spark, etc.

What is Data Science Process?

Data Science Process is the lifecycle of Data Science. It consists of a chronological set of steps. This process is distributed in 6 subparts as:

- Phase 1—Discovery
- Phase 2—Data preparation
- Phase 3- Model Planning
- Phase 4—Model building
- Phase 5-Operationalize
- Phase 6-Communicate Results

Common tools for Model Building

- SAS Enterprise Miner
- WEKA
- SPCS Modeler
- MATLAB
- Alpine Miner
- Statistica

Data Science python implementation

By now you all have gained a lot of theoretical knowledge about the concept. Now let's see how we implement the concept using python.

Here, I am using titanic data, you can download titanic.csv from Kaggle. I am using Google Colab for this, you can use Jupyter Notebook or any other tool for this.

Loading Data to Google Colab

To know about Google Colab please click.

There are three ways of uploading a dataset to Google Colab, the following is the way I think is simple, you guys can search and use the other 2 ways.

I use Google Colab, because of the following reasons:

- As unlike Jupyter Notebook, I need not install any libraries
- being a web application, the processing speed is high (as we get the advantage of using GPUs and TPUs)
- Also, we can perform Machine Learning on systems with low RAM and system capabilities (I think the most important of all)

```
from google.colab import files
uploaded = files.upload()
```

Click upload and select the file you want to upload; in my case it is "titanic.csv". After that, I will start uploading automatically.

Preparing the Notebook (Google Colab)

Here, we will be setting up the environment of the notebook/Google Colab. And then we will be importing all the required python libraries.

```
from IPython.core.display import HTML
HTML("""
<style>
.output_png {
    display: table-cell;
    text-align: center;
    vertical-align: middle;
}
</style>
""");
%matplotlib inline

import warnings
warnings.filterwarnings('ignore')
warnings.filterwarnings('ignore', category=DeprecationWarning)

import pandas as pd
pd.options.display.max_columns = 100

from matplotlib import pyplot as plt
import numpy as np

import seaborn as sns

import pylab as plot
params = {
    'axes.labelsize': "large",
    'xtick.labelsize': 'x-large',
    'legend.fontsize': 20,
    'figure.dpi': 150,
    'figure.figsize': [25, 7]
}
plot.rcParams.update(params)
```

Loading The Dataset

We will be using pandas to load the dataset.

```
data = pd.read_csv('titanic.csv')
```

Print The Shape or Size of The Data

```
data.shape
```

Output

Since I am using a CSV file of 891 rows and 12 columns so I will be getting the output as (891,12)

Print the top 5 rows of the Data

```
print(data.head())
```

Output

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

To get a High-Level description of the Dataset

```
data.describe()
```

Output

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

The above output gives us the impression that 177 is missing from the age column, as the count value for age does not match with count value of other columns. so, to add that we do the following:

```
data['Age'] = data['Age'].fillna(data['Age'].median())
```

Now if we do data.describe(), we get the following output :

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.361582	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	13.019697	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	22.000000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	35.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

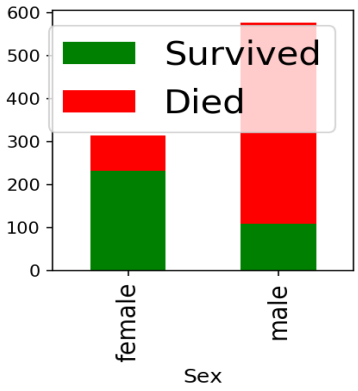
Data Visualization

You may like to read Python MatPlotLib and Python Seaborn chapter. Under this topic, we will visualize and try to understand the dataset.

Visualization of Sex and Survival Chances

```
data['Died'] = 1 - data['Survived']
data.groupby('Sex').agg('sum')[['Survived', 'Died']].plot(kind='bar',
figsize=(3, 3),
stacked=True, colors=['g', 'r']);
```

Output



We can visualize the above figure in terms of ratios as

```
data.groupby('Sex').agg('mean')[['Survived', 'Died']].plot(kind='bar',
figsize=(3, 3),
stacked=True, colors=['g', 'r']);
```

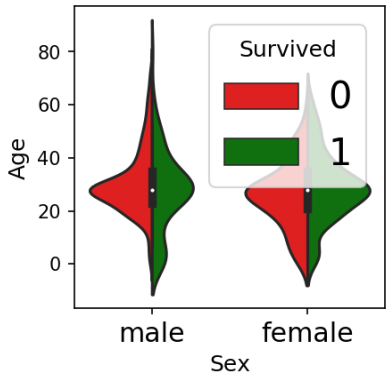
Output



Visualization of the correlation of age variable with men and women

```
fig = plt.figure(figsize=(3, 3))
sns.violinplot(x='Sex', y='Age',
hue='Survived', data=data, split=True,
palette={0: "r", 1: "g"});
```

Output

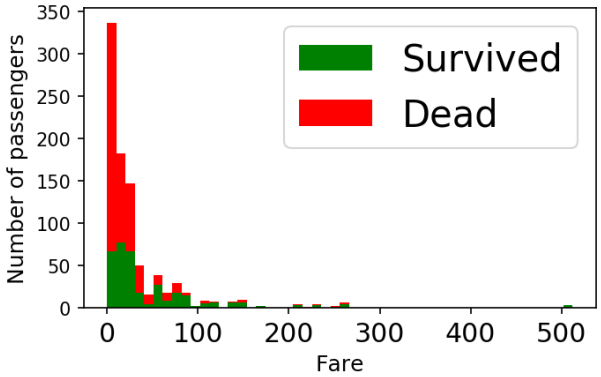


So, from the above, we inferred that woman survived more than men

Visualization of the relationship between fare ticket and their survival chances

```
figure = plt.figure(figsize=(5, 3))
plt.hist([data[data['Survived'] == 1]['Fare'], data[data['Survived'] ==
0]['Fare']],
         stacked=True, color = ['g','r'],
         bins = 50, label = ['Survived','Dead'])
plt.xlabel('Fare')
plt.ylabel('Number of passengers')
plt.legend();
```

Output

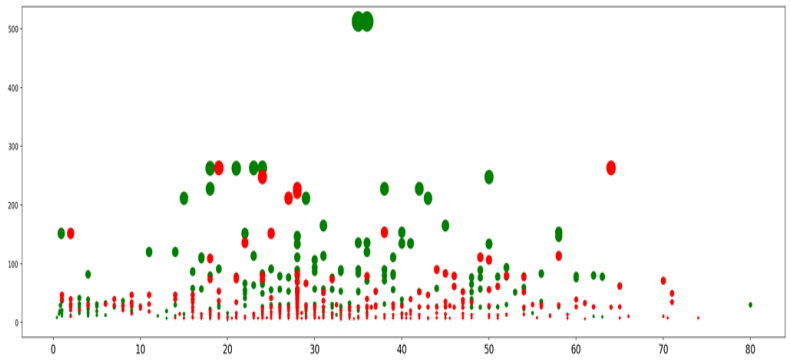


Visualization of the relationship between fare ticket, age, and their survival chances

```
plt.figure(figsize=(25, 7))
ax = plt.subplot()

ax.scatter(data[data['Survived'] == 1]['Age'], data[data['Survived'] ==
1]['Fare'],
           c='green', s=data[data['Survived'] == 1]['Fare'])
ax.scatter(data[data['Survived'] == 0]['Age'], data[data['Survived'] ==
0]['Fare'],
           c='red', s=data[data['Survived'] == 0]['Fare']);
```

Output



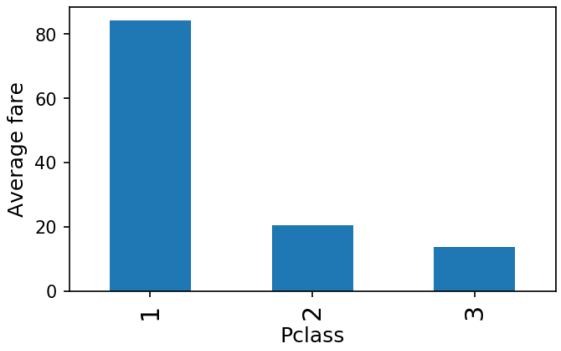
We can observe different clusters:

- Large green dots between x=20 and x=45: adults with the largest ticket fares
- Small red dots between x=10 and x=45, adults from lower classes on the boat
- Small greed dots between x=0 and x=7: these are the children that were saved

Visualization of the relationship between fare ticket and class

```
ax = plt.subplot()
ax.set_ylabel('Average fare')
data.groupby('Pclass').mean()['Fare'].plot(kind='bar', figsize=(5, 3),
ax = ax);
```

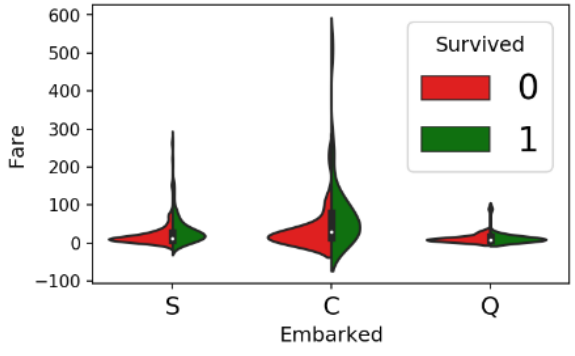
Output



Visualization of the relationship between fare ticket, embarked, and their survival chances

```
fig = plt.figure(figsize=(5, 3))
sns.violinplot(x='Embarked', y='Fare', hue='Survived', data=data,
split=True, palette={0: "r", 1: "g"});
```

Output



From the above visualization, we can infer that the passengers who paid the highest or who were set in C, survived the most

Data Wrangling Python implementation

Here we will be cleaning the data to make it ready for a machine-learning algorithm to work on it, i.e. will fill missing values, see if the data is continuous or not and see if data need any modifications to be done?

```
data.describe()
```

Output

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

From the above data, we get the impression that the minimum age of the passengers is 0.42 i.e. 5 months. This is very suspicious as according to the news, we had a two-month-old baby on board, i.e. the minimum age should be 0.19.
Also, in the "fare" field, we find some of the values to be (0,0). which either means that those passengers traveled free, or the data is missing.
So, let us first check whether any of the cells in the data is empty or not?

```
data.count()
```

Output

```
PassengerId    891
Survived       891
Pclass         891
Name           891
Sex            891
Age            714
SibSp          891
Parch          891
Ticket         891
Fare           891
Cabin          204
Embarked       889
dtype: int64
```

So, from the above, output it is crystal clear that we have some data missing or the corresponding cells are empty in the "embarked", "age" and "cabin" columns.

So, what to do now. Either we must delete the columns with missing data, or we did have to fill the missing places. Now, I will be telling you how I reduce the number of missing values.

There are three ways to fill the missing value

Replace each cell with value '0' with numpy.NaN

```
data = data.replace(0, np.NaN)
print(data.isnull().sum())
```

Output

```
PassengerId      0
Survived       549
Pclass           0
Name             0
Sex              0
Age              0
SibSp          608
Parch          678
Ticket           0
Fare            15
Cabin          687
Embarked         2
dtype: int64
```

So, we can see that to some extent we were able to clean the data, but some of the data got corrupted. So, let's try another way and see the result

Replace each missing value by the Column Mean

```
data.fillna(data.mean(), inplace=True)
# count the number of NaN values in each column
print(data.isnull().sum())
```

Output

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

Here, we see that we were able to rectify "age" and nothing got corrupted. But we were not able to rectify "cabin" and "embarked". So now let's try another way.

Replacing each numpy.NaN value with zero

```
data = data.replace(np.NaN, 0)
# count the number of NaN values in each column
print(data.isnull().sum())
```

Output

```
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked       0
dtype: int64
```

So, by using this I was able to achieve a "no null" situation.

Note: But replacing missing values I had achieved but it is quite possible that it may result in loss of data or may change the meaning of data.

Conclusion

In this chapter, we studied what is data, what is data science, what is data mugging, difference between business intelligence and data science, difference between machine learning and data science, difference between artificial intelligence and data science, advantages of data science, disadvantages of data science, what are the skills required to be a data scientist, data science components, what is data science process and python implementation of some of the data science concepts . Hope you were able to understand each and everything.

4

Linear Regression

Overview

In this chapter, we explore the concept of regression, examining its various types and applications. You'll gain insights into linear, logistic, and polynomial regression, among others. We'll also delve into the advantages and limitations of linear regression, key terms, and techniques such as gradient descent and the coefficient of determination. This comprehensive overview will equip you with the knowledge to understand and apply regression techniques effectively.

What is Regression and Why is it called so?

Regression is an ML algorithm that can be trained to predict really numbered outputs, like temperature, stock price, etc. Regression is based on a hypothesis that can be linear, quadratic, polynomial, non-linear, etc.

Advantages/Features of Linear Regression

- Linear regression is an extremely simple method.
- It is very easy and intuitive to use and understand.
- A person with only the knowledge of high school mathematics can understand and use it.
- In addition, it works in most cases. Even when it doesn't fit the data exactly, we can use it to find the nature of the relationship between the two variables.

Disadvantages/Shortcomings of Linear Regression

- By its definition, linear regression only models' relationships between dependent and independent variables that are linear. It assumes there is a straight-line relationship between them which is incorrect sometimes. Linear regression is very sensitive to the anomalies in the data (or outliers).
- Take for example most of your data lies in the range 0-10. If due to any reason only one of the data items comes out of the range, say for example 15, this significantly influences the regression coefficients.
- Another disadvantage is that if we have several parameters than the number of samples available then the model starts to model the noise rather than the relationship between the variables

Types of Regression

1. Linear regression: The formula for linear regression is Y' = bX + A.

2. Logistic regression: The equation for logistic regression is

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2$$

3. Polynomial regression: The equation for polynomial regression is

$$\beta_0 + \beta_0 x_1 + \epsilon$$

4. Stepwise regression: The formula for stepwise regression is

$$b_{j.std} = b_j(s_x * s_y^{-1})$$

5. Ridge regression: The formula for ridge regression is

$$\beta = (X^T X + \lambda * I)^{-1} X^T y$$

6. Lasso regression: Lasso regression is: .

$$N^{-1} \sum_{i=1}^{N} f(x_i, y_I, \alpha, \beta)$$

7. ElasticNet regression: The penalty function is given by:

$$||\beta||_1 = \sum_{j=1}^{p} |\beta_j|$$

8. Support Vector regression

9. Decision Tree regression

10. Random Forest regression

Key terms

Estimator: A formula or algorithm for generating estimates of parameters, given relevant data.

Bias: An estimate is unbiased if its expectation equals the value of the parameter being estimated; otherwise, it is biased.

Efficiency: An estimator A is more efficient than an estimator B if A has a smaller sampling variance — that is, if the particular values generated by A are more tightly clustered around their expectation.

Consistency: An estimator is consistent if the estimates it produces converge on the true parameter value as the sample size increases without limit.

Standard error of the Regression (SER): An estimate of the standard deviation of the error term in a regression model.

R-squared: A standardized measure of the goodness of fit for a regression model.

Standard error of regression coefficient: An estimate of the standard deviation of the sampling distribution for the coefficient in question.

P-value: When a small p-value is found, the two possibilities are that we happened to draw a low-probability unrepresentative sample or that the null hypothesis is in fact false.

Significance level: For a hypothesis test, this is the smallest p-value for which we will not reject the null hypothesis.

T-test: The t-test (or z-test, which is the same thing asymptotically) is a common test for the null hypothesis that a particular regression parameter, βi, has some specific value (commonly zero, but generically βH0).

F-test: A common procedure for jointly testing a set of linear restrictions on a regression model.

Multicollinearity

A situation where there is a high degree of correlation among the independent variables in a regression model — or, more generally, where some of the Xs are close to being linear combinations of other Xs. Symptoms include large standard errors and the inability to produce precise parameter estimates.

Omitted variable bias

Bias in the estimation of regression parameters that arises when a relevant independent variable is omitted from a model and the omitted variable is correlated with one or more of the included variables.

Log variables

A key point to remember about logs is that for small changes, the change in the log of a variable is a good approximation to the proportional change in the variable itself. For example, if log(y) changes by 0.04, y changes by about 4%.

Interaction terms

Pairwise products of the "original" independent variables. The inclusion of interaction terms in a regression allows for the possibility that the degree to which xi affects y depends on the value of some other variable x j.

Independent Variable

An independent variable is a variable that is changed or controlled in a scientific experiment to test the effects on the dependent variable.

Dependent Variable

A dependent variable is a variable being tested and measured in a scientific experiment. The dependent variable is 'dependent' on the independent variable. As the experimenter changes the independent variable, the effect on the dependent variable is observed and recorded.

Regularization

These seek to both minimize the sum of the squared error of the model on the training data and also reduces the complexity of the model

Gradient Descent

Gradient descent is an optimization algorithm used to minimize some function by iteratively moving in the direction of steepest descent as defined by the negative of the gradient. In machine learning, we use gradient descent to update the parameters of our model. Parameters refer to coefficients in Linear Regression and weights in neural networks.

Cost Function

A Loss Function or Cost Function tells us “How good” our model is at making predictions for a given set of parameters. The cost function has its own curve and its own gradients. The slope of this curve tells us how to update our parameters to make the model more accurate.

Given the cost function:

$$f(m,b) = \frac{1}{N}\sum_{i=1}^{n}(y_i - (mx_i + b))^2$$

The gradient can be calculated as:

$$f'(m,b) = \begin{bmatrix} \frac{df}{dm} \\ \frac{df}{db} \end{bmatrix} = \begin{bmatrix} \frac{1}{N}\sum -2x_i(y_i - (mx_i + b)) \\ \frac{1}{N}\sum -2(y_i - (mx_i + b)) \end{bmatrix}$$

Python Implementation of Gradient Descent

```
def update_weights(m, b, X, Y, learning_rate):
    m_deriv = 0
    b_deriv = 0
    N = len(X)
    for i in range(N):
        # Calculate partial derivatives
        # -2x(y - (mx + b))
        m_deriv += -2*X[i] * (Y[i] - (m*X[i] + b))

        # -2(y - (mx + b))
        b_deriv += -2*(Y[i] - (m*X[i] + b))
```

```
    # We subtract because the derivatives point in direction of steepes
t ascent
    m -= (m_deriv / float(N)) * learning_rate
    b -= (b_deriv / float(N)) * learning_rate

    return m, b
```

Remembering Variables With DRYMIX

When results are plotted in graphs, the convention is to use the independent variable as the x-axis and the dependent variable as the y-axis. The DRYMIX acronym can help keep the variables straight:

- D is the dependent variable
- R is the responding variable
- Y is the axis on which the dependent or responding variable is graphed (the vertical axis)
- M is the manipulated variable or the one that is changed in an experiment
- I am the independent variable
- X is the axis on which the independent or manipulated variable is graphed (the horizontal axis)

Simple Linear Regression Model

The simple linear regression model is represented like this: **y = (β0 +β1 + E)**

The two factors involved in a simple linear regression analysis are designated by mathematical c onvention as x and y. The equation known as the regression model explains how y is connected to x. The linear regression model also includes an EU-represented error term or the Greek epsilon letter. The word “error” is used to compensate for the variation in y which the linear relationship between x and y can't explain. There are also parameters which reflect the studying population. These parameters of the model are represented by (β0+β1x).

The simple linear regression equation is graphed as a straight line.

The simple linear regression equation is represented like this: **E(y) = (β0 +β1*x)**

- β0 is the y-intercept of the regression line.
- β1 is the slope.
- E(y) is the mean or expected value of y for a given value of x.

A regression line can show a positive linear relationship, a negative linear relationship, or no relationship.

- When the graphed line is flat (not inclined), there is no connection between the two variables in the simple linear regression.
- A positive linear relationship exists when the regression line is pitted upward at the bottom end of the line at the y-intercept (axis) of the graph and the upper end of the line upward to the pole.
- When the regression line descends toward the upper part at the y-intercept (axis) of the graph and the lower part of the line descends to the bottom, a negative linear relationship exists toward the x-intercept (axis).

$$\hat{y} = \beta0 + \beta1*x$$

$$b_1 = \frac{SS_{xy}}{SS_{xx}}$$

$$b_0 = \bar{y} - b_1\bar{x}$$

$$SS_{xy} = \sum x_i y_i - \frac{\left(\sum x_i\right)\left(\sum y_i\right)}{n}$$

$$SS_{xx} = \sum x_i^2 - \frac{\left(\sum x_i\right)^2}{n} = (n-1)s_x^2$$

Important Note

- Regression analysis is not used to interpret cause-and-effect relationships (a mechanism where one event makes another event happen, i.e. each event is dependent on one-another) between variables. Regression analysis can, however, indicate how variables are related or to what extent variables are associated with each other.
- It is also known as bivariate regression or regression analysis

Sum of Square Errors

This is the sum of differences between the points and the regression line. It can serve as a measure of how well the line fits the data.

$$SSE = \sum_{i=1}^{n} (x_i - \bar{x})^2$$

Standard Estimate of Errors

The mean error is zero. If se (sigma epsilon) is small, the errors appear to be close to zero. The model then fits well with the results. We can therefore use a linear model as a measure of its suitability. Se (sigma epsilon) provides an estimator of se

$$\sigma_{est} = \sqrt{\frac{\sum (Y - Y')^2}{N}}$$

Coefficient of Determination

To measure the strength of the linear relationship we use the coefficient of determination. R^2 takes on any value between zero and one.

- R^2 = 1: Perfect match between the line and the data points.
- R^2 = 0: There is no linear relationship between x and y.

$$r^2 = 1 - \frac{\Sigma (y - y')^2}{\Sigma (y - \bar{y}')^2}$$

Simple Linear Regression Example

Let's take the example of Housing Price Prediction, the data that I am using is KC House Prices, you can download it from Kaggle. Feel free to use any dataset, there are some very good datasets available on kaggle and with Google Colab.

```
import pandas as pd

df = pd.read_csv("kc_house_data.csv")
df.describe()
```

Output

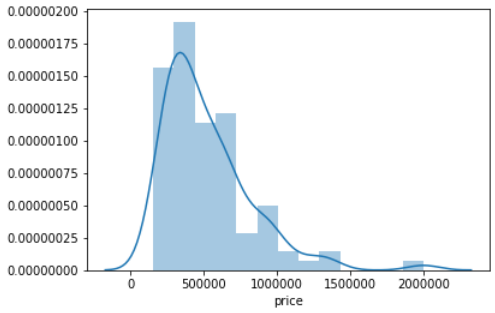
	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long	sqft_living15	sqft_lot15
count	2.161300e+04	2.161300e+04	21613.000000	21613.000000	21613.000000	2.161300e+04	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000
mean	4.580302e+09	5.400881e+05	3.370842	2.114757	2079.899736	1.510697e+04	1.494309	0.007542	0.234303	3.409430	7.656873	1788.390691	291.509045	1971.005136	84.402258	98077.939805	47.560053	-122.213896	1986.552492	12768.455652
std	2.876566e+09	3.671272e+05	0.930062	0.770163	918.440897	4.142051e+04	0.539989	0.086517	0.766318	0.650743	1.175459	828.090978	442.575043	29.373411	401.679240	53.505026	0.138564	0.140828	685.391304	27304.179631
min	1.000102e+06	7.500000e+04	0.000000	0.000000	290.000000	5.200000e+02	1.000000	0.000000	0.000000	1.000000	1.000000	290.000000	0.000000	1900.000000	0.000000	98001.000000	47.155900	-122.519000	399.000000	651.000000
25%	2.123049e+09	3.219500e+05	3.000000	1.750000	1427.000000	5.040000e+03	1.000000	0.000000	0.000000	3.000000	7.000000	1190.000000	0.000000	1951.000000	0.000000	98033.000000	47.471000	-122.328000	1490.000000	5100.000000
50%	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000000	7.618000e+03	1.500000	0.000000	0.000000	3.000000	7.000000	1560.000000	0.000000	1975.000000	0.000000	98065.000000	47.571800	-122.230000	1840.000000	7620.000000
75%	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000000	1.068800e+04	2.000000	0.000000	0.000000	4.000000	8.000000	2210.000000	560.000000	1997.000000	0.000000	98118.000000	47.678000	-122.125000	2360.000000	10083.000000
max	9.900000e+09	7.700000e+06	33.000000	8.000000	13540.000000	1.651359e+06	3.500000	1.000000	4.000000	5.000000	13.000000	9410.000000	4820.000000	2015.000000	2015.000000	98199.000000	47.777600	-121.315000	6210.000000	871200.000000

From the above data, we can see that we have a lot of tables, but simple linear regression can only process two columns, so we select the "price" and "sqrt_living". Here we will take the first 100 rows for our demonstration.

Before fitting the data let's analyze the data

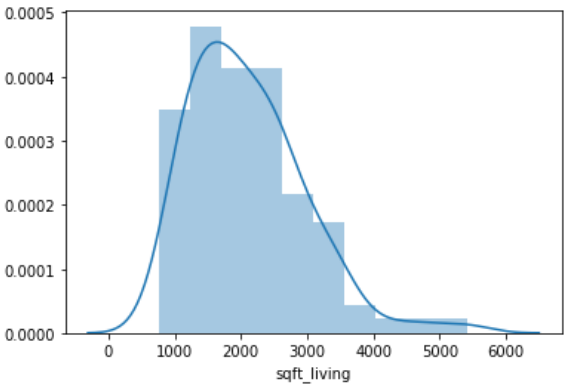
```
import seaborn as sns
df = pd.read_csv("kc_house_data.csv")
sns.distplot(df['price'][1:100])
```

Output




```
import seaborn as sns
df = pd.read_csv("kc_house_data.csv")
sns.distplot(df['sqrt_living'][1:100])
```

Output



Demo using Numpy

In the below code, we will just use numpy to perform linear regression.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline
def estimate_coef(x, y):

    # number of observations/points
    n = np.size(x)
    # mean of x and y vector
    m_x, m_y = np.mean(x), np.mean(y)

    # calculating cross-deviation and deviation about x
    SS_xy = np.sum(y*x) - n*m_y*m_x
    SS_xx = np.sum(x*x) - n*m_x*m_x

    # calculating regression coefficients
    b_1 = SS_xy / SS_xx
    b_0 = m_y - b_1*m_x

    return(b_0, b_1)
def plot_regression_line(x, y, b):
    # plotting the actual points as scatter plot
    plt.scatter(x, y, color = "m",
                   marker = "o", s = 30)

    # predicted response vector
    y_pred = b[0] + b[1]*x

    # plotting the regression line
    plt.plot(x, y_pred, color = "g")

    plt.xlabel('x')
    plt.ylabel('y')

    # function to show plot
    plt.show()
def main():

    # observations
  df = pd.read_csv("kc_house_data.csv")
  y= df['price'][1:100]
  x= df['sqft_living'][1:100]

  plt.scatter(x, y)
```

```
  plt.xlabel('x')
  plt.ylabel('y')

    # function to show plot
  plt.show()

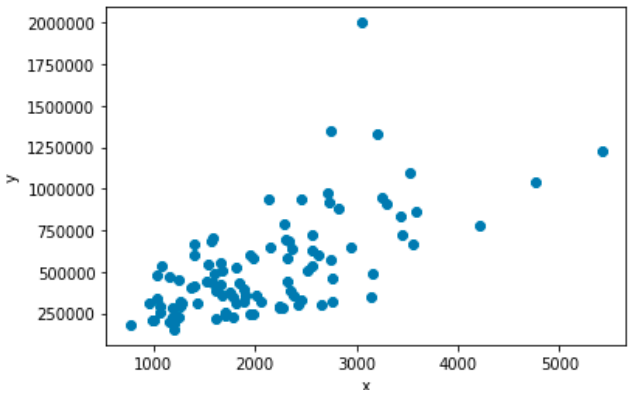
  b = estimate_coef(x, y)
  print("Estimated coefficients:\nb_0 = {}  \nb_1 = {}".format(b[0], b[
1]))

    # plotting regression line
  plot_regression_line(x, y, b)

if __name__ == "__main__":
    main()
```

Output

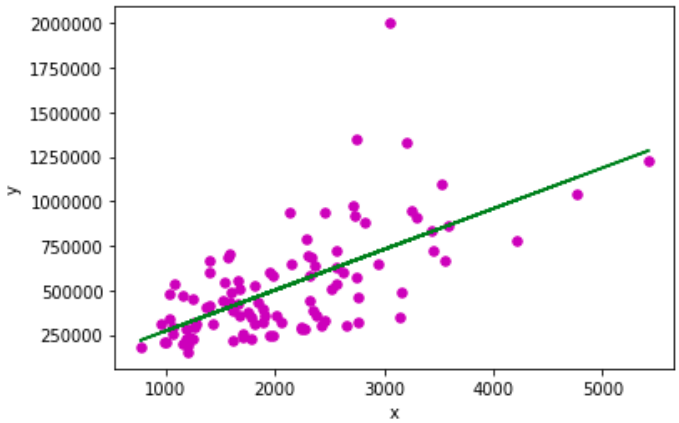
The input data can be visualized as:



After, executing the code we get the following output
Estimated coefficients: b_0 = 41517.979725295736 b_1 = 229.10249314945074

So, the Linear Regression equation becomes
[price] = 41517.979725795736+ 229.10249374945074*[sqrt_living]
i.e y = b [0] + b[1]*x

Let's see the final plot with the Regression Line



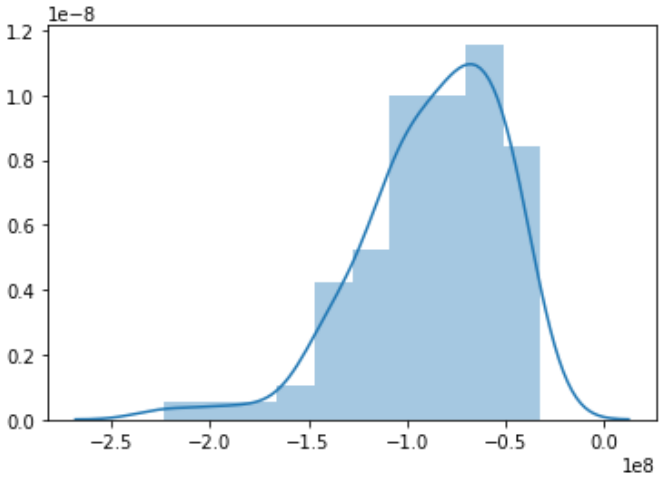
To predict the values, we run the following command

```
print("For x =100", "the predicted value would be",(b[1]*100+b[0]))
```

So, the predicted value that we get is 64428.22904024081

Since the machine must learn, hence would be doing some errors in predicting, so the let's see what percentage of error it is performing at each given point

```
y_diff = y -(b[1]+b[0]*x)
sns.distplot(y_diff)
```



Demo using Sklearn

In the below code, we will learn how to code linear regression using SkLearn

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
df = pd.read_csv("kc_house_data.csv")

# Select features and target variable
y = df['price'][1:100]
x = df['sqft_living'][1:100]

# Import train_test_split for splitting the data into train and test
sets
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.2, random_state=101)

# Import LinearRegression and create an instance of the Linear
Regression class
from sklearn.linear_model import LinearRegression
```

```
lr = LinearRegression()

# Reshape x_train as fit requires a matrix
x_train = x_train.values.reshape(-1,1)

# Fit the model to the training data
lr.fit(x_train, y_train)

# Print the coefficients
print("b[0] =", lr.intercept_)  # prints the intercept
print("b[1] =", lr.coef_)       # prints the slope

# Reshape x_test as predict requires a matrix
x_test = x_test.values.reshape(-1,1)

# Predict the target variable for the test data
pred = lr.predict(x_test)
plt.scatter(x_train, y_train)

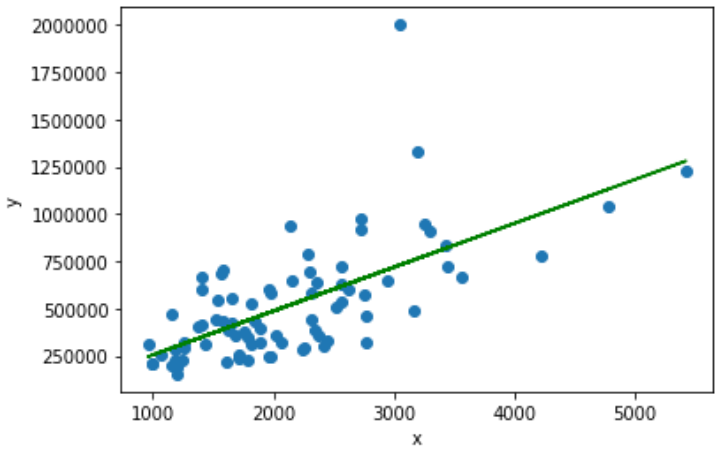
# Predicted response vector
y_pred = lr.intercept_ + lr.coef_ * x_train

plt.plot(x_train, y_pred, color="g")

# Add labels to the plot
plt.xlabel('Square Footage of Living Space')
plt.ylabel('Price')

# Show the plot
plt.show()
```

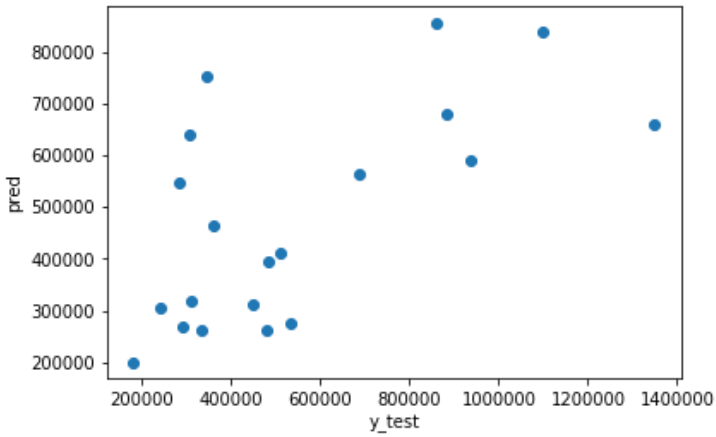
Output



After, executing the code we get the following output
b[0] = 21803.55365770642 b[1] = [232.07541739]

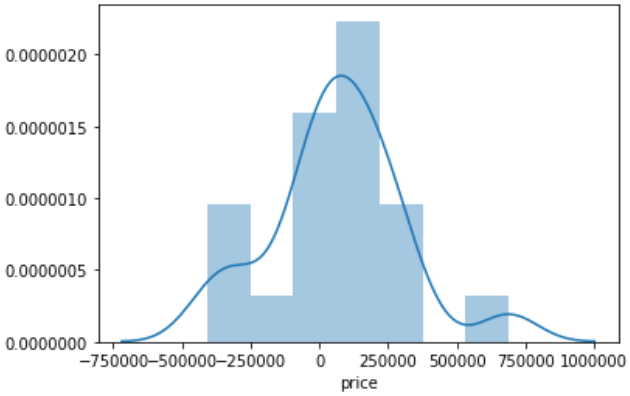
So, the Linear Regression equation becomes:
[price] = 21803.55365770642+ 232.07541739*[sqrt_living]
i.e y = b[0] + b[1]*x

Let's now see a graphical representation between predicted value wrt test set



Since the machine must learn, hence would be doing some errors in predicting, so the let's see what percentage of error it is performing at each given point

```
sns.distplot((y_test-pred))
```



Demo using TensorFlow

In this, I will not be using "kc_house_data.csv". To make it as simple as possible, I used some dummy data and then I will explain each and everything

```
np.random.seed(101)
tf.set_random_seed(101)
```

We first set the seed value for both NumPy and TensorFlow. The seed value is used for random number generation.

```
x = np.linspace(0, 50, 50)
y = np.linspace(0, 50, 50)
```

We now generate dummy data using the NumPy's linspace function, which generated 50 equally distributed points between 0 and 50

```
x += np.random.uniform(-4, 4, 50)
y += np.random.uniform(-4, 4, 50)
```

Since the data so generated is too perfect, so we add a little bit of uniformly distributed white noise.

```
n = len(x) # Number of data points
# Plot of Training Data
plt.scatter(x, y)
plt.xlabel('x')
plt.xlabel('y')
plt.title("Training Data")
plt.show()
```

Here, we thus plot the graphical representation of the dummy data.

```
X = tf.placeholder("float")
Y = tf.placeholder("float")
W = tf.Variable(np.random.randn(), name = "W")
b = tf.Variable(np.random.randn(), name = "b")
```

Here we are setting X and Y as the actual training data and the W and b as the trainable data, where:

- W means Weight
- b means bias
- X means the dependent variable
- Y means the independent variable

We must give initial weights and bias to the model. So, we initialize them with some random values.

```
learning_rate = 0.01
training_epochs = 1000
```

Here, we assume the learning rate to be 0.01 i.e. the gradient descent value would increase/decrease by 0.01 and we will train the model 1000 times or for 1000 epochs

```
# Hypothesis
y_pred = tf.add(tf.multiply(X, W), b)

# Mean Squared Error Cost Function
cost = tf.reduce_sum(tf.pow(y_pred-Y, 2)) / (2 * n)

# Gradient Descent Optimizer
optimizer = tf.train.GradientDescentOptimizer(learning_rate).minimize(c
ost)

# Global Variables Initializer
init = tf.global_variables_initializer()
```

Next thing, we must set the hypothesis, cost function, the optimizer (which will make sure that model is learning on every epoch) and the global variable initializer.

- Since we are using simple linear regression hence the hypothesis is set as:
- Depedent_Variable*Weight+Bias or Depedent_Variable*b[1]+b[0]
- We choose the cost function to be mean squared error cost function

- We choose the optimizer as Gradient Descent with the motive of minimizing the cost with given learning rate
- We use TensorFlow's global variable initialize the global variable so that we can re-use the value of variables

```
# Starting the Tensorflow Session
with tf.Session() as sess:

    # Initializing the Variables
    sess.run(init)

    # Iterating through all the epochs
    for epoch in range(training_epochs):

        # Feeding each data point into the optimizer using Feed Diction
ary
        for (_x, _y) in zip(x, y):
            sess.run(optimizer, feed_dict = {X : _x, Y : _y})

        # Displaying the result after every 50 epochs
        if (epoch + 1) % 50 == 0:
            # Calculating the cost a every epoch
            c = sess.run(cost, feed_dict = {X : x, Y : y})
            print("Epoch", (epoch + 1), ": cost =", c, "W =", sess.run(
W), "b =", sess.run(b))
```

- tf.Session()
  we start the TensorFlow session and name the Session as sess. We then initialize all the required variable using the
- sess.run(init)
  initialize all the required variable using 'init'
- for epoch in range(training_epochs)
  create a for loop that runs till ***epoch*** becomes equal to ***training_epocs***
- for (_x, _y) in zip (x,y)
  this is used to form a set of values from the given data and then parse through the so formed sets
- sess.run(optimizer, feed_dict = {X: _x, Y: _y})
  used to run the first optimizer iteration on each the data points
- if(epoch+1) %50==0
  to break the given iterations into a batch of 50 each
- c= sess.run (cost, feed_dict= {X: x, Y: y})
  performs the training on the given data points
- print ("Epoch", (epoch+1),": cost", c, "W =", sess.run(W), "b =", sess.run(b)
  print the values after each 50 epochs

```
#Storing necessary values to be used outside the Session
training_cost = sess.run(cost, feed_dict ={X: x, Y: y})
weight = sess.run(W)
bias = sess.run(b)
```

The above code is used to store the values for the next session

```
#Calculating the predictions
predictions = weight * x + bias
print("Training cost =", training_cost, "Weight =", weight, "bias =", b
ias, '\n')
# Plotting the Results
plt.plot(x, y, 'ro', label ='Original data')
plt.plot(x, predictions, label ='Fitted line')
plt.title('Linear Regression Result')
plt.legend()
plt.show()
```

The above code demonstrates how we can use the trained model to predict values

LR_TensorFlow.py

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# Set random seeds for reproducibility
np.random.seed(101)
tf.set_random_seed(101)
# Generating random linear data
# There will be 50 data points ranging from 0 to 50
x = np.linspace(0, 50, 50)
y = np.linspace(0, 50, 50)

# Adding noise to the random linear data
x += np.random.uniform(-4, 4, 50)
y += np.random.uniform(-4, 4, 50)

n = len(x)  # Number of data points

# Plot of Training Data
plt.scatter(x, y)
plt.xlabel('x')
plt.ylabel('y')
plt.title("Training Data")
plt.show()

# TensorFlow placeholders for inputs and outputs
X = tf.placeholder("float")
Y = tf.placeholder("float")

# TensorFlow variables for weights and bias
W = tf.Variable(np.random.randn(), name="W")
b = tf.Variable(np.random.randn(), name="b")
```

```
learning_rate = 0.01
training_epochs = 1000

# Hypothesis
y_pred = tf.add(tf.multiply(X, W), b)

# Mean Squared Error Cost Function
cost = tf.reduce_sum(tf.pow(y_pred - Y, 2)) / (2 * n)

# Gradient Descent Optimizer
optimizer =
tf.train.GradientDescentOptimizer(learning_rate).minimize(cost)

# Global Variables Initializer
init = tf.global_variables_initializer()

# Starting the TensorFlow Session
with tf.Session() as sess:
    # Initializing the Variables
    sess.run(init)

    # Iterating through all the epochs
    for epoch in range(training_epochs):
        # Feeding each data point into the optimizer using Feed
Dictionary
        for (_x, _y) in zip(x, y):
            sess.run(optimizer, feed_dict={X: _x, Y: _y})

        # Displaying the result after every 50 epochs
        if (epoch + 1) % 50 == 0:
            # Calculating the cost at every epoch
            c = sess.run(cost, feed_dict={X: x, Y: y})
            print("Epoch", (epoch + 1), ": cost =", c, "W =",
sess.run(W), "b =", sess.run(b))

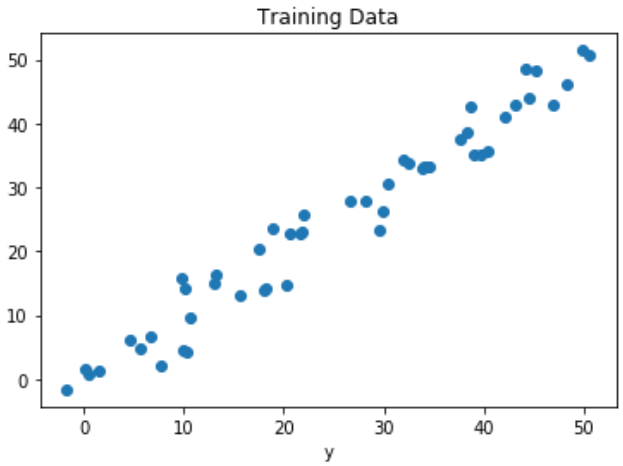
    # Storing necessary values to be used outside the Session
    training_cost = sess.run(cost, feed_dict={X: x, Y: y})
    weight = sess.run(W)
    bias = sess.run(b)

# Calculating the predictions
predictions = weight * x + bias
print("Training cost =", training_cost, "Weight =", weight, "bias =",
bias, '\n')

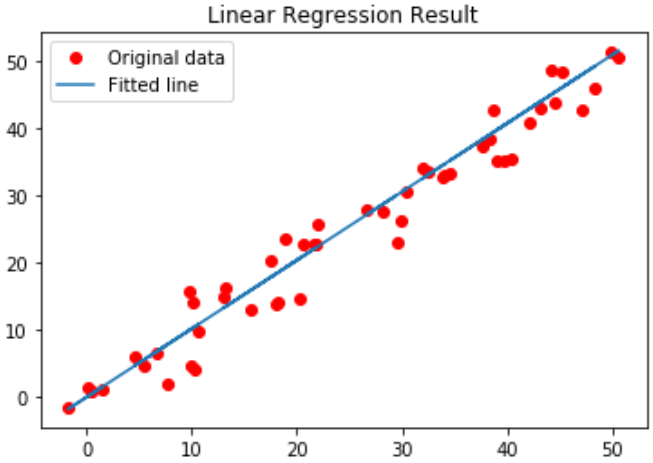
# Plotting the Results
plt.plot(x, y, 'ro', label='Original data')
plt.plot(x, predictions, label='Fitted line')
```

```
plt.title('Linear Regression Result')
plt.legend()
plt.show()
```

Output




```
Epoch 50 : cost = 5.886804 W = 0.9951241 b = 1.2381054
Epoch 100 : cost = 5.7912707 W = 0.99812365 b = 1.0914398
Epoch 150 : cost = 5.7119684 W = 1.0008028 b = 0.96044314
Epoch 200 : cost = 5.6459413 W = 1.0031956 b = 0.8434396
Epoch 250 : cost = 5.5907993 W = 1.0053328 b = 0.73893565
Epoch 300 : cost = 5.544605 W = 1.0072418 b = 0.6455924
Epoch 350 : cost = 5.5057893 W = 1.008947 b = 0.56222016
Epoch 400 : cost = 5.473066 W = 1.01047 b = 0.48775372
Epoch 450 : cost = 5.445385 W = 1.0118302 b = 0.42124188
Epoch 500 : cost = 5.4219036 W = 1.0130452 b = 0.36183503
Epoch 550 : cost = 5.4019217 W = 1.0141305 b = 0.3087744
Epoch 600 : cost = 5.3848577 W = 1.0150996 b = 0.26138142
Epoch 650 : cost = 5.370246 W = 1.0159653 b = 0.21905117
Epoch 700 : cost = 5.3576994 W = 1.0167387 b = 0.18124233
Epoch 750 : cost = 5.346891 W = 1.0174292 b = 0.14747268
Epoch 800 : cost = 5.337558 W = 1.0180461 b = 0.11730959
Epoch 850 : cost = 5.3294754 W = 1.0185971 b = 0.09036876
Epoch 900 : cost = 5.322459 W = 1.0190892 b = 0.06630599
Epoch 950 : cost = 5.3163586 W = 1.0195289 b = 0.044813465
Epoch 1000 : cost = 5.3110337 W = 1.0199214 b = 0.02561676
Training cost = 5.3110337 Weight = 1.0199214 bias = 0.02561676
```



After, executing the code we get the following output
b[0] = 1.0199214 b[1] = 0.02561676

So, the Linear Regression equation becomes
[y] = 1.0199214+ 1.0199214*[x]
i.e. y = b[0] + b[1]*x

Conclusion

In this chapter, we studied what is regression and what is it called so, advantages of linear regression, disadvantages of linear regression types of regression, key terms, gradient descent, remembering variables with DRYMIX, simple linear regression model, sum of square errors, standard error of estimate, coefficient of determination, simple linear regression example and python implementation of simple linear regression using numpy, sklearn and TensorFlow. Hope you were able to understand each and everything.

5

Logistic Regression

Overview

In this chapter, we'll delve into the world of generics in C#. We'll start by understanding the fundamental concepts and syntax associated with generics. By the end, you'll be equipped with the knowledge to leverage generics effectively, enhancing the flexibility and reusability of your code. Get ready for an insightful exploration that will boost your programming skills and improve your C# projects.

What is Logistic Regression?

Logistic regression is an appropriate regression analysis to be performed when the dependent variable is dichotomous (binary). Logistic regression is a statistical model, like all regression analyses. Logistic regression is used to characterize data and to illustrate the relationship between one dependent binary variable and one or more independent variables at mean, ordinal, interval, or ratio level. The probability of classification problems with two potential outcomes is explained by logistic regression. It is an extension of the classification problem linear regression model.

What is it used for?

Logistic regression is used to describe data and to explain the relationship between one dependent binary variable and one or more nominal, ordinal, interval or ratio-level independent variables.

Difference between Linear and Logistic Regression

BASIS FOR COMPARISON	LINEAR REGRESSION	LOGISTIC REGRESSION
Basic	The data is modeled using a straight line.	The probability of some obtained event is represented as a linear function of a combination of predictor variables.
Linear relationship between the dependent and independent variables	Is required	Not required
The independent variable	Could be correlated with each other. (Especially in multiple linear regression)	Should not be correlated with each other (no multicollinearity exists).
Outcome	the outcome (dependent variable) is continuous. It can have any one of an infinite number of possible values.	the outcome (dependent variable) has only a limited number of possible values.
Dependent Variable	Linear regression is used when your response variable is continuous. For instance, weight, height, number of hours, etc.	Logistic regression is used when the response variable is categorical in nature. For instance, yes/no, true/false, red/green/blue, 1st/2nd/3rd/4th, etc.
Equation	Linear regression gives an equation that is of the form Y = MX + C, which means equation with degree 1.	logistic regression gives an equation which is of the form Y = $e^X + e^{-X}$
Coefficient Interpretation	Coefficient definition of independent variables is very straightforward (i.e. keeping all other variables unchanged, the dependent variable is supposed to raise / decrease by xxx as a unit change in that variable).	depends on the family (binomial, Poisson, etc.) and link (log, logit, inverse-log, etc.) you use, the interpretation is different.
Error Minimization Technique	uses ordinary least squares method to minimize the errors and arrive at a best possible fit	uses a maximum likelihood method to arrive at the solution.

Why is Logistic Regression called so?

The meaning of the term regression is very simple: any process that attempts to find relationships between variables is called regression. Logistic regression is regression because it finds relationships between variables. It is logistic because it uses a logistic function as a link function. Hence the full name.

What is the goal of Logistic Regression?

The goal of logistic regression is to correctly predict the category of outcome for individual cases using the most parsimonious model. To accomplish this goal, a model is created that includes all predictor variables that are useful in predicting the response variable. In other words, the goal of logistic regression is to find the best fitting (yet biologically reasonable) model to describe the relationship between the dichotomous characteristic of interest (dependent variable = response or outcome variable) and a set of independent (predictor or explanatory) variables.

Advantages/Features of Logistic Regression

- Logistic Regression performs well when the dataset is linearly separable.
- Logistic regression is less prone to over-fitting, but it can be overfit in high dimensional datasets. You should consider Regularization (L1 and L2) techniques to avoid over-fitting in these scenarios.
- Logistic Regression not only gives a measure of how relevant a predictor (coefficient size) is, but also its direction of association (positive or negative).
- Logistic regression is easier to implement, interpret and very efficient to train.

Disadvantages/Shortcomings of Logistic Regression

- The main limitation of Logistic Regression is the assumption of linearity between the dependent variable and the independent variables. In the real world, the data is rarely linearly separable. Most of the time data would be a jumbled mess.
- If the number of observations is lesser than the number of features, Logistic Regression should not be used, otherwise, it may lead to overfitting.
- Logistic Regression can only be used to predict discrete functions. Therefore, the dependent variable of Logistic Regression is restricted to the discrete number set. This restriction itself is problematic, as it is prohibitive to the prediction of continuous data.

Types of Logistic Regression

Binary Logistic Regression

The categorical response has only two 2 possible outcomes. Example: Spam or Not

Multinomial Logistic Regression

Three or more categories without ordering. Example: Predicting which food is preferred more (Veg, Non-Veg, Vegan)

Ordinal Logistic Regression

Three or more categories with ordering. Example: Movie rating from 1 to 5

Key Terms

For reading about the basic terms of regression, please read the chapter Linear Regression

Logit

According to Wikipedia, In statistics, the logit function or the log-odds is the logarithm of the odds ***p***/(1 − ***p***) where p is the probability. It is a type of function that creates a map of probability values from [0,1] to $(-\infty, +\infty)$ It is the inverse of the sigmoidal "logistic" function or logistic transform used in mathematics, especially in statistics.

In deep learning, the term logits layer is popularly used for the last neuron layer of neural networks used for classification tasks, which produce raw prediction values as real numbers ranging from $(-\infty, +\infty)$.

Logistic Function

According to Wikipedia, The logistic function is a sigmoid function, which takes any real input t, ($t \in \mathbb{R}$), and outputs a value between zero and one; for the logit, this is interpreted as taking input log-odds and having output probability. The standard logistic function $\sigma : \mathbb{R} \to (0,1)$ is defined as follows:

$$\sigma(t) = \frac{e^t}{e^t+1} = \frac{1}{1+e^{-t}}$$

Inverse of Logistic Function

According to Wikipedia, We can now define the logit (log odds) function as the inverse $g = \sigma^{-1}$ of the standard logistic function. It is easy to see that it satisfies:

$$g(p(x)) = \sigma^{-1}(p(x)) = \operatorname{logit} p(x) = \ln\left(\frac{p(x)}{1-p(x)}\right) = \beta_0 + \beta_1 x,$$

and equivalently, after exponentiating both sides we have the odds:

$$\frac{p(x)}{1-p(x)} = e^{\beta_0+\beta_1 x}.$$

were,

- g is the logit function. The equation for g(p(x)) illustrates that the logit (i.e., log-odds or natural logarithm of the odds) is equivalent to the linear regression expression.
- ln denotes the natural logarithm.
- The formula for p(x) illustrates that the probability of the dependent variable for a given case is equal to the value of the logistic function of the linear regression expression. This is important as it shows that the value of the linear regression expression can vary from negative to positive infinity and yet, after transformation, the resulting probability p(x) ranges between 0 and 1.
- β_0 is the intercept from the linear regression equation (the value of the criterion when the predictor is equal to zero)
- β_1 is the regression coefficient multiplied by some value of the predictor.
- Base 'e' denotes the exponential function.

Odds

According to Wikipedia, the odds of the dependent variable equaling a case (given some linear combination x of the predictors) are equivalent to the exponential function of the linear regression expression. This illustrates how the logit serves as a link function between the probability and the linear regression expression. Given that the logit ranges between negative

and positive infinity, it provides an adequate criterion upon which to conduct linear regression and the logit is easily converted back into the odds.

So, we define odds of the dependent variable equaling a case (given some linear combination x of the predictors) as follows:

$$\text{odds} = e^{\beta_0 + \beta_1 x}$$

Odds Ration

According to Wikipedia, for a continuous independent variable, the odds ratio can be defined as:

$$\text{OR} = \frac{\text{odds}(x+1)}{\text{odds}(x)} = \frac{\left(\frac{F(x+1)}{1-F(x+1)}\right)}{\left(\frac{F(x)}{1-F(x)}\right)} = \frac{e^{\beta_0+\beta_1(x+1)}}{e^{\beta_0+\beta_1 x}} = e^{\beta_1}$$

This exponential relationship provides an interpretation for β_1: The odds multiply by e^{β_1} for every 1-unit increase in x.

For a binary independent variable, the odds ratio is defined as $\frac{ad}{bc}$ where ***a***, ***b***, ***c,*** and ***d*** are cells in a 2×2 contingency table.

Multiple Explanatory Variable

If there are multiple explanatory variables, the above expression $\beta_0 + \beta_1 x$ can be revised to

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m = \beta_0 + \sum_{i=1}^{m} \beta_i x_i$$

Then when this is used in the equation relating the log odds of success to the values of the predictors, the linear regression will be a multiple regression with ***m*** explanators; the parameters β_j for all ***j*** = 0, 1, 2, ..., ***m*** is all estimated.

Again, the more traditional equations are:

$$\log \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m$$

and

$$p = \frac{1}{1 + b^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m)}}$$

where usually b = e.

Logistic Regression

Logistic regression produces a logistic curve that is restricted to values from 0 to 1. Logistic regression is like linear regression, but the curve is developed using a normal logarithm rather than the likelihood of "odds" in the goal variable. In fact, the predictors must not be distributed uniformly or have the same variation in the individual groups.

The logistic Regression Equation is given by

$$\frac{p}{1-p} = e^{b_0 + b_1 * x}$$

Taking natural log on both sides we get

$$\ln \frac{p}{1-p} = b_0 + b_1 * x$$

Till now, we have seen the equation for one variable, so now following is the equation when the number of variables is more than one

$$p = \frac{1}{1 + b^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m)}}$$

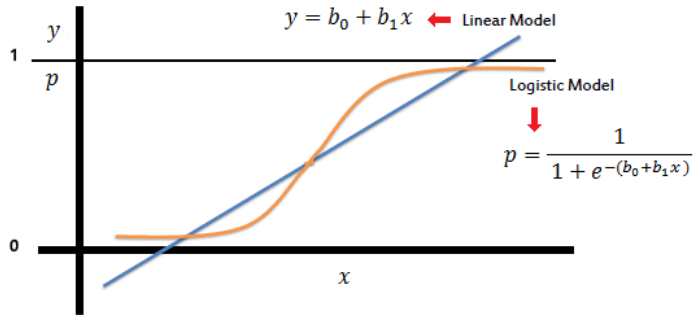
where usually b = e. OR

$$\log \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m$$

Let me use an example, to explain in which cases we will use logistic Regression:

In cases where borders are predefined, a linear regression will fail as though it could predict beyond the borders if we are going to use a Linear Regression. Let us take, for example, the example we used in the last chapter of the housing price prediction, so there are chances when you predict that linear regression predicts the price too high, which is not possible practically, or too low, as it might be negative.

Since there are only two possible outcomes for binary classification, but input data need not be expressed uniformly, it is often seen that the data point class' 0' is located at the judgment boundary of class '1'. Since the sigmoid feature is a circle, it improves health.



Logistic Regression Example

Let's take the example of the IRIS dataset, you can directly import it from the sklearn dataset repository. Feel free to use any dataset, there are some very good datasets available on kaggle and with Google Colab.

Using SKLearn

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
```

In the above code, we are importing the required libraries, that we will be using

```
iris = datasets.load_iris()
```

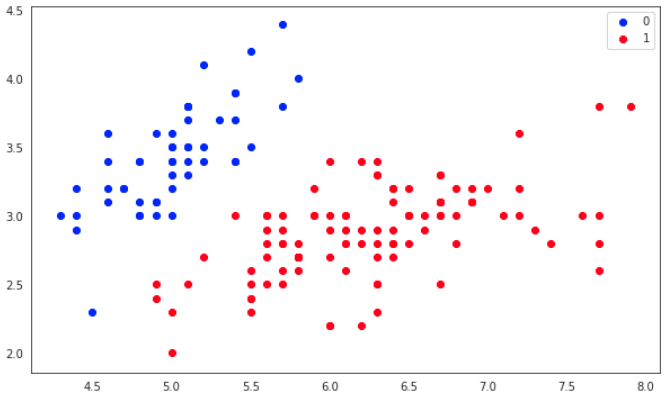
Loading the IRIS dataset from the sklearn dataset repository

```
X = iris.data[:, :2]
y = (iris.target != 0) * 1
```

Manipulating and pre-processing the data, so that it can be fed to the model

```
plt.figure(figsize=(10, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='b', label='0')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='r', label='1')
plt.legend();
```

Let us try to visualize the imported data



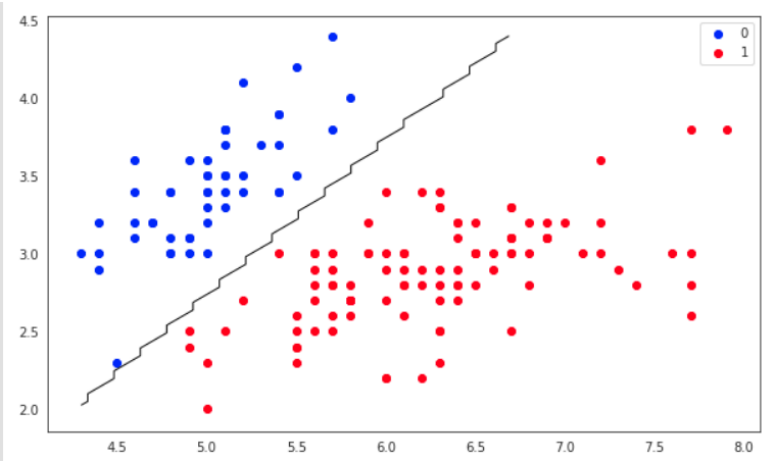

```
model = LogisticRegression(C=1e2)
%time model.fit(X, y)
print(model.intercept_, model.coef_,model.n_iter_)
```

In the above code, we are timing the training process using the "%time", and then printing the model parameters.

The output that I got for training is:
CPU times: user 2.45 ms, sys: 1.06 ms, total: 3.51 ms Wall time: 1.76 ms
model.itercept: [-33.08987216]
model.coeffiecent: [[14.75218964 -14.87575477]]
number of iterations: [12]

```
plt.figure(figsize=(10, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='b', label='0')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='r', label='1')
plt.legend()
x1_min, x1_max = X[:,0].min(), X[:,0].max(),
x2_min, x2_max = X[:,1].min(), X[:,1].max(),
xx1, xx2 = np.meshgrid(np.linspace(x1_min, x1_max), np.linspace(x2_min,
 x2_max))
grid = np.c_[xx1.ravel(), xx2.ravel()]
probs = model.predict(grid).reshape(xx1.shape)
plt.contour(xx1, xx2, probs, [0.5], linewidths=1, colors='black');
```

The above code lets us visualize the regression line with respect to the input data.




```
pred = model.predict(X[1:2])
print(pred)
```

In the above code, we predict that the class of the sample X[1:2], and the class result out to be [0], which is correct

LR_Sklearn.py

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.linear_model import LogisticRegression

iris = datasets.load_iris()

X = iris.data[:, :2]
y = (iris.target != 0) * 1

plt.figure(figsize=(10, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='b', label='0')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='r', label='1')
plt.legend();

model = LogisticRegression(C=1e2)
%time model.fit(X, y)
print(model.intercept_, model.coef_,model.n_iter_)

plt.figure(figsize=(10, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='b', label='0')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='r', label='1')
plt.legend()
x1_min, x1_max = X[:,0].min(), X[:,0].max(),
x2_min, x2_max = X[:,1].min(), X[:,1].max(),
xx1, xx2 = np.meshgrid(np.linspace(x1_min, x1_max), np.linspace(x2_min,
 x2_max))
```

```
grid = np.c_[xx1.ravel(), xx2.ravel()]
probs = model.predict(grid).reshape(xx1.shape)
plt.contour(xx1, xx2, probs, [0.5], linewidths=1, colors='black');
```

Using Numpy

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
```

In the above code, we are importing the required libraries, that we will be using

```
iris = datasets.load_iris()
```

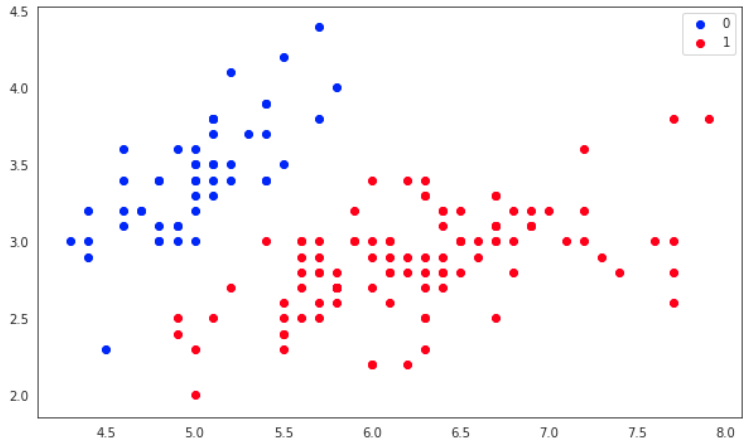
Loading the IRIS dataset from the sklearn dataset repository

```
X = iris.data[:, :2]
y = (iris.target != 0) * 1
```

Manipulating and pre-processing the data, so that it can be fed to the model.

```
plt.figure(figsize=(10, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='b', label='0')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='r', label='1')
plt.legend();
```

Let us try to visualize the imported data




```
class LogisticRegression:
    def __init__(self, lr=0.01, num_iter=100000, fit_intercept=True, ve
rbose=False):
        self.lr = lr
        self.num_iter = num_iter
        self.fit_intercept = fit_intercept
        self.verbose = verbose

    def __add_intercept(self, X):
        intercept = np.ones((X.shape[0], 1))
        return np.concatenate((intercept, X), axis=1)
```

```
    def __sigmoid(self, z):
        return 1 / (1 + np.exp(-z))
    def __loss(self, h, y):
        return (-y * np.log(h) - (1 - y) * np.log(1 - h)).mean()

    def fit(self, X, y):
        if self.fit_intercept:
            X = self.__add_intercept(X)

        # weights initialization
        self.theta = np.zeros(X.shape[1])

        for i in range(self.num_iter):
            z = np.dot(X, self.theta)
            h = self.__sigmoid(z)
            gradient = np.dot(X.T, (h - y)) / y.size
            self.theta -= self.lr * gradient

            z = np.dot(X, self.theta)
            h = self.__sigmoid(z)
            loss = self.__loss(h, y)

            if(self.verbose ==True and i % 10000 == 0):
                print(f'loss: {loss} \t')

    def predict_prob(self, X):
        if self.fit_intercept:
            X = self.__add_intercept(X)

        return self.__sigmoid(np.dot(X, self.theta))

    def predict(self, X):
        return self.predict_prob(X).round()
```

In the above code, we created a user-defined class "LogisticRegression", which contain all the methods required to result out the desired regression line

- __init__ - constructor to initialize all the required variables with default values or initial values
- __add_intercept - to find the model intercept value
- __sigmoid - function to return sigmoid curve
- __loss - function to return loss
- fit - function to calculate and return the regression line
- predict_prob - helper function used for prediction
- predict - function to return predicted value

```
model = LogisticRegression(lr=0.1, num_iter=3000)
%time model.fit(X, y)
```

In the above code, we intiantiate the LogisticRegression class, and then provide 'X' and 'y' as the parameters to the fit function to result out the desired regression line

```
preds = model.predict(X[1:2])
print(preds)
```

In the above code, we ask the model to tell the class to which our sample X[1:2] belong and the result that we get is [0.], which is correct

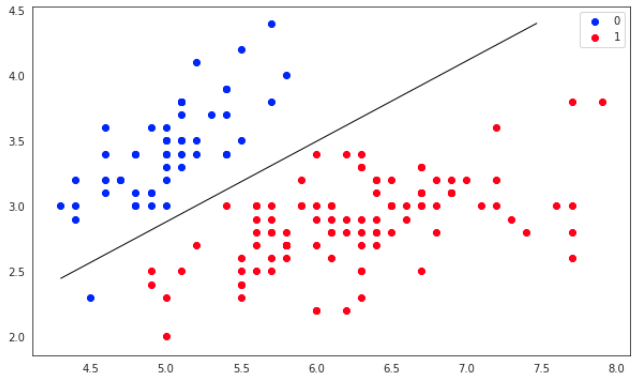
```
print(model.theta)
```

Now we print the parameter values of the resulting model.

Parameter values of my model are:
[-1.44894305 4.25546329 -6.89489245]

```
plt.figure(figsize=(10, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='b', label='0')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='r', label='1')
plt.legend()
x1_min, x1_max = X[:,0].min(), X[:,0].max(),
x2_min, x2_max = X[:,1].min(), X[:,1].max(),
xx1, xx2 = np.meshgrid(np.linspace(x1_min, x1_max), np.linspace(x2_min,
 x2_max))
grid = np.c_[xx1.ravel(), xx2.ravel()]
probs = model.predict_prob(grid).reshape(xx1.shape)
plt.contour(xx1, xx2, probs, [0.5], linewidths=1, colors='black');
```

The above code will provide us with a visualization of the generated regression line with respect to the input data.



LR_NumPy.py

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets

# Load the iris dataset
iris = datasets.load_iris()

# Select the first two features
X = iris.data[:, :2]
```

```
# Convert target to binary classification problem
y = (iris.target != 0) * 1

# Plot the data
plt.figure(figsize=(10, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='b', label='0')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='r', label='1')
plt.legend()

class LogisticRegression:
    def __init__(self, lr=0.01, num_iter=100000, fit_intercept=True,
verbose=False):
        self.lr = lr
        self.num_iter = num_iter
        self.fit_intercept = fit_intercept
        self.verbose = verbose

    def __add_intercept(self, X):
        intercept = np.ones((X.shape[0], 1))
        return np.concatenate((intercept, X), axis=1)

    def __sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def __loss(self, h, y):
        return (-y * np.log(h) - (1 - y) * np.log(1 - h)).mean()

    def fit(self, X, y):
        if self.fit_intercept:
            X = self.__add_intercept(X)

        # Weights initialization
        self.theta = np.zeros(X.shape[1])

        for i in range(self.num_iter):
            z = np.dot(X, self.theta)
            h = self.__sigmoid(z)
            gradient = np.dot(X.T, (h - y)) / y.size
            self.theta -= self.lr * gradient

            z = np.dot(X, self.theta)
            h = self.__sigmoid(z)
            loss = self.__loss(h, y)

            if self.verbose and i % 10000 == 0:
                print(f'loss: {loss} \t')

    def predict_prob(self, X):
```

```
        if self.fit_intercept:
            X = self.__add_intercept(X)

        return self.__sigmoid(np.dot(X, self.theta))

    def predict(self, X):
        return self.predict_prob(X).round()

# Create and train the logistic regression model
model = LogisticRegression(lr=0.1, num_iter=3000)
%time model.fit(X, y)

# Make a prediction for a specific example
preds = model.predict(X[1:2])
print(preds)

# Print model parameters
print(model.theta)

# Plot the decision boundary
plt.figure(figsize=(10, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='b', label='0')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='r', label='1')
plt.legend()

x1_min, x1_max = X[:, 0].min(), X[:, 0].max()
x2_min, x2_max = X[:, 1].min(), X[:, 1].max()
xx1, xx2 = np.meshgrid(np.linspace(x1_min, x1_max), np.linspace(x2_min,
x2_max))
grid = np.c_[xx1.ravel(), xx2.ravel()]
probs = model.predict_prob(grid).reshape(xx1.shape)

plt.contour(xx1, xx2, probs, [0.5], linewidths=1, colors='black')
plt.show()
```

Using TensorFlow

```
from __future__ import print_function
import tensorflow as tf
```

In the above code, we are importing the required libraries

```
# Import MNIST data
from tensorflow.examples.tutorials.mnist import input_data
mnist = input_data.read_data_sets("/tmp/data/", one_hot=True)
```

Using the TensorFlow's dataset library, we are importing MNIST dataset

```
# Parameters
learning_rate = 0.01
training_epochs = 100
```

```
batch_size = 100
display_step = 50
```

In the above code, we are assigning the values to all the global parameters.

```
# tf Graph Input
x = tf.placeholder(tf.float32, [None, 784]) # mnist data image of shape
 28*28=784
y = tf.placeholder(tf.float32, [None, 10]) # 0-
9 digits recognition => 10 classes

# Set model weights
W = tf.Variable(tf.zeros([784, 10]))
b = tf.Variable(tf.zeros([10]))
```

Here we are setting X and Y as the actual training data and the W and b as the trainable data, where:

- W means Weight
- b means bias
- X means the dependent variable
- Y means the independent variable

```
# Construct model
pred = tf.nn.softmax(tf.matmul(x, W) + b) # Softmax

# Minimize error using cross entropy
cost = tf.reduce_mean(-
tf.reduce_sum(y*tf.log(pred), reduction_indices=1))
# Gradient Descent
optimizer = tf.train.GradientDescentOptimizer(learning_rate).minimize(c
ost)

# Initialize the variables (i.e. assign their default value)
init = tf.global_variables_initializer()
```

In the above code, we are

- setting the model using the SoftMax method
- cost calculation will be based on the reduced mean method
- the optimizer is chosen to be Gradient Descent which minimizes cost
- the variable initializer is chosen to be a global variable initializer

```
# Start training
with tf.Session() as sess:
    # Run the initializer
    sess.run(init)

    # Training cycle
    for epoch in range(training_epochs):
        avg_cost = 0.
        total_batch = int(mnist.train.num_examples / batch_size)
```

```
        # Loop over all batches
        for i in range(total_batch):
            batch_xs, batch_ys = mnist.train.next_batch(batch_size)

            # Run optimization op (backprop) and cost op (to get loss
value)
            _, c = sess.run([optimizer, cost], feed_dict={x: batch_xs,
y: batch_ys})

            # Compute average loss
            avg_cost += c / total_batch

        # Display logs per epoch step
        if (epoch + 1) % display_step == 0:
            print("Epoch:", '%04d' % (epoch + 1), "cost=",
"{:.9f}".format(avg_cost))

    # Store the final cost, weight, and bias after training
    training_cost = sess.run(cost, feed_dict={x: batch_xs, y:
batch_ys})
    weight = sess.run(W)
    bias = sess.run(b)

print("Training cost:", training_cost)
print("Weight:", weight)
print("Bias:", bias)
```

In the above code, we start training, and the cost is printed after every 50 epochs. Here since the number of data points are large and processing in one go may cause crashes, we train in batches.

```
print("W",weight,"\nb",bias)
eq= tf.math.sigmoid((tf.matmul(x, weight) + bias))
```

In the above code, we print the weight and bias of the learned model and then form the Logistic Regression Equation.

LR_tensorflow.py

```
from __future__ import print_function

import tensorflow as tf

# Import MNIST data
from tensorflow.examples.tutorials.mnist import input_data
mnist = input_data.read_data_sets("/tmp/data/", one_hot=True)

# Parameters
learning_rate = 0.01
training_epochs = 100
```

```
batch_size = 100
display_step = 50

# tf Graph Input
x = tf.placeholder(tf.float32, [None, 784])  # MNIST data image of
shape 28*28=784
y = tf.placeholder(tf.float32, [None, 10])   # 0-9 digits recognition
=> 10 classes

# Set model weights
W = tf.Variable(tf.zeros([784, 10]))
b = tf.Variable(tf.zeros([10]))

# Construct model
pred = tf.nn.softmax(tf.matmul(x, W) + b)  # Softmax

# Minimize error using cross entropy
cost = tf.reduce_mean(-tf.reduce_sum(y * tf.log(pred),
reduction_indices=1))
# Gradient Descent
optimizer =
tf.train.GradientDescentOptimizer(learning_rate).minimize(cost)

# Initialize the variables (i.e. assign their default value)
init = tf.global_variables_initializer()

# Start training
with tf.Session() as sess:
    # Run the initializer
    sess.run(init)

    # Training cycle
    for epoch in range(training_epochs):
        avg_cost = 0.
        total_batch = int(mnist.train.num_examples / batch_size)

        # Loop over all batches
        for i in range(total_batch):
            batch_xs, batch_ys = mnist.train.next_batch(batch_size)

            # Run optimization op (backprop) and cost op (to get loss
value)
            _, c = sess.run([optimizer, cost], feed_dict={x: batch_xs,
y: batch_ys})

            # Compute average loss
            avg_cost += c / total_batch
```

```
        # Display logs per epoch step
        if (epoch + 1) % display_step == 0:
            print("Epoch:", '%04d' % (epoch + 1), "cost=",
"{:.9f}".format(avg_cost))

    # Store the final cost, weight, and bias after training
    training_cost = sess.run(cost, feed_dict={x: batch_xs, y:
batch_ys})
    weight = sess.run(W)
    bias = sess.run(b)

print("W", weight, "\nb", bias)

# Calculate the sigmoid of the linear combination
eq = tf.math.sigmoid((tf.matmul(x, weight) + bias))
```

The output that I got is

```
Epoch: 0050 cost= 0.304499915
Epoch: 0100 cost= 0.283256568
W [[0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]
b [-0.3819625   0.36264426  0.09707038 -0.26997086  0.00440918  1.3265812
 -0.10331464  0.6451546  -1.442353   -0.23826294]
```

Conclusion

In this chapter, we studied what is regression, types of regression, what is logistic regression, what it is used for, difference between linear and logistic regression, why is logistic regression called so, what is the goal of logistic regression, advantages of logistic regression, disadvantages of logistic regression, types of logistic regression, key terms, logistic regression, logistic regression using numpy, sklearn and TensorFlow, and their python implementation . Hope you were able to understand each and everything.

6

Multiple Linear Regression

Overview

In this chapter, we explore Multiple Linear Regression, covering its key concepts, calculation methods, and practical implementations. Prepare yourself for a comprehensive journey that will empower you to effectively leverage MLR for analyzing and predicting data with multiple features.

When should we use Multiple Linear Regression?

Multiple Linear Regression is an extended version of simple Linear regression, with one most important difference being the number of features it can handle. Multiple Linear Regression can handle more than 1 feature. So, we should use Multiple Linear Regression in cases where the dataset is uniformly distributed and has more than 1 feature to process.

How do we calculate multiple Linear Regression?

The formula of the linear regression doesn't change, it remains y= m*X+b, only the number of coefficients increases

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + ... + \beta_p x_{ip} + \epsilon$$

where, for $i = n$ observations:

$y_i = \text{dependent variable}$

$x_i = \text{expanatory variables}$

$\beta_0 = \text{y-intercept (constant term)}$

$\beta_p = \text{slope coefficients for each explanatory variable}$

$\epsilon = \text{the model's error term (also known as the residuals)}$

Advantages/Features of Multiple Linear Regression

- The chances of getting a better fit increase as the generated models are dependent on more than 1 feature
- Multiple Linear Regression can detect outliers and anomalies very effectively.

Disadvantages/Shortcomings of Multiple Linear Regression

- The problem of overfitting is very prevalent here, as we can use all features to generate the model, so the model can start "memorizing" the values
- Accuracy decreases as the linearity of the dataset decreases.

Multiple Linear Regression

Multiple linear regression (MLR) is the statistical technique used to predict the outcome of a response variable using several prepared variables. The aim is to model linear relationships between the explanation (independent) and response variables (dependent) to a multiple linear regression (MLR).

In essence, multiple regression is the extension of ordinary least-squares (OLS) regression that involves more than one explanatory variable.

Simple linear regression is a means of making estimates for a single variable based on information about a specific variable identified by an observer or statistician. Only when one has two continuous variables— one independent variable and one dependent variable can linear regression be used. The independent variable is the parameter that is used to calculate the dependent variable or outcome. There are several explanatory variables provided by the multiple regression model.

The multiple regression model is based on the following assumptions:

- **Linearity:** There is a linear relationship between the dependent variables and the independent variables.
- **Correlation:** The independent variables are not too highly correlated with each other.
- yi observations are selected independently and randomly from the population.

- **Normal Distribution:** Residuals should be normally distributed with a mean of 0 and variance σ.

When interpreting the results of multiple regression, beta coefficients are valid while holding all other variables constant ("all else equal"). The output from a multiple regression can be displayed horizontally as an equation, or vertically in table form.

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + ... + \beta_p x_{ip} + \epsilon$$

where, for $i = n$ observations:

y_i = dependent variable

x_i = expanatory variables

β_0 = y-intercept (constant term)

β_p = slope coefficients for each explanatory variable

ϵ = the model's error term (also known as the residuals)

Multiple Linear Regression Example

Let's take the example of the IRIS dataset, you can directly import it from the sklearn dataset repository. Feel free to use any dataset, there some very good datasets available on Kaggle and with Google Colab.

Using SkLearn

```
from pandas import DataFrame
from sklearn import linear_model
import statsmodels.api as sm
```

In the above code, we import the required python libraries.

```
Stock_Market = {
    'Year': [
        2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017,
2017, 2017,
        2016, 2016, 2016, 2016, 2016, 2016, 2016, 2016, 2016, 2016,
2016, 2016
    ],
    'Month': [
        12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
    ],
    'Interest_Rate': [
        2.75, 2.5, 2.5, 2.5, 2.5, 2.5, 2.5, 2.25, 2.25, 2.25, 2, 2,
        2, 1.75, 1.75, 1.75, 1.75, 1.75, 1.75, 1.75, 1.75, 1.75, 1.75,
1.75
    ],
    'Unemployment_Rate': [
        5.3, 5.3, 5.3, 5.3, 5.4, 5.6, 5.5, 5.5, 5.5, 5.6, 5.7, 5.9,
        6, 5.9, 5.8, 6.1, 6.2, 6.1, 6.1, 6.1, 5.9, 6.2, 6.2, 6.1
    ],
    'Stock_Index_Price': [
```

```
        1464, 1394, 1357, 1293, 1256, 1254, 1234, 1195, 1159, 1167,
1130, 1075,
        1047, 965, 943, 958, 971, 949, 884, 866, 876, 822, 704, 719
    ]
}
```

In the above code, we are defining our data.

```
df = DataFrame(Stock_Market, columns=['Year', 'Month', 'Interest_Rate',
'Unemployment_Rate', 'Stock_Index_Price'])

X = df[['Interest_Rate', 'Unemployment_Rate']]
Y = df['Stock_Index_Price']
```

In the above code, we are pre-processing the data.

```
regr = linear_model.LinearRegression()
regr.fit(X, Y)
```

In the above code, we are generating the model

```
print('Intercept: \n', regr.intercept_)
print('Coefficients: \n', regr.coef_)
```

In the above code, we are printing the parameters of the generated model

the output that I am getting is:
Intercept: 1798.4039776258546
Coefficients: [345.54008701 -250.14657137]

```
# Prediction with sklearn
New_Interest_Rate = 2.75
New_Unemployment_Rate = 5.3
print('Predicted Stock Index Price: \n',
regr.predict([[New_Interest_Rate, New_Unemployment_Rate]]))
```

In the above code, we are predicting the stock price corresponding to the given feature values.

MLR_SkLearn.py

```
from pandas import DataFrame
from sklearn import linear_model
import statsmodels.api as sm

Stock_Market = {
    'Year': [
        2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017,
2017, 2017,
        2016, 2016, 2016, 2016, 2016, 2016, 2016, 2016, 2016, 2016,
2016, 2016
    ],
    'Month': [
        12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
    ],
```

```
    'Interest_Rate': [
        2.75, 2.5, 2.5, 2.5, 2.5, 2.5, 2.5, 2.25, 2.25, 2.25, 2, 2,
        2, 1.75, 1.75, 1.75, 1.75, 1.75, 1.75, 1.75, 1.75, 1.75, 1.75,
1.75
    ],
    'Unemployment_Rate': [
        5.3, 5.3, 5.3, 5.3, 5.4, 5.6, 5.5, 5.5, 5.5, 5.6, 5.7, 5.9,
        6, 5.9, 5.8, 6.1, 6.2, 6.1, 6.1, 6.1, 5.9, 6.2, 6.2, 6.1
    ],
    'Stock_Index_Price': [
        1464, 1394, 1357, 1293, 1256, 1254, 1234, 1195, 1159, 1167,
1130, 1075,
        1047, 965, 943, 958, 971, 949, 884, 866, 876, 822, 704, 719
    ]
}

df = DataFrame(Stock_Market, columns=[
    'Year', 'Month', 'Interest_Rate', 'Unemployment_Rate',
'Stock_Index_Price'
])

X = df[['Interest_Rate', 'Unemployment_Rate']]
Y = df['Stock_Index_Price']

# with sklearn
regr = linear_model.LinearRegression()
regr.fit(X, Y)

print('Intercept: \n', regr.intercept_)
print('Coefficients: \n', regr.coef_)

# prediction with sklearn
New_Interest_Rate = 2.75
New_Unemployment_Rate = 5.3
print('Predicted Stock Index Price: \n',
regr.predict([[New_Interest_Rate, New_Unemployment_Rate]]))
```

Output

Intercept: 1798.4039776258546
Coefficients: [345.54008701 -250.14657137]
Predicted Stock Index Price: [1422.86238865]

```
print_model = model.summary()
print(print_model)
```

Output

```
                            OLS Regression Results
==============================================================================
Dep. Variable:      Stock_Index_Price   R-squared:                       0.898
Model:                            OLS   Adj. R-squared:                  0.888
Method:                 Least Squares   F-statistic:                     92.07
Date:                Sun, 27 Oct 2019   Prob (F-statistic):           4.04e-11
Time:                        11:55:09   Log-Likelihood:                -134.61
No. Observations:                  24   AIC:                             275.2
Df Residuals:                      21   BIC:                             278.8
Df Model:                           2
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          1798.4040    899.248      2.000      0.059     -71.685    3668.493
Interest_Rate   345.5401    111.367      3.103      0.005     113.940     577.140
Unemployment_Rate -250.1466    117.950     -2.121      0.046    -495.437      -4.856
==============================================================================
Omnibus:                        2.691   Durbin-Watson:                   0.530
Prob(Omnibus):                  0.260   Jarque-Bera (JB):                1.551
Skew:                          -0.612   Prob(JB):                        0.461
Kurtosis:                       3.226   Cond. No.                         394.
==============================================================================

Warnings:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Using NumPy

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In the above code, we are importing the necessary libraries.

```
my_data = pd.read_csv('home.txt',names=["size","bedroom","price"])
```

In the above code, we are importing the data. You can download the "home.txt" file from the chapter.

```
# We need to normalize the features using mean normalization
my_data = (my_data - my_data.mean()) / my_data.std()

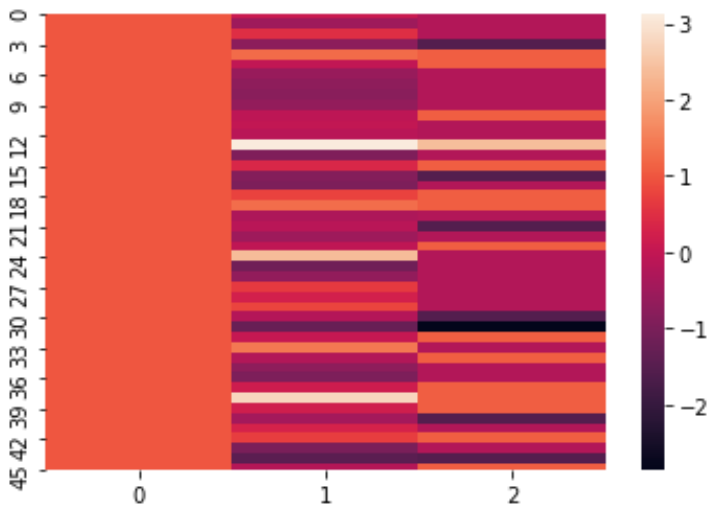
# Setting the matrices
X = my_data.iloc[:, 0:2]
ones = np.ones([X.shape[0], 1])
X = np.concatenate((ones, X), axis=1)

y = my_data.iloc[:, 2:3].values  # .values converts it from
pandas.core.frame.DataFrame to numpy.ndarray
theta = np.zeros([1, 3])
```

In the above code, we are preprocessing the data.

```
sns.heatmap(X)
```

Let us visualize the data using a heatmap.




```
def computeCost(X, y, theta):
    to_be_summed = np.power(((X @ theta.T) - y), 2)
    return np.sum(to_be_summed) / (2 * len(X))

def gradientDescent(X, y, theta, iters, alpha):
    cost = np.zeros(iters)
    for i in range(iters):
        theta = theta - (alpha / len(X)) * np.sum(X * (X @ theta.T -
y), axis=0)
        cost[i] = computeCost(X, y, theta)

    return theta, cost
```

In the above code, we are defining the methods for finding the cost and for gradient descent

```
#set hyperparameters
alpha = 0.01
iters = 1000
```

In the above code, we are setting the value for the hyperparameters.

```
g,cost = gradientDescent(X,y,theta,iters,alpha)
print(g)

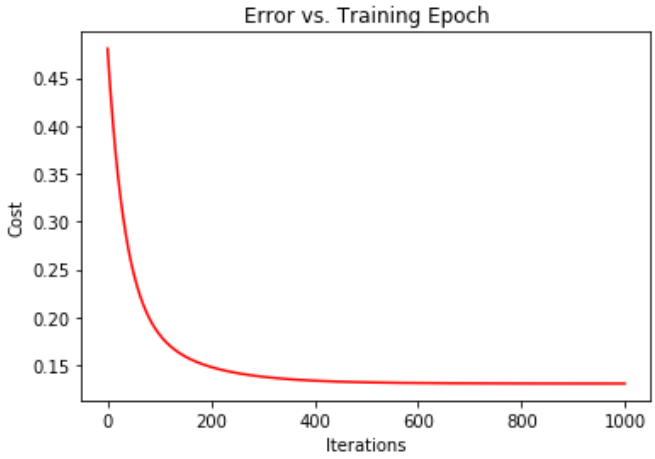
finalCost = computeCost(X,y,g)
print(finalCost)
```

In the above code, we are calling the methods for fitting the model

The output that I am getting is
[[-1.10868761e-16 8.78503652e-01 -4.69166570e-02]] 0.13070336960771892

```
fig, ax = plt.subplots()
ax.plot(np.arange(iters), cost, 'r')
ax.set_xlabel('Iterations')
ax.set_ylabel('Cost')
ax.set_title('Error vs. Training Epoch')
```

In the above code, we are generating the graph of Error vs Training Epochs



MLR_NumPy.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

my_data = pd.read_csv('home.txt', names=["size", "bedroom", "price"])

# Normalize the features using mean normalization
my_data = (my_data - my_data.mean()) / my_data.std()

# Setting the matrices
X = my_data.iloc[:, 0:2]
ones = np.ones([X.shape[0], 1])
X = np.concatenate((ones, X), axis=1)

y = my_data.iloc[:, 2:3].values  # .values converts it from
pandas.core.frame.DataFrame to numpy.ndarray
theta = np.zeros([1, 3])

sns.heatmap(X)

# Compute cost
def computeCost(X, y, theta):
    tobesummed = np.power(((X @ theta.T) - y), 2)
    return np.sum(tobesummed) / (2 * len(X))

def gradientDescent(X, y, theta, iters, alpha):
    cost = np.zeros(iters)
    for i in range(iters):
        theta = theta - (alpha / len(X)) * np.sum(X * (X @ theta.T -
y), axis=0)
        cost[i] = computeCost(X, y, theta)
```

```
    return theta, cost

# Set hyperparameters
alpha = 0.01
iters = 1000

g, cost = gradientDescent(X, y, theta, iters, alpha)
print(g)

finalCost = computeCost(X, y, g)
print(finalCost)

fig, ax = plt.subplots()
ax.plot(np.arange(iters), cost, 'r')
ax.set_xlabel('Iterations')
ax.set_ylabel('Cost')
ax.set_title('Error vs. Training Epoch')
```

Using TensorFlow

```
import matplotlib.pyplot as plt
import tensorflow as tf
import tensorflow.contrib.learn as skflow
from sklearn.utils import shuffle
import numpy as np
import pandas as pd
import seaborn as sns
```

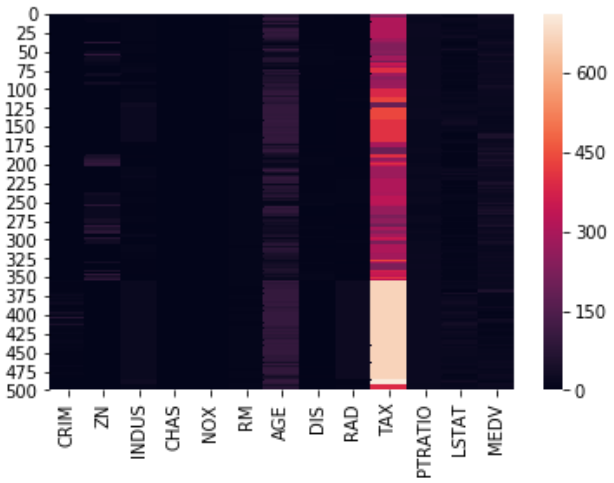
In the above code, we are importing the required libraries

```
df = pd.read_csv("boston.csv", header=0)
print (df.describe())
```

In the above code, we are importing the dataset. You can download the dataset from Kaggle

```
sns.heatmap(df)
```

Let us visualize the data.



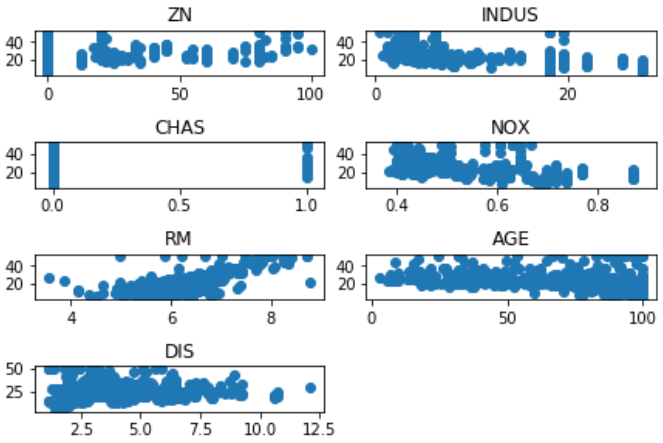
```
f, ax1 = plt.subplots()

y = df['MEDV']

for i in range (1,8):
    number = 420 + i
    ax1.locator_params(nbins=3)
    ax1 = plt.subplot(number)
    plt.title(list(df)[i])
    ax1.scatter(df[df.columns[i]],y) #Plot a scatter draw of the  datap
oints
plt.tight_layout(pad=0.4, w_pad=0.5, h_pad=1.0)

plt.show()
```

Let us visualize each dataset column separately.




```
X = tf.placeholder("float", name="X") # create symbolic variables
Y = tf.placeholder("float", name = "Y")
```

In the above code, we are defining the actual trainable variables.

```
with tf.name_scope("Model"):

    w = tf.Variable(tf.random_normal([2], stddev=0.01), name="b0") # cr
eate a shared variable
    b = tf.Variable(tf.random_normal([2], stddev=0.01), name="b1") # cr
eate a shared variable

    def model(X, w, b):
        return tf.multiply(X, w) + b # We just define the line as X*w +
 b0

    y_model = model(X, w, b)
```

In the above code, we are defining the model.

```
with tf.name_scope("CostFunction"):
    cost = tf.reduce_mean(tf.pow(Y-
y_model, 2)) # use sqr error for cost function

train_op = tf.train.AdamOptimizer(0.001).minimize(cost)
```

In the above code, we are defining the cost function and the cost optimizer function.

```
sess = tf.Session()
init = tf.initialize_all_variables()
tf.train.write_graph(sess.graph, '/home/bonnin/linear2','graph.pbtxt')

cost_op = tf.summary.scalar("loss", cost)
merged = tf.summary.merge_all()
sess.run(init)
writer = tf.summary.FileWriter('/home/bonnin/linear2', sess.graph)
```

In the above code, we create the garph file which can be used to visualize the model on TensorBoard.

```
xvalues = df[[df.columns[2], df.columns[4]]].values.astype(float)
yvalues = df[df.columns[12]].values.astype(float)
b0temp=b.eval(session=sess)
b1temp=w.eval(session=sess)
```

In the above code, we are making sure that the values are accessible to us even after the session ends.

```
for a in range (1,50):
    cost1=0.0
    for i, j in zip(xvalues, yvalues):
        sess.run(train_op, feed_dict={X: i, Y: j})
        cost1+=sess.run(cost, feed_dict={X: i, Y: i})/506.00
    xvalues, yvalues = shuffle (xvalues, yvalues)
    print ("Cost over iterations",cost1)
    b0temp=b.eval(session=sess)
    b1temp=w.eval(session=sess)
```

In the above code, we are doing training.

```
print("the final equation comes out to be", b0temp,"+",b1temp,"*X","\n
Cost :",cost1)
```

In the above code, we are printing the model and the final cost.

The output that I am getting is
the final equation comes out to be [4.7545404 7.7991614] + [1.0045488 7.807921] *X
Cost: 75.29625831573846

```
Cost over iterations 43.97842698800473
Cost over iterations 13.06591106139919
Cost over iterations 2.4179102472165828
Cost over iterations 2.8702509063977018
Cost over iterations 6.534330847706246
Cost over iterations 10.304406913372823
Cost over iterations 14.966009442514125
Cost over iterations 18.846999664080474
Cost over iterations 22.603467994056643
Cost over iterations 26.423013513738454
Cost over iterations 31.607062128692732
Cost over iterations 35.1912616948365
Cost over iterations 40.05923127562632
Cost over iterations 46.18331915210831
Cost over iterations 51.3385801202224
Cost over iterations 57.081333356412514
Cost over iterations 62.86792002742002
Cost over iterations 69.2435826840608
Cost over iterations 75.29625831573846
```

MLR_TensorFlow.py

```
import tensorflow as tf
import tensorflow.contrib.learn as skflow
from sklearn.utils import shuffle
import numpy as np
import pandas as pd

df = pd.read_csv("boston.csv", header=0)
print (df.describe())

f, ax1 = plt.subplots()
import seaborn as sns
sns.heatmap(df)

y = df['MEDV']

for i in range (1,8):
    number = 420 + i
    ax1.locator_params(nbins=3)
    ax1 = plt.subplot(number)
    plt.title(list(df)[i])
    ax1.scatter(df[df.columns[i]],y) #Plot a scatter draw of the  datap
oints
plt.tight_layout(pad=0.4, w_pad=0.5, h_pad=1.0)

plt.show()

X = tf.placeholder("float", name="X") # create symbolic variables
Y = tf.placeholder("float", name = "Y")

with tf.name_scope("Model"):

    w = tf.Variable(tf.random_normal([2], stddev=0.01), name="b0") # cr
eate a shared variable
```

```
    b = tf.Variable(tf.random_normal([2], stddev=0.01), name="b1") # cr
eate a shared variable

    def model(X, w, b):
        return tf.multiply(X, w) + b # We just define the line as X*w +
 b0

    y_model = model(X, w, b)

with tf.name_scope("CostFunction"):
    cost = tf.reduce_mean(tf.pow(Y-
y_model, 2)) # use sqr error for cost function

train_op = tf.train.AdamOptimizer(0.001).minimize(cost)


sess = tf.Session()
init = tf.initialize_all_variables()
tf.train.write_graph(sess.graph, '/home/bonnin/linear2','graph.pbtxt')

cost_op = tf.summary.scalar("loss", cost)
merged = tf.summary.merge_all()
sess.run(init)
writer = tf.summary.FileWriter('/home/bonnin/linear2', sess.graph)

xvalues = df[[df.columns[2], df.columns[4]]].values.astype(float)
yvalues = df[df.columns[12]].values.astype(float)
b0temp=b.eval(session=sess)
b1temp=w.eval(session=sess)

for a in range (1,50):
    cost1=0.0
    for i, j in zip(xvalues, yvalues):
        sess.run(train_op, feed_dict={X: i, Y: j})
        cost1+=sess.run(cost, feed_dict={X: i, Y: i})/506.00
    xvalues, yvalues = shuffle (xvalues, yvalues)
    print ("Cost over iterations",cost1)
    b0temp=b.eval(session=sess)
    b1temp=w.eval(session=sess)

print("the final equation comes out to be", b0temp,"+",b1temp,"*X")
```

Conclusion

In this chapter, we studied what is regression, types of regression and why should we use multiple linear regression, how do we calculate multiple linear regression, advantages of multiple linear regression, disadvantages of multiple linear regression, multiple linear regression example using sklearn, numpy, and TensorFlow. Hope you were able to understand each and everything.

7

Decision Tree

Overview

In this chapter, we delve into decision trees, exploring their structure, usage, and types. Learn how decision trees support decision-making through their intuitive, tree-like models and gain insights into their advantages, limitations, and practical implementations.

What is Decision Tree?

A decision tree is a tool to support decision making using tree-like a graph or decision model and its potential consequences, including chance event results, resource costs and utility. It is one way to display an algorithm containing only conditions for control.

Decision Trees (DTs) are a non-parametric supervised learning method used for both classification and regression. Decision trees learn from data to approximate a sine curve with a set of if-then-else decision rules. The deeper the tree, the more complex the decision rules, and the fitter the model.

The Decision Tree constructs a Category or Regression System as a tree framework, hence named the CART Tree. This disconnects a data set at the same time into smaller and smaller subsets along a decision tree. The ultimate result is a tree of leaf nodes and judgment nodes. There are two or more divisions of a judgment node. A designation or judgment is the leaf node. The highest node in a tree is the best predictor or the root node. All categorical and numerical data can be handled by judgment arbors.

When is Decision Tree Used?

- When the user has an objective, he is trying to achieve max profit, optimize cost
- When there are several courses of action
- There is a calculated measure of the benefit of the various alternatives
- When there are events beyond the control of the decision-maker i.e. environment factor.
- Uncertainty concerning which outcome will happen

Assumptions of Decision Tree

- In the beginning, the whole training set is considered as the root.
- Feature values are preferred to be categorical. If the values are continuous then they are discretized prior to building the model.
- Records are distributed recursively based on attribute values.
- Order to placing attributes as root or internal node of the tree is done by using some statistical approach.

Key Terms

- **Root Node** - It represents the entire population or sample, and this further gets divided into two or more homogeneous sets.
- **Splitting** - It is a process of dividing a node into two or more sub-nodes.
- **Decision Node** - When a sub-node splits into further sub-nodes, then it is called a decision node.
- **Leaf/ Terminal Node** - Nodes do not split is called Leaf or Terminal node.
- **Pruning -** When we remove the sub-nodes of a decision node, this process is called pruning. You can say the opposite process of splitting.
- **Branch / Sub-Tree** - A subsection of the entire tree is called branch or sub-tree.
- **Parent and Child Node** - A node, which is divided into sub-nodes is called the parent node of sub-nodes whereas sub-nodes are the child of the parent node.
- **Entropy** - A decision tree is constructed top-down from the root node and includes partitioning the data in subsets which contain similar (homogeneous) instances. To measure the homogeneity of a set, ID 3 algorithm uses entropy. If the sample are completely homogeneous the entropy is zero and it has an entropy of one if the sample is divided equally.

- **Information Gain** - The gain in information is based on an entropy decrease following an attribute split in a dataset. The decision tree is all about finding a feature which gives the highest level of information (i.e. the most uniform branch).
- **Internal Node** - An internal node (also known as an inner node, inode for short, or branch node) is any node of a tree that has child nodes
- **Branch** - The lines connecting elements are called "branches".

How to make a Decision Tree?

Step 1: Calculate the entropy of the target.

Step 2: The dataset is divided into various attributes. The entropy is determined for every unit. The total entropy for the split is then added proportionally. This entropy is replaced before the break from the entropy. The result is the Information Gain or decrease in entropy.

Step 3: Choose the attribute with the largest information gain as the decision node, divide the dataset by its branches and repeat the same process on every branch.

Advantage/Features of Decision Tree

- Decision trees require less effort for data preparation as compared to other algorithms during pre-processing.
- Do not require normalization of data.
- Do not require scaling of data as well.
- Missing values in the data also do NOT affect the process of building a decision tree to any considerable extent.
- A Decision tree model is very intuitive and easy to explain to technical teams as well as stakeholders.

Disadvantages/Shortcomings of Decision Tree

- A small change in the data can cause a large change in the structure of the decision tree causing instability.
- For a Decision tree sometimes, calculation can go far more complex compared to other algorithms.
- It often involves a higher time to train the model.
- Its training is relatively expensive because the complexity and time taken are more.
- Decision Tree algorithm is inadequate for applying regression and predicting continuous values.

Types of Decision tree

- **Categorical Variable Decision Tree**: Decision Tree which has a categorical target variable then it called a categorical variable decision tree. For Example, I say Will Bangladesh be able to beat India? so I can have the decision parameter as YES or NO.
- **Continuous Variable Decision Tree**: Decision Tree has a continuous target variable then it is called Continuous Variable Decision Tree. For Example, I say What is the percentage of chances that India will win? so here we have a varying value.
- **ID3 (Iterative Dichotomiser 3)**
- **C4.5 (successor of ID3)**
- **CART (Classification and Regression Tree)**
- **CHAID (CHi-squared Automatic Interaction Detector)**
- **MARS (Multivariate Adaptive Regression Splines)**

- **Conditional Inference Trees.**

What is the importance of a Decision Tree?

- Simple to easy, use, understand and explain
- They do feature selection and variable screening
- They require very little efforts to prepare and produce
- Can live with nonlinear relationships and parameters also
- Can use conditional probability-based reasoning or Bayes theorem
- They provide strategic answers to uncertain situations

Regression Tree vs Classification Tree

	Regression Tree	Classification Tree
Dependent Variable	Used when the dependent variable is continuous	Used when the dependent variable is categorical
Unseen Data	Makes prediction with the mean value	Makes prediction with mode values
Predictor Space	divides into a distinct and non-overlapping region	divides into a distinct and non-overlapping region
Approach Followed	Top-Down Greedy	Top-Down Greedy

Entropy and Information Gain Calculations

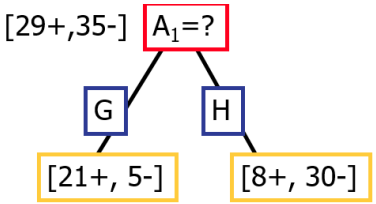
Entropy is something is very important when we must find the Information Gain, which in turn places a vital role in selecting the next attribute.

Following is the formula we will use during this chapter,

$$E(T,X) = \sum_{c \in X} P(c)E(c)$$

$$Gain(S,A) \equiv Entropy(S) - \sum_{v \in D_A} \frac{|S_v|}{|S|} Entropy(S_v)$$

Example 1



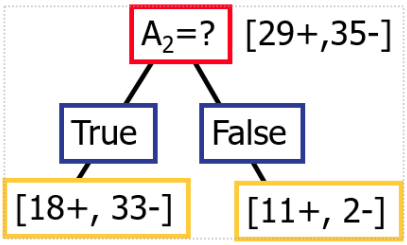
Entropy([29+,35-]) = -29/64*log2 (29/64) – 35/64*log2 (35/64) = 0.99

Entropy([21+,5-]) = -21/26*log2 (21/26) - 5/26*log2 (5/26) = 0.71

Entropy([8+,30-]) = -8/38*log2 (8/38) - 30/38 *log2 (30/38) = 0.74

Information Gain([29+,35-]),A1) = 0.99 - (26/64)*0.71 - (38/64)*0.74 = -0.15

Example 2



Entropy([29+,35-]) = -29/64*log2 (29/64) – 35/64*log2 (35/64) = 0.99

Entropy([21+,5-]) = -18/51*log2 (18/51) - 33/51*log2 (33/51) = 0.94

Entropy([8+,30-]) = -11/13*log2 (11/13) - 2/13 *log2 (2/13) = 0.62

Information Gain([29+,35-]),A1) = 0.99 - (52/64)*0.94 - (13/64)*0.62 = 0.1

So, we can see that the information gain is bigger in case of example 2(IG=0.1) as compared to example 1(IG=-0.15)

Decision Tree Example

Till now we studied theory, now let's try out some hands-on. I will take a demo dataset and will construct a decision tree based upon that dataset.

I am taking the following dataset, where we make a decision on whether we will play or not based upon the given environmental conditions.

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Weak	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Strong	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Calculation of Gini Index for "Outlook" feature

1. We can see that Outlook has 5 instances (5/14) as "Sunny", 4 instances (4/14) as "Overcast" and 5 instances as "Rain".

2. Outlook = Sunny, Play Tennis = Yes, we have 2 instances (2/5)
Outlook = Sunny, Play Tennis = No, we have 3 instances (3/5)
Hence, the Gini Index comes out to be:
= 1 - ((2/5)^2+(3/5)^2)
= 0.48

3. Outlook = Overcast, Play Tennis = Yes, we have 4 instances (4/4)
Outlook = Overcast, Play Tennis = No, we have 0 instances (0/4)
Hence, the Gini Index comes out to be:
= 1 - ((4/4)^2+(0/4)^2)
= 0

4. Outlook = Rain, Play Tennis = Yes, we have 3 instances (3/5)
Outlook = Rain, Play Tennis = No, we have 2 instances (2/5)
Hence, the Gini Index comes out to be:
= 1 - ((3/5)^2+(2/5)^2)
= 0.48

5. So, the final Gini Index is:
= (5/14)*0.48 + (4/14)*0 + (5/14)*0.48
= 0.34

Calculations of Gini Index of "Temperature" feature

1. We can see that, Temperature has 4 instances (4/14) as "Hot", 4 instances (4/14) as "Cool", 6 instances (6/14) as "Mild"

2. Temperature = Hot, Play Tennis = Yes, we have 2 instances (2/4)
Temperature = Hot, Play Tennis = No, we have 3 instances (2/4)
Hence, the Gini Index comes out to be:
= 1 - ((2/4)^2+(2/4)^2)
= 0.5

3. Temperature = Cool, Play Tennis = Yes, we have 3 instances (3/4)
Temperature = Cool, Play Tennis = No, we have 1 instance (1/4)
Hence, the Gini Index comes out to be:
= 1 - ((3/4)^2+(1/4)^2)
= 0.375

4. Temperature = Mild, Play Tennis = Yes, we have 4 instances (4/6)
Temperature = Mild, Play Tennis = No, we have 2 instances (2/6)
Hence, the Gini Index comes out to be:
= 1 - ((4/6)^2+(2/6)^2)
= 0.44

5. So So,e final Gini Index is:
= (4/14)*0.5 + (4/14)*0.375 + (6/14)*0.44
= 0.44

Calculations of Gini Index of "Humidity" feature

1. We can see that, Humidity has 7 instances (7/14) as "High", 7 instances (7/14) as "Normal"

2. Humidity = High, Play Tennis = Yes, we have 3 instances (3/7)
Humidity = High, Play Tennis = No, we have 4 instances (4/7)
Hence, the Gini Index comes out to be:
= 1 - ((3/7)^2+(4/7)^2)
= 0.49

3. Humidity = Normal, Play Tennis = Yes, we have 6 instances (6/7)
Humidity = Normal, Play Tennis = No, we have 1 instance (1/7)
Hence, the Gini Index comes out to be:
= 1 - ((6/7)^2+(1/7)^2)
= 0.24

4. So, the final Gini Index is:
= (7/14)*0.49 + (7/14)*0.24
= 0.365

Calculations of Gini Index of "Wind" feature

1. We can see that, Wind has 8 instances (8/14) as "Weak", 6 instances (6/14) as "Strong"

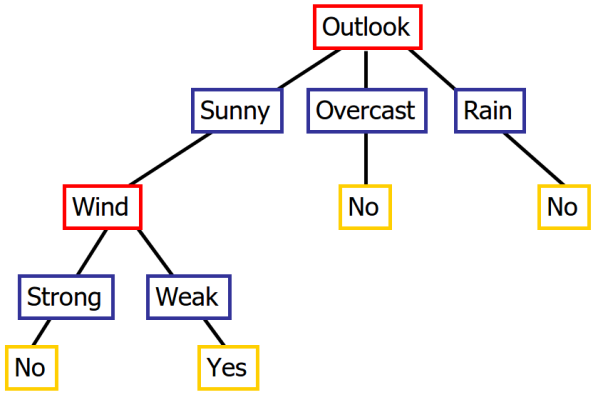
2. Wind = Strong, Play Tennis = Yes, we have 3 instances (3/6)
Wind = Strong, Play Tennis = No, we have 4 instances (3/6)
Hence, the Gini Index comes out to be:
= 1 - ((3/6)^2+(3/6)^2)
= 0.5

3. Wind = Weak, Play Tennis = Yes, we have 6 instances (6/8)
Wind = Weak, Play Tennis = No, we have 2 instances (2/8)
Hence, the Gini Index comes out to be:
= 1 - ((6/8)^2+(2/8)^2)
= 0.375

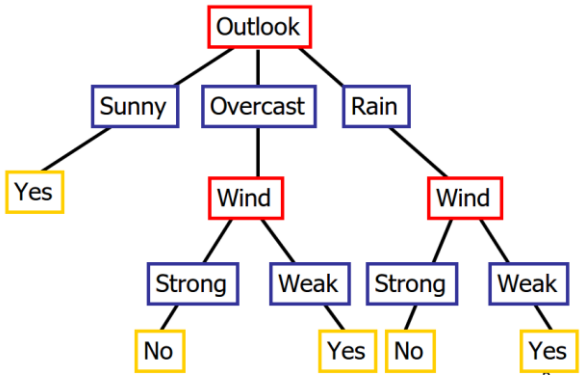
4. So, the final Gini Index is:
= (8/14)*0.375 + (6/14)*0.5
= 0.43

Since the Gini Index of Outlook is the smallest, we choose "outlook"

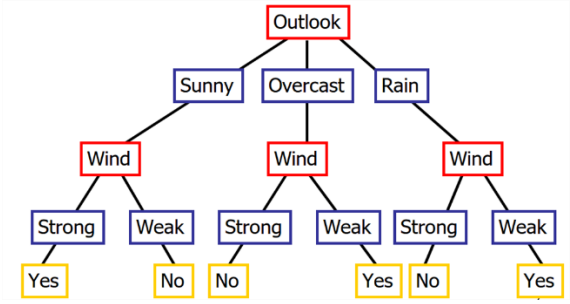
Decision Tree using Conjunction (AND Operation)



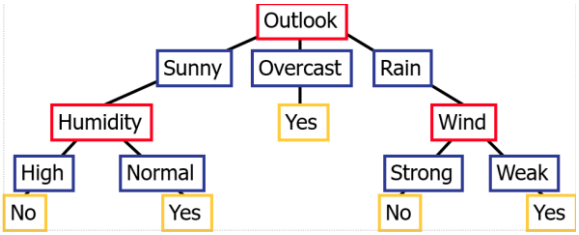
Decision Tree using Disjunction (OR Operations)



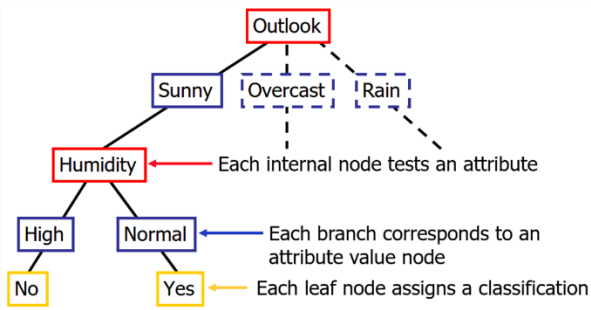
Decision Tree using XOR



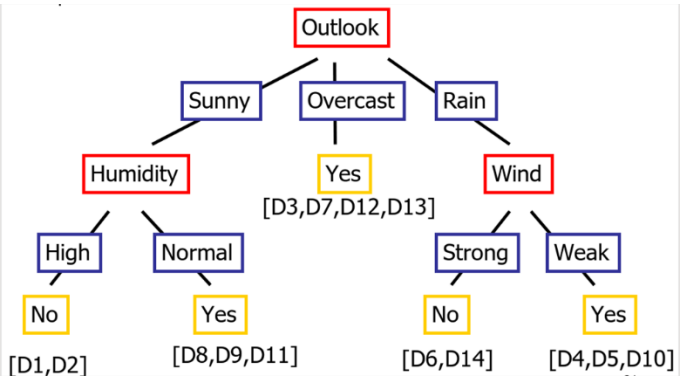
Decision Tree using Disjunction of Conjunctions (OR of AND Operations)



Let us look at a rough decision tree, to look at how exactly the decision tree will look like

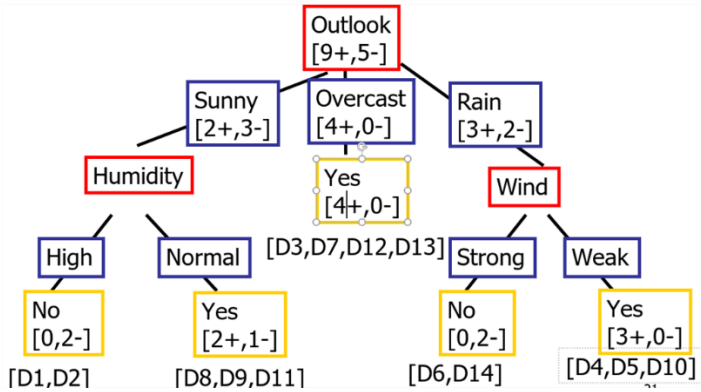


From the above trees we can't clearly infer anything, so let me show you how exactly the Decision Tree will look for the given dataset and then we will try to understand what exactly that tree means and how to use the tree to our benefit.



You may be wondering why did we take "Outlook" as root node", it is because of the Gini Index value, calculated earlier

Following is the Decision Tree with the Weights which can be used to calculate entropy and information gain.



Now let's try to understand the tree,

R1: If (Outlook=Sunny) _| (Humidity=High) Then PlayTennis=No
The above statement means that if Outlook is Sunny and Humidity is high, then we do not play

R2: If (Outlook=Sunny) _| (Humidity=Normal) Then PlayTennis=Yes
The above statement means that if Outlook is Sunny and Humidity is normal, then we play

R3: If (Outlook=Overcast) Then PlayTennis=Yes
The above statement means that if Outlook is Overcast, then we play

R4: If (Outlook=Rain) -| (Wind=Strong) Then PlayTennis=Yes

The above statement means that if Outlook is Rain or Wild is Strong, then we play

Python Implementation of Decision Tree

Let's take the example of the UCI Zoo Dataset, you can directly import it from the sklearn dataset repository. Feel free to use any dataset, there are some very good datasets available on kaggle and with Google Colab.

Using NumPy

```
import pandas as pd
import numpy as np
from pprint import pprint

# Import the dataset and define the feature as well as the target
datasets / columns
dataset = pd.read_csv('zoo.csv',

names=['animal_name','hair','feathers','eggs','milk',

'airborne','aquatic','predator','toothed','backbone',

'breathes','venomous','fins','legs','tail','domestic','catsize','class'
])

# We drop the animal names since this is not a good feature to split
the data on
dataset = dataset.drop('animal_name', axis=1)
```

```
def entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    entropy = np.sum([(-counts[i]/np.sum(counts)) *
np.log2(counts[i]/np.sum(counts)) for i in range(len(elements))])
    return entropy

def InfoGain(data, split_attribute_name, target_name="class"):
    # Calculate the entropy of the total dataset
    total_entropy = entropy(data[target_name])

    # Calculate the entropy of the dataset
    vals, counts = np.unique(data[split_attribute_name],
return_counts=True)

    # Calculate the weighted entropy
    Weighted_Entropy = np.sum([(counts[i]/np.sum(counts)) *
entropy(data.where(data[split_attribute_name]==vals[i]).dropna()[target
_name]) for i in range(len(vals))])

    # Calculate the information gain
    Information_Gain = total_entropy - Weighted_Entropy
    return Information_Gain

def ID3(data, originaldata, features, target_attribute_name="class",
parent_node_class=None):
    # Define the stopping criteria
    # If all target_values have the same value, return this value
    if len(np.unique(data[target_attribute_name])) <= 1:
        return np.unique(data[target_attribute_name])[0]

    # If the dataset is empty, return the mode target feature value in
the original dataset
    elif len(data) == 0:
        return
np.unique(originaldata[target_attribute_name])[np.argmax(np.unique(orig
inaldata[target_attribute_name], return_counts=True)[1])]

    # If the feature space is empty, return the mode target feature
value of the direct parent node
    elif len(features) == 0:
        return parent_node_class

    # Grow the tree
    else:
        # Set the default value for this node
        parent_node_class =
np.unique(data[target_attribute_name])[np.argmax(np.unique(data[target_
attribute_name], return_counts=True)[1])]
```

```
        # Select the feature which best splits the dataset
        item_values = [InfoGain(data, feature, target_attribute_name)
for feature in features]
        best_feature_index = np.argmax(item_values)
        best_feature = features[best_feature_index]

        # Create the tree structure
        tree = {best_feature: {}}

        # Remove the best feature from the feature space
        features = [i for i in features if i != best_feature]

        # Grow branches for each value of the best feature
        for value in np.unique(data[best_feature]):
            # Split the dataset along the value of the feature
            sub_data = data.where(data[best_feature] == value).dropna()

            # Recursively call ID3 for each sub dataset
            subtree = ID3(sub_data, dataset, features,
target_attribute_name, parent_node_class)

            # Add the subtree to the tree
            tree[best_feature][value] = subtree

        return tree

def predict(query, tree, default=1):
    for key in list(query.keys()):
        if key in list(tree.keys()):
            try:
                result = tree[key][query[key]]
            except:
                return default
            result = tree[key][query[key]]

            if isinstance(result, dict):
                return predict(query, result)
            else:
                return result

def train_test_split(dataset):
    training_data = dataset.iloc[:80].reset_index(drop=True)
    testing_data = dataset.iloc[80:].reset_index(drop=True)
    return training_data, testing_data

training_data = train_test_split(dataset)[0]
testing_data = train_test_split(dataset)[1]
```

```
def test(data, tree):
    queries = data.iloc[:, :-1].to_dict(orient="records")
    predicted = pd.DataFrame(columns=["predicted"])

    for i in range(len(data)):
        predicted.loc[i, "predicted"] = predict(queries[i], tree, 1.0)

    print('The prediction accuracy is: ',
(np.sum(predicted["predicted"] == data["class"]) / len(data)) * 100,
'%')

tree = ID3(training_data, training_data, training_data.columns[:-1])
pprint(tree)
test(testing_data, tree)
```

the output that I got is

{'legs': {0: {'fins': {0.0: {'toothed': {0.0: 7.0, 1.0: 3.0}},
1.0: {'eggs': {0.0: 1.0, 1.0: 4.0}}}},
2: {'hair': {0.0: 2.0, 1.0: 1.0}},
4: {'hair': {0.0: {'toothed': {0.0: 7.0, 1.0: 5.0}}, 1.0: 1.0}},
6: {'aquatic': {0.0: 6.0, 1.0: 7.0}}, 8: 7.0}}

The prediction accuracy is: 85.71428571428571 %

Using SKLearn

```
# Import the DecisionTreeClassifier
from sklearn.tree import DecisionTreeClassifier

# Import the dataset
dataset = pd.read_csv('zoo.csv',

names=['animal_name','hair','feathers','eggs','milk',

'airborne','aquatic','predator','toothed','backbone',

'breathes','venomous','fins','legs','tail','domestic','catsize','class'
])

# We drop the animal names since this is not a good feature to split
the data on
dataset = dataset.drop('animal_name', axis=1)

train_features = dataset.iloc[:80, :-1]
test_features = dataset.iloc[80:, :-1]
train_targets = dataset.iloc[:80, -1]
test_targets = dataset.iloc[80:, -1]
```

```
tree = DecisionTreeClassifier(criterion='entropy').fit(train_features,
train_targets)

prediction = tree.predict(test_features)

print("The prediction accuracy is: ", tree.score(test_features,
test_targets) * 100, "%")
```

The output that I got is:

The prediction accuracy is: 80.95238095238095 %

Using TensorFlow

```
import numpy as np
import pandas as pd
import tensorflow as tf
from functools import reduce
import matplotlib.pyplot as plt
%matplotlib inline

np.random.seed(1943)
tf.set_random_seed(1943)

iris = [((5.1, 3.5, 1.4, 0.2), (1, 0, 0)),
        ((4.9, 3.0, 1.4, 0.2), (1, 0, 0)),
        ((4.7, 3.2, 1.3, 0.2), (1, 0, 0)),
        ((4.6, 3.1, 1.5, 0.2), (1, 0, 0)),
        ((5.0, 3.6, 1.4, 0.2), (1, 0, 0)),
        ((5.4, 3.9, 1.7, 0.4), (1, 0, 0)),
        ((4.6, 3.4, 1.4, 0.3), (1, 0, 0)),
        ((5.0, 3.4, 1.5, 0.2), (1, 0, 0)),
        ((4.4, 2.9, 1.4, 0.2), (1, 0, 0)),
        ((4.9, 3.1, 1.5, 0.1), (1, 0, 0)),
        ((5.4, 3.7, 1.5, 0.2), (1, 0, 0)),
        ((4.8, 3.4, 1.6, 0.2), (1, 0, 0)),
        ((4.8, 3.0, 1.4, 0.1), (1, 0, 0)),
        ((4.3, 3.0, 1.1, 0.1), (1, 0, 0)),
        ((5.8, 4.0, 1.2, 0.2), (1, 0, 0)),
        ((5.7, 4.4, 1.5, 0.4), (1, 0, 0)),
        ((5.4, 3.9, 1.3, 0.4), (1, 0, 0)),
        ((5.1, 3.5, 1.4, 0.3), (1, 0, 0)),
        ((5.7, 3.8, 1.7, 0.3), (1, 0, 0)),
        ((5.1, 3.8, 1.5, 0.3), (1, 0, 0)),
        ((5.4, 3.4, 1.7, 0.2), (1, 0, 0)),
        ((5.1, 3.7, 1.5, 0.4), (1, 0, 0)),
        ((4.6, 3.6, 1.0, 0.2), (1, 0, 0)),
        ((5.1, 3.3, 1.7, 0.5), (1, 0, 0)),
        ((4.8, 3.4, 1.9, 0.2), (1, 0, 0)),
        ((5.0, 3.0, 1.6, 0.2), (1, 0, 0)),
```

```
        ((5.0, 3.4, 1.6, 0.4), (1, 0, 0)),
        ((5.2, 3.5, 1.5, 0.2), (1, 0, 0)),
        ((5.2, 3.4, 1.4, 0.2), (1, 0, 0)),
        ((4.7, 3.2, 1.6, 0.2), (1, 0, 0)),
        ((4.8, 3.1, 1.6, 0.2), (1, 0, 0)),
        ((5.4, 3.4, 1.5, 0.4), (1, 0, 0)),
        ((5.2, 4.1, 1.5, 0.1), (1, 0, 0)),
        ((5.5, 4.2, 1.4, 0.2), (1, 0, 0)),
        ((4.9, 3.1, 1.5, 0.1), (1, 0, 0)),
        ((5.0, 3.2, 1.2, 0.2), (1, 0, 0)),
        ((5.5, 3.5, 1.3, 0.2), (1, 0, 0)),
        ((4.9, 3.1, 1.5, 0.1), (1, 0, 0)),
        ((4.4, 3.0, 1.3, 0.2), (1, 0, 0)),
        ((5.1, 3.4, 1.5, 0.2), (1, 0, 0)),
        ((5.0, 3.5, 1.3, 0.3), (1, 0, 0)),
        ((4.5, 2.3, 1.3, 0.3), (1, 0, 0)),
        ((4.4, 3.2, 1.3, 0.2), (1, 0, 0)),
        ((5.0, 3.5, 1.6, 0.6), (1, 0, 0)),
        ((5.1, 3.8, 1.9, 0.4), (1, 0, 0)),
        ((4.8, 3.0, 1.4, 0.3), (1, 0, 0)),
        ((5.1, 3.8, 1.6, 0.2), (1, 0, 0)),
        ((4.6, 3.2, 1.4, 0.2), (1, 0, 0)),
        ((5.3, 3.7, 1.5, 0.2), (1, 0, 0)),
        ((5.0, 3.3, 1.4, 0.2), (1, 0, 0)),
        ((7.0, 3.2, 4.7, 1.4), (0, 1, 0)),
        ((6.4, 3.2, 4.5, 1.5), (0, 1, 0)),
        ((6.9, 3.1, 4.9, 1.5), (0, 1, 0)),
        ((5.5, 2.3, 4.0, 1.3), (0, 1, 0)),
        ((6.5, 2.8, 4.6, 1.5), (0, 1, 0)),
        ((5.7, 2.8, 4.5, 1.3), (0, 1, 0)),
        ((6.3, 3.3, 4.7, 1.6), (0, 1, 0)),
        ((4.9, 2.4, 3.3, 1.0), (0, 1, 0)),
        ((6.6, 2.9, 4.6, 1.3), (0, 1, 0)),
        ((5.2, 2.7, 3.9, 1.4), (0, 1, 0)),
        ((5.0, 2.0, 3.5, 1.0), (0, 1, 0)),
        ((5.9, 3.0, 4.2, 1.5), (0, 1, 0)),
        ((6.0, 2.2, 4.0, 1.0), (0, 1, 0)),
        ((6.1, 2.9, 4.7, 1.4), (0, 1, 0)),
        ((5.6, 2.9, 3.6, 1.3), (0, 1, 0)),
        ((6.7, 3.1, 4.4, 1.4), (0, 1, 0)),
        ((5.6, 3.0, 4.5, 1.5), (0, 1, 0)),
        ((5.8, 2.7, 4.1, 1.0), (0, 1, 0)),
        ((6.2, 2.2, 4.5, 1.5), (0, 1, 0)),
        ((5.6, 2.5, 3.9, 1.1), (0, 1, 0)),
        ((5.9, 3.2, 4.8, 1.8), (0, 1, 0)),
        ((6.1, 2.8, 4.0, 1.3), (0, 1, 0)),
        ((6.3, 2.5, 4.9, 1.5), (0, 1, 0)),
        ((6.1, 2.8, 4.7, 1.2), (0, 1, 0)),
```

```
        ((6.4, 2.9, 4.3, 1.3), (0, 1, 0)),
        ((6.6, 3.0, 4.4, 1.4), (0, 1, 0)),
        ((6.8, 2.8, 4.8, 1.4), (0, 1, 0)),
        ((6.7, 3.0, 5.0, 1.7), (0, 1, 0)),
        ((6.0, 2.9, 4.5, 1.5), (0, 1, 0)),
        ((5.7, 2.6, 3.5, 1.0), (0, 1, 0)),
        ((5.5, 2.4, 3.8, 1.1), (0, 1, 0)),
        ((5.5, 2.4, 3.7, 1.0), (0, 1, 0)),
        ((5.8, 2.7, 3.9, 1.2), (0, 1, 0)),
        ((6.0, 2.7, 5.1, 1.6), (0, 1, 0)),
        ((5.4, 3.0, 4.5, 1.5), (0, 1, 0)),
        ((6.0, 3.4, 4.5, 1.6), (0, 1, 0)),
        ((6.7, 3.1, 4.7, 1.5), (0, 1, 0)),
        ((6.3, 2.3, 4.4, 1.3), (0, 1, 0)),
        ((5.6, 3.0, 4.1, 1.3), (0, 1, 0)),
        ((5.5, 2.5, 4.0, 1.3), (0, 1, 0)),
        ((5.5, 2.6, 4.4, 1.2), (0, 1, 0)),
        ((6.1, 3.0, 4.6, 1.4), (0, 1, 0)),
        ((5.8, 2.6, 4.0, 1.2), (0, 1, 0)),
        ((5.0, 2.3, 3.3, 1.0), (0, 1, 0)),
        ((5.6, 2.7, 4.2, 1.3), (0, 1, 0)),
        ((5.7, 3.0, 4.2, 1.2), (0, 1, 0)),
        ((5.7, 2.9, 4.2, 1.3), (0, 1, 0)),
        ((6.2, 2.9, 4.3, 1.3), (0, 1, 0)),
        ((5.1, 2.5, 3.0, 1.1), (0, 1, 0)),
        ((5.7, 2.8, 4.1, 1.3), (0, 1, 0)),
        ((6.3, 3.3, 6.0, 2.5), (0, 0, 1)),
        ((5.8, 2.7, 5.1, 1.9), (0, 0, 1)),
        ((7.1, 3.0, 5.9, 2.1), (0, 0, 1)),
        ((6.3, 2.9, 5.6, 1.8), (0, 0, 1)),
        ((6.5, 3.0, 5.8, 2.2), (0, 0, 1)),
        ((7.6, 3.0, 6.6, 2.1), (0, 0, 1)),
        ((4.9, 2.5, 4.5, 1.7), (0, 0, 1)),
        ((7.3, 2.9, 6.3, 1.8), (0, 0, 1)),
        ((6.7, 2.5, 5.8, 1.8), (0, 0, 1)),
        ((7.2, 3.6, 6.1, 2.5), (0, 0, 1)),
        ((6.5, 3.2, 5.1, 2.0), (0, 0, 1)),
        ((6.4, 2.7, 5.3, 1.9), (0, 0, 1)),
        ((6.8, 3.0, 5.5, 2.1), (0, 0, 1)),
        ((5.7, 2.5, 5.0, 2.0), (0, 0, 1)),
        ((5.8, 2.8, 5.1, 2.4), (0, 0, 1)),
        ((6.4, 3.2, 5.3, 2.3), (0, 0, 1)),
        ((6.5, 3.0, 5.5, 1.8), (0, 0, 1)),
        ((7.7, 3.8, 6.7, 2.2), (0, 0, 1)),
        ((7.7, 2.6, 6.9, 2.3), (0, 0, 1)),
        ((6.0, 2.2, 5.0, 1.5), (0, 0, 1)),
        ((6.9, 3.2, 5.7, 2.3), (0, 0, 1)),
        ((5.6, 2.8, 4.9, 2.0), (0, 0, 1)),
```

```
        ((7.7, 2.8, 6.7, 2.0), (0, 0, 1)),
        ((6.3, 2.7, 4.9, 1.8), (0, 0, 1)),
        ((6.7, 3.3, 5.7, 2.1), (0, 0, 1)),
        ((7.2, 3.2, 6.0, 1.8), (0, 0, 1)),
        ((6.2, 2.8, 4.8, 1.8), (0, 0, 1)),
        ((6.1, 3.0, 4.9, 1.8), (0, 0, 1)),
        ((6.4, 2.8, 5.6, 2.1), (0, 0, 1)),
        ((7.2, 3.0, 5.8, 1.6), (0, 0, 1)),
        ((7.4, 2.8, 6.1, 1.9), (0, 0, 1)),
        ((7.9, 3.8, 6.4, 2.0), (0, 0, 1)),
        ((6.4, 2.8, 5.6, 2.2), (0, 0, 1)),
        ((6.3, 2.8, 5.1, 1.5), (0, 0, 1)),
        ((6.1, 2.6, 5.6, 1.4), (0, 0, 1)),
        ((7.7, 3.0, 6.1, 2.3), (0, 0, 1)),
        ((6.3, 3.4, 5.6, 2.4), (0, 0, 1)),
        ((6.4, 3.1, 5.5, 1.8), (0, 0, 1)),
        ((6.0, 3.0, 4.8, 1.8), (0, 0, 1)),
        ((6.9, 3.1, 5.4, 2.1), (0, 0, 1)),
        ((6.7, 3.1, 5.6, 2.4), (0, 0, 1)),
        ((6.9, 3.1, 5.1, 2.3), (0, 0, 1)),
        ((5.8, 2.7, 5.1, 1.9), (0, 0, 1)),
        ((6.8, 3.2, 5.9, 2.3), (0, 0, 1)),
        ((6.7, 3.3, 5.7, 2.5), (0, 0, 1)),
        ((6.7, 3.0, 5.2, 2.3), (0, 0, 1)),
        ((6.3, 2.5, 5.0, 1.9), (0, 0, 1)),
        ((6.5, 3.0, 5.2, 2.0), (0, 0, 1)),
        ((6.2, 3.4, 5.4, 2.3), (0, 0, 1)),
        ((5.9, 3.0, 5.1, 1.8), (0, 0, 1))]

feature = np.vstack([np.array(i[0]) for i in iris])
label = np.vstack([np.array(i[1]) for i in iris])

x = feature[:, 2:4]  # use "Petal length" and "Petal width" only
y = label
d = x.shape[1]

def tf_kron_prod(a, b):
    res = tf.einsum('ij,ik->ijk', a, b)
    res = tf.reshape(res, [-1, tf.reduce_prod(res.shape[1:])])
    return res

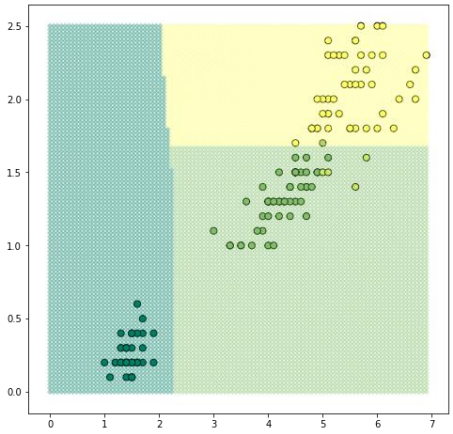
def tf_bin(x, cut_points, temperature=0.1):
    # x is a N-by-1 matrix (column vector)
    # cut_points is a D-dim vector (D is the number of cut-points)
    # this function produces a N-by-(D+1) matrix, each row has only one
element being one and the rest are all zeros
    D = cut_points.get_shape().as_list()[0]
    W = tf.reshape(tf.linspace(1.0, D + 1.0, D + 1), [1, -1])
```

```
    cut_points = tf.contrib.framework.sort(cut_points)  # make sure
cut_points is monotonically increasing
    b = tf.cumsum(tf.concat([tf.constant(0.0, shape=[1]), -cut_points],
0))
    h = tf.matmul(x, W) + b
    res = tf.nn.softmax(h / temperature)
    return res

def nn_decision_tree(x, cut_points_list, leaf_score, temperature=0.1):
    # cut_points_list contains the cut_points for each dimension of
feature
    leaf = reduce(tf_kron_prod,
                  map(lambda z: tf_bin(x[:, z[0]:z[0] + 1], z[1],
temperature), enumerate(cut_points_list)))
    return tf.matmul(leaf, leaf_score)
num_cut = [1, 1]  # "Petal length" and "Petal width"
num_leaf = np.prod(np.array(num_cut) + 1)
num_class = 3

sess = tf.InteractiveSession()
x_ph = tf.placeholder(tf.float32, [None, d])
y_ph = tf.placeholder(tf.float32, [None, num_class])
cut_points_list = [tf.Variable(tf.random_uniform([i])) for i in
num_cut]
leaf_score = tf.Variable(tf.random_uniform([num_leaf, num_class]))
y_pred = nn_decision_tree(x_ph, cut_points_list, leaf_score,
temperature=0.1)
loss = tf.reduce_mean(tf.losses.softmax_cross_entropy(logits=y_pred,
onehot_labels=y_ph))
opt = tf.train.AdamOptimizer(0.1)
train_step = opt.minimize(loss)
sess.run(tf.global_variables_initializer())
for i in range(1000):
    _, loss_e = sess.run([train_step, loss], feed_dict={x_ph: x, y_ph:
y})
    if i % 200 == 0:
        print(loss_e)
print('error rate %.2f' % (1 -
np.mean(np.argmax(y_pred.eval(feed_dict={x_ph: x}), axis=1) ==
np.argmax(y, axis=1))))
sample_x0 = np.repeat(np.linspace(0, np.max(x[:,0]), 100),
100).reshape(-1,1)
sample_x1 = np.tile(np.linspace(0, np.max(x[:,1]), 100).reshape(-1,1),
[100,1])
sample_x = np.hstack([sample_x0, sample_x1])
sample_label = np.argmax(y_pred.eval(feed_dict={x_ph: sample_x}),
axis=1)
plt.figure(figsize=(8,8))
```

```
plt.scatter(x[:,0],
            x[:,1],
            c=np.argmax(y, axis=1),
            marker='o',
            s=50,
            cmap='summer',
            edgecolors='black')
plt.scatter(sample_x0.flatten(),
            sample_x1.flatten(),
            c=sample_label.flatten(),
            marker='D',
            s=20,
            cmap='summer',
            edgecolors='none',
            alpha=0.33)
```



The output that I got is

1.1529802
0.32481167
0.1562062
0.15097024
0.118773825
error rate 0.04

Conclusion

In this chapter, we studied what is decision tree, when is decision tree used, assumptions of decision tree, key terms, how to make decision tree, advantages of decision tree, disadvantages of decision tree, types of decision tree, what is the importance of decision tree, regression tree vs classification tree, entropy, information gain and gini index calculations, decision tree example, python implementation of decision tree using sklearn, numpy, and TensorFlow. Hope you were able to understand each and everything.

8

Naïve Base

Overview

In this chapter, we delve into the essentials of probability theory and random variables. We cover key concepts like deterministic and probabilistic experiments, types of events, and probability distributions. Explore practical applications, including Bayes' Theorem and Naive Bayes Classifier, to understand how to handle uncertainty and make informed predictions in various scenarios.

Key terms

Coin: A coin has two sides, Head and Tail. If an event consists of more than one coin, then coins are considered as distinct, if not otherwise stated.

Die: The die has six faces marked 11, 2, 3, 4, 5 and 6. If we have more than one dice, then all dice are considered as distinct, if not otherwise stated.

Playing Cards: A pack of playing cards has 52 cards. There are 4 suits (spade, heart, diamond, and club) each having 13 cards. There are two colors, red (heart and diamond) and black (spade and club) each having 26 cards. In 13 cards of each suit, there are 3 face cards namely king, queen and jack so there are in all ’12 face cards. Also, there are 16 honor cards, 4 of each suit namely ace, king, queen, and jack.

Types of Experiments

1. **Deterministic Experiment** Those experiments, which when repeated under identical conditions produce the same result or outcome are known as a deterministic experiment,
2. **Probabilistic/Random Experiment** Those experiments, which when repeated under identical conditions, do not produce the same outcome every time but the outcome in a trial is one of the several possible outcomes called a random experiment.

Important Definitions

- Trial Let a random experiment, be repeated under identical conditions, then the experiment is called a Trial.
- Sample Space The set of all possible outcomes of an experiment is called the sample space of the experiment and it is denoted by S.
- Event A subset of the sample space associated with a random experiment is called an event or case.
- Sample Points The outcomes of an experiment is called the sample point.
- Certain Event An event that must occur, whatever be the outcomes, is called a certain or sure event.
- Impossible Event An event that cannot occur in a particular random experiment is called an impossible event.
- Elementary Event An event certainly only one sample point is called elementary event or indecomposable events.
- Favorable Event Let S be the sample space associated with a random experiment and let E ⊂ S. Then, the elementary events belonging to E are known as the favorable event to E.
- Compound Events An event certainly more than one sample point is called compound events or decomposable events.

Probability

If there are n elementary events associated with a random experiment and m of them are favorable to an event A, then the probability of happening or occurrence of A, denoted by P(A), is given by P(A) = m / n = Number of favorable cases / Total number of possible cases

Types of Events

Equally likely events

The given events are said to be equally likely if none of them is expected to occur in preference to the other.

Mutually exclusive events

A set of events is said to be mutually exclusive if the happening of one excludes the happening of the other. If A and B are mutually exclusive, then P(A ∩ B) = 0

Exhaustive events

A set of events is said to be exhaustive if the performance of the experiment always results in the occurrence of at least one of them. If E1, E2, …, En are exhaustive events, then El ∪ E2 ∪ … ∪ En = S i.e., P(E1 ∪ E2 ∪ E3 ∪ … ∪ En) = 1

Independent events

Two events A and B associated with a random experiment are independent if the probability of occurrence or non-occurrence of A is not affected by the occurrence or non-occurrence of B. i.e., P(A ∩ B) = P(A) P(B)

The complement of an Event

Let A be an event in a sample space S~the complement of A is the set of all sample points of the space other than the sample point in A and it is denoted by, A' or A = {n : n ∈ S, n ∉ A}
(i) P(A ∪ A') = S
(ii) P(A ∩ A') = φ (iii) P(A')' = A

Partition of a Sample Space

The events A1, A2…., An represent a partition of the sample space S, if they are pairwise disjoint, exhaustive and have non-zero probabilities. i.e.,
(i) Ai ∩ Aj = φ; i ≠ j; i,j= 1,2, …. , n
(ii) A1 ∪ A2 ∪ … ∪ An = S
(iii) P(Ai) > 0, ∀ i = 1,2, …., n

Important Results on Probability

(i) If a set of events A1, A2…., An are mutually exclusive, then
A1 ∩ A2 ∩ A3 ∩ …∩ An = φ
P(A1 ∪ A2 ∪ A3 ∪… ∪ An) = P(A1) + (A2) + … + P(An) and A1 ∩ A2 ∩ A3 ∩ …∩ An = 0

(ii) If a set of events A1, A2…., An are exhaustive, then P(A1 ∪ A2 ∪ … ∪ An) = 1

(iii) The probability of an impossible event is O. i.e., P(A) = 0 if A is impossible event.,

(iv) Probability of any event in a sample space is 1. i.e., P(A) = 1

(v) Odds in favor of A = P(A) / P(A)

(vi) Odds in Against of A = P(A) / P(A)

(vii) Addition Theorem of Probability

(a) For two events A and B
P(A ∪ B) = P(A) + P(B) – P(A ∩ B)

(b) For three events A, B and C
P(A ∪ B ∪ C) = P(A) + P(B) + P(C) -P(A ∩ B) – P(B ∩ C) – P(A ∩ C) + P(A ∩ B ∩ C) (c) For n events A1, A2, An

(viii) If A and B are two events, then P(A ∩ B) ≤ P(A) ≤ P(A ∪ B) ≤ P(A) + P(B)

(ix) If A and B are two events associated with a random experiment, then

(a) $P(A \cap B) = P(B) – P(A \cap B)$
(b) $P(A \cap B) = P(A) – P(A \cap B)$
(c)$P [(A \cap B) \cup (A \cap B)] = P(A) + P(B) – 2P(A \cap B)$
(d) $P(A \cap B) = 1- P(A \cup B)$
(e) $P(A \cup B) = 1- P(A \cap B)$
(f) $P(A) = P(A \cap B) + P(A \cap B)$.
(g) $P(B) = P(A \cap B) + P(B \cap A)$

(x)
(a) P (exactly one of A, B occurs) $= P(A) + P(B) – 2P(A \cap B) = P(A \cup B) – P(A \cap B)$
(b) P(neither A nor B) $= P(A' \cap B') = 1 – P(A \cup B)$

(xi) If A, B and C are three events, then
(a) P(exactly one of A, B, C occurs) $= P(A) + P(B) + P(C) – 2P(A \cap B) – 2P(B \cap C) – 2P(A \cap C) + 3P(A \cap B \cap C)$
(b) P (at least two of A, B, C occurs) $= P(A \cap B) + P(B \cap C) + P(C \cap A) – 2P(A \cap B \cap C)$
(c) P (exactly two of A, B, C occurs) . $= P(A \cap B) + P(B \cap C) + P(A \cap C) – 3P(A \cap B \cap C)$

(xii)
(a) $P(A \cup B) = P(A) + P(B)$, if A and B are mutually exclusive events.
(b) $P(A \cup B \cup C) = P(A) + P(B) + P(C)$, if A, Band C are mutually exclusive events.

(xiii) $P(A) = 1- P(A)$

(xiv) $P(A \cup B) = P(S) = 1, P(\varphi) = 0$

(xv) $P(A \cap B) = P(A) \times P(B)$, if A and B are independent events.

(xvi) If A and B are independent events associated with a random experiment, then

(a) A and B are independent events.
(b) A and B are independent events. (c) A and B are independent events.

(xvii) If A1, A2…., An are independent events associated with a random experiment, the probability of occurrence of at least one

$= P(A1 \cup A2 \cup …. \cup An)$
$= 1 – P(A1 \cup A2 \cup …. \cup An)$
$= 1 – P(A1)P(A2)…P(An)$

(xviii) If $B \subseteq A$, then $P(A \cap B) = P(A) – P(B)$

Conditional Probability

Let A and B be two events associated with a random experiment, then, the probability of occurrence of event A under the condition that B has already occurred and $P(B) \neq 0$, is called the conditional probability.
i.e., $P(A/B) = P(A \cap B) / P(B)$

If A has already occurred and $P (A) \neq 0$, then
$P(B/A) = P(A \cap B) / P(A)$

Also, $P(A / B) + P (A / B) = 1$

Multiplication Theorem on Probability

(i) If A and B are two events associated with a random experiment, then
P(A ∩ B) = P(A)P(B /A), IF P(A) ≠ 0
OR
P(A ∩ B) = P(B)P(A /B), IF P(B) ≠ 0

(ii) If A1, A2…., An are n events associated with a random experiment, then
P(A1 ∩ A2 ∩…. ∩ An) = P(A1) P(A2 / A1) P(A3 / (A1 ∩ A2)) …P(An / (A1 ∩ A2 ∩ A3 ∩…∩A n – 1))

Total Probability

Let S be the sample space and let E1, E2…., En be n mutually exclusive and exhaustive events associated with a random experiment. If A is any event which occurs with E1 or E2 or … or En then

P(A) = P(E1)P(A / E1) + P(E2)P(A / E2) + … + P(En) P(A / En)

Bayes' Theorem

Let S be the sample space and let E1, E2…, En, be n mutually exclusive and exhaustive events associated with a random experiment. If A is any event which occurs with E1 or E2 or … or En then the probability of occurrence of Ei, when A occurred,

were,
P (Ei), i = 1,2, n is known as the prior probabilities
P (A / Ei), i = 1, 2, n is called the likelihood probabilities
P (Ei / A), i = 1, 2, …, n is called the posterior probabilities

$$Prosterior = \frac{Likelihood * Prior}{Evidence}$$

OR

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

where A and B are events and P (B) ≠ 0.

- P (A | B) is a conditional probability: the likelihood of event A occurring given that B is true.
- P (B | A) is also a conditional probability: the likelihood of event B occurring given that A is true.
- P (A) and P (B) are the probabilities of observing A and B independently of each other; this is known as the marginal probability.

Random Variable

Let U or S be a sample space associated with a given random experiment. A real-valued function X defined on U or S, i: e.,

X: U → R is called a random variable.

There are two types of random variables.

Discrete random variable

If the range of the real function X: U → R is a finite set or an infinite set of real numbers, it is called a discrete random variable.

Continuous Random Variable

If the range of X is an interval (a, b) of R, then X is called a continuous random variable. e.g., In tossing of two coins S = {HH, HT, TH, TT}, let X denotes the number of heads in the tossing of two coins, then X(HH) = 2, X(TH) = 1, X(TT) = 0

Probability Distribution

If a random variable X takes values X1, X2…., Xn with respective probabilities P1, P2…., Pn then is known as the probability distribution of X, or Probability distribution gives the values of the random variable along with the corresponding probabilities.

Mathematical Expectation/Mean

If X is a discrete random variable which assume values X1, X2…., Xn with respective probabilities P1, P2…., Pn then the mean x of X is defined as

$$E(X) = X = P1X1 + P2X2 + \ldots + PnXn = \Sigma ni = 1\ PiXi$$

Important Results

(i) Variance V(X) = σ2x = E(X2) – (E(X))2
where, E(X2) = Σni = 1 x2iP(xi)

(ii) Standard Deviation √V(X) = σx = √E(X2) – (E(X))2

(iii) If Y = a X + b, then
(a) E(Y) = E(aX + b) = aE(X) + b
(b) σ2y = a2V(Y) = a2σ2x
(c) σy = √V(Y) = |a|σx

(iv) If Z = aX2 + bX + c, then
E(Z) = E(aX2 + bX + c) = aE(X2) + bE(X) + c

Bayes' Theorem Explanation using an example

Let us try to understand the above formula through an example:

Question

Talking about C-SharpCorner, if a person visits C-SharpCorner, the chances of he/she revisiting are 60%, the chances of a person liking a particular chapter are 75%. The chances of a person liking the chapter and coming back are 75%. So, we need to find the probability of a person re-visiting the website given that he/she doesn't like the chapter.

Solution

A: A person re-visits the website
B: A person likes a chapter

So, P(A) = 0.6, P(A') = 0.4
P(B) = 0.75 , P(B') =0.25
P(A|B) = 0.75, P(A'|B) = 0.25

P(B|A') = P((A'|B)*P(B))/ P(A')
= (0.25*0.75)/0.4
= 0.46875

So, from the above calculations, a person will revisit ~47% time if he/she doesn't like a particular chapter

When is Naive Bayes Classifier Used?

- **Real-time prediction:** Naive Bayes Algorithm is fast and always ready to learn hence best suited for real-time predictions.
- **Multi-class prediction:** The probability of multi-classes of any target variable can be predicted using a Naive Bayes algorithm.
- **Recommendation system:** Collaborative filtering is used to create a recommendation framework for Naive Bayes classifier. The system employs data mining and machine learning techniques to filter information not previously visible and then predict whether a user would like a given resource.
- **Text classification/ Sentiment Analysis/ Spam Filtering:** Naive Bayes performed better or had a higher success rate in texts classification because of its best performance with multi class problems and its independence rule it is therefore used in sentiment analysis and spam filters.

Difference between Bayes and Naive Bayes Algorithm

In the Bayes naive class, we assume that the features are independent of class rather than modelling their complete conditionally class distribution. The Bayes classification approximates the Bayes classification. A description by Bayes is best understood as a form of assessment. Suppose we seek to estimate the class of ("classify") an observation is given a vector of features. Denote the class C and the vector of features (F1, F2…, Fk). Given a probability model underlying the data (that is, given the joint distribution of (C, F1, F2…, Fk), the Bayes classification function chooses a class by maximizing the probability of the class given the observed features: argmaxc P(C=c | F1=f1…,Fk=fk)

Advantages/Features of Naive Bayes Algorithm

- Easy to implement.
- Fast
- If the independence assumption holds then it works more efficiently than other algorithms.
- It requires less training data.
- It is highly scalable.
- It can make probabilistic predictions.
- Can handle both continuous and discrete data.
- Insensitive to irrelevant features.
- It can work easily with missing values.
- Easy to update on the arrival of new data.
- Best suited for text classification problems.

Disadvantages/Shortcomings of Naive Bayes Algorithm

- The strong assumption about the features to be independent which is hardly true in real-life applications.
- Data scarcity.
- Chances of loss of accuracy.
- Zero Frequency i.e. if the category of any categorical variable is not seen in the training data set, then the model assigns a zero probability to that category and then a prediction cannot be made.

Assumptions of Naive Bayes Algorithm

- All the features are independent, that is there are no dependencies between any of the features.
- Each of the features is given equal or the same importance or equal weight

Things to keep in mind to get the best out of Naive Bayes Algorithm

- **Categorical Inputs:** Naive Bayes assumes label attributes such as binary, categorical or nominal.
- **Gaussian Inputs:** If the input variables are evaluated in real terms, a Gaussian distribution is assumed. If the uniform distribution of your data is gaussian or near-gaussian, then the algorithm will work best. The elimination of outliers can be expected (for example, values greater than 3 or 4 defaults to mean).
- **Classification Problems:** Naive Bayes works best with binary and multiclass classification.
- **Log Probabilities:** To calculate the probabilities of different class values, many small numbers have to be multiplied together. To avoid the sub flow of numerical precision, a log transformation should be used.
- **Kernel Functions:** More complicated representations such as several kernel density functions can be used rather than using a Gaussian distribution for integer input values.
- **Update Probabilities:** When new data becomes available, you can simply update the probabilities of your model. This can be helpful if the data changes frequently.

Types of Naive Bayes Algorithm

Multinomial Naive Bayes

This is primarily used to assess whether the paper is grouped into the groups of athletics, governance, science and other. The characteristics / predictors of the type of classification are the frequency of the terms in the paper.

Bernoulli Naive Bayes

This is comparable to the Bayes multinomial, but the predictors are Boolean. For eg, the parameters used to display the variable class only support values yes or no, if a term is put or not in the code.

Gaussian Naive Bayes

We assume that these values are sampled from a Gaussian distribution if the previewers have a continued value and are not discrete.

Semi-supervised parameter estimation

Given that a naive Baye classification is trained from marked data, a semi-supervised training algorithm can be constructed which can learn from a combination of marked and unmarked data by running a supervised learning algorithm through a loop.

What is Naive Bayes?

Type	Long	Not Long	Sweet	Not Sweet	Yellow	Not Yellow	Total
Banana	400	100	350	150	450	50	500
Orange	0	300	150	150	300	0	300
Other	100	100	150	50	50	150	200
Total	500	500	650	350	800	200	1000

So, the objective of the classifier is to predict if a given fruit is a ‘Banana’ or ‘Orange’ or ‘Other’ when only the 3 features (long, sweet and yellow) are known.

So, to predict this we need to find 3 probabilities. Let’s start

1. We first calculate the "Prior" probabilities for each of the class of fruits
P[Y=Banana] = 500/1000 = 0.5
P[Y=Orange] = 300/1000 = 0.3
P[Y=Other] = 200/1000 = 0.2

2. We then compute the probability of evidence that goes in the denominator
P[x1=Long] = 500/100 = 0.5
P[x2=Sweet] = 650/100 = 0.65
P[x3=Yellow] = 800/100 = 0.8

3. Now we calculate the probability of likelihood of evidence that goes in the numerator
P[x1=Long | Y=Banana] = 400/500 = 0.8
P[x2=Sweet | Y=Banana] = 350/500 = 0.7
P[x3=Yellow | Y=Banana] = 450/500 = 0.9

4. At the end we substitute all the values in the Naive Bayes Formula,

a. P(Banana | Long, Sweet and Yellow)
= ((P(Long | Banana) * P(Sweet | Banana) * P(Yellow | Banana))*P(Banana))/ (P(Long) * P(Sweet) * P(Yellow))
= ((0.8 * 0.7 * 0.9) * 0.5)/(0.5 * 0.65 * 0.8)
= 0.97

b. P(Orange | Long, Sweet and Yellow)
= ((P(Long | Orange) * P(Sweet | Orange) * P(Yellow | Orange))*P(Orange))/ (P(Long) * P(Sweet) * P(Yellow))
= 0

c. P(Others | Long, Sweet and Yellow)
= ((P(Long | Others) * P(Sweet | Others) * P(Yellow | Others))*P(Others))/ (P(Long) * P(Sweet) * P(Yellow))
= 0.07

So, from the Naive Bayes Classifier, we predict the fruit is a Banana.

Python Implementation of Decision Tree

Let's take the example of the IRIS dataset, you can directly import it from the sklearn dataset repository or download it from the chapter. Feel free to use any dataset, there are some very good datasets available on Kaggle and with Google Colab.

Using functions

```
from csv import reader
from math import sqrt
from math import exp
from math import pi

# Load a CSV file
def load_csv(filename):
    dataset = list()
    with open(filename, 'r') as file:
```

```
        csv_reader = reader(file)
        for row in csv_reader:
            if not row:
                continue
            dataset.append(row)
    return dataset

# Convert string column to float
def str_column_to_float(dataset, column):
    for row in dataset:
        row[column] = float(row[column].strip())

# Convert string column to integer
def str_column_to_int(dataset, column):
    class_values = [row[column] for row in dataset]
    unique = set(class_values)
    lookup = dict()
    for i, value in enumerate(unique):
        lookup[value] = i
        print('[%s] => %d' % (value, i))
    for row in dataset:
        row[column] = lookup[row[column]]
    return lookup

# Split the dataset by class values, returns a dictionary
def separate_by_class(dataset):
    separated = dict()
    for i in range(len(dataset)):
        vector = dataset[i]
        class_value = vector[-1]
        if (class_value not in separated):
            separated[class_value] = list()
        separated[class_value].append(vector)
    return separated

# Calculate the mean of a list of numbers
def mean(numbers):
    return sum(numbers)/float(len(numbers))

# Calculate the standard deviation of a list of numbers
def stdev(numbers):
    avg = mean(numbers)
    variance = sum([(x-avg)**2 for x in numbers]) / float(len(numbers)-
1)
    return sqrt(variance)

# Calculate the mean, stdev and count for each column in a dataset
def summarize_dataset(dataset):
```

```
    summaries = [(mean(column), stdev(column), len(column)) for column
in zip(*dataset)]
    del(summaries[-1])
    return summaries

# Split dataset by class then calculate statistics for each row
def summarize_by_class(dataset):
    separated = separate_by_class(dataset)
    summaries = dict()
    for class_value, rows in separated.items():
        summaries[class_value] = summarize_dataset(rows)
    return summaries

# Calculate the Gaussian probability distribution function for x
def calculate_probability(x, mean, stdev):
    exponent = exp(-((x-mean)**2 / (2 * stdev**2 )))
    return (1 / (sqrt(2 * pi) * stdev)) * exponent

# Calculate the probabilities of predicting each class for a given row

def calculate_class_probabilities(summaries, row):
    total_rows = sum([summaries[label][0][2] for label in summaries])
    probabilities = dict()
    for class_value, class_summaries in summaries.items():
        probabilities[class_value] = summaries[class_value][0][2]/float
(total_rows)
        for i in range(len(class_summaries)):
            mean, stdev, _ = class_summaries[i]
            probabilities[class_value] *= calculate_probability(row[i],
 mean, stdev)
    return probabilities

# Predict the class for a given row
def predict(summaries, row):
    probabilities = calculate_class_probabilities(summaries, row)
    best_label, best_prob = None, -1
    for class_value, probability in probabilities.items():
        if best_label is None or probability > best_prob:
            best_prob = probability
            best_label = class_value
    return best_label

# Make a prediction with Naive Bayes on Iris Dataset
filename = 'iris.csv'
dataset = load_csv(filename)
for i in range(len(dataset[0])-1):
    str_column_to_float(dataset, i)
# convert class column to integers
```

```
str_column_to_int(dataset, len(dataset[0])-1)
# fit model
model = summarize_by_class(dataset)
# define a new record
row = [5.7,2.9,4.2,1.3]
# predict the label
label = predict(model, row)
print('Data=%s, Predicted: %s' % (row, label))
```

The output that I am getting is

[Iris-versicolor] => 0
[Iris-sentosa] => 1
[Iris-virginica] => 2

Data = [5.7, 2.9, 4.2, 1.3], Predicted: 0

Using Sklearn

```
from sklearn import datasets

#Load dataset
wine = datasets.load_wine()
# print the names of the 13 features
print ("Features: ", wine.feature_names)

# print the label type of wine(class_0, class_1, class_2)
print ("Labels: ", wine.target_names)

# Import train_test_split function
from sklearn.model_selection import train_test_split

# Split dataset into training set and test set
X_train, X_test, y_train, y_test = train_test_split(wine.data, wine.tar
get, test_size=0.3,random_state=109)

#Import Gaussian Naive Bayes model
from sklearn.naive_bayes import GaussianNB

#Create a Gaussian Classifier
gnb = GaussianNB()

#Train the model using the training sets
gnb.fit(X_train, y_train)

#Predict the response for test dataset
y_pred = gnb.predict(X_test)

print("y_pred: ",y_pred)

#Import scikit-learn metrics module for accuracy calculation
```

```
from sklearn import metrics

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

The output that I got is

Features: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_phenols', 'proanthocyanins', 'color_intensity', 'hue', 'od280/od315_of_diluted_wines', 'proline']

Labels: ['class_0' 'class_1' 'class_2']

y_pred: [0 0 1 2 0 1 0 0 1 0 2 2 2 2 0 1 1 0 0 1 2 1 0 2 0 0 1 2 0 1 2 1 1 0 1 1 0 2 2 0 2 1 0 0 0 2 2 0 1 1 2 0 0 2]

Accuracy: 0.9074074074074074

Using TensorFlow

```
from IPython import embed
from matplotlib import colors
from matplotlib import pyplot as plt
from sklearn import datasets
import numpy as np
import tensorflow as tf
from sklearn.utils.fixes import logsumexp

class TFNaiveBayesClassifier:
    dist = None

    def fit(self, X, y):
        # Separate training points by class (nb_classes * nb_samples * nb_features)
        unique_y = np.unique(y)
        points_by_class = np.array([
            [x for x, t in zip(X, y) if t == c]
            for c in unique_y
        ])

        # Estimate mean and variance for each class / feature
        # shape: nb_classes * nb_features
        mean, var = tf.nn.moments(tf.constant(points_by_class), axes=[1])

        # Create a 3x2 univariate normal distribution with the known mean and variance
        self.dist = tf.distributions.Normal(loc=mean, scale=tf.sqrt(var))

    def predict(self, X):
        assert self.dist is not None
```

```
        nb_classes, nb_features = map(int, self.dist.scale.shape)

        # Conditional probabilities log P(x|c) with shape (nb_samples,
nb_classes)
        cond_probs = tf.reduce_sum(
            self.dist.log_prob(
                tf.reshape(
                    tf.tile(X, [1, nb_classes]), [-1, nb_classes,
nb_features]
                )
            ),
            axis=2
        )

        # Uniform priors
        priors = np.log(np.array([1. / nb_classes] * nb_classes))

        # Posterior log probability, log P(c) + log P(x|c)
        joint_likelihood = tf.add(priors, cond_probs)

        # Normalize to get (log)-probabilities
        norm_factor = tf.reduce_logsumexp(joint_likelihood, axis=1,
keep_dims=True)
        log_prob = joint_likelihood - norm_factor

        # Exp to get the actual probabilities
        return tf.exp(log_prob)

if __name__ == '__main__':
    iris = datasets.load_iris()
    # Only take the first two features
    X = iris.data[:, :2]
    y = iris.target

    tf_nb = TFNaiveBayesClassifier()
    tf_nb.fit(X, y)

    # Create a regular grid and classify each point
    x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
    y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 30),
                         np.linspace(y_min, y_max, 30))
    s = tf.Session()
    Z = s.run(tf_nb.predict(np.c_[xx.ravel(), yy.ravel()]))
    # Extract probabilities of class 2 and 3
    Z1 = Z[:, 1].reshape(xx.shape)
    Z2 = Z[:, 2].reshape(xx.shape)
```

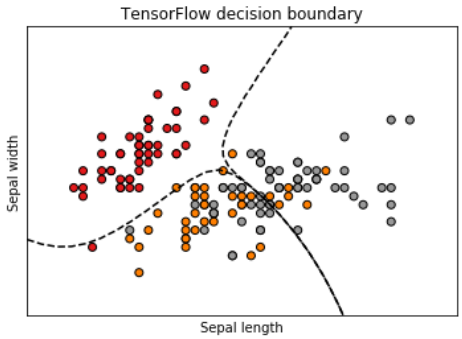
```
    # Plot
    fig = plt.figure(figsize=(5, 3.75))
    ax = fig.add_subplot(111)

    ax.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Set1, edgecolor='k')
    # Swap signs to make the contour dashed (MPL default)
    ax.contour(xx, yy, -Z1, [-0.5], colors='k')
    ax.contour(xx, yy, -Z2, [-0.5], colors='k')

    ax.set_xlabel('Sepal length')
    ax.set_ylabel('Sepal width')
    ax.set_title('TensorFlow decision boundary')
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(y_min, y_max)
    ax.set_xticks(())
    ax.set_yticks(())

    plt.tight_layout()
    fig.savefig('tf_iris.png', bbox_inches='tight')
```

The output that I got is:



Conclusion

In this chapter, we studied key terms, types of experiments, important definitions, probability, types of events, conditional probability, total probability, Baye's theorem, random variable, probability distributions, Baye's theorem explanation using an example, when is naive Bayes classifier used, difference between Bayes and naive Bayes algorithms, advantages of naive Bayes algorithm, disadvantages of naive Bayes algorithm, assumptions of naive Bayes algorithm, things to keep in mind to get the best out of naive Bayes, types of naive Bayes algorithm, what is naive Bayes, Python implementation of naive Bayes algorithm using functions, sklearn and tensorflow. Hope you were able to understand each and everything.

9

K Nearest Neighbor

Overview

In this chapter, we delve into the K-Nearest Neighbors (KNN) algorithm, covering its core concepts, applications, and practical implementations. Prepare yourself for an insightful journey that will enhance your understanding of KNN's role in classification and regression, its comparison with K-Means clustering, and its implementation using various tools and techniques.

What is K nearest neighbor used for?

KNN can be used for statistical questions of both gradation and regression. Nonetheless, it is more widely used in business grouping problems. K-NN is often used to locate "related" objects in a search; when the job is some sort of "find things like this." We use k-NN when we have less than 20 features, normally normalized or we have lots of training data per instance (attributes).

What is KNN?

KNN is a non-parametric lazy learning algorithm, i.e. it makes no assumptions on the underlying data, nor does it have an explicit or very limited training process.

The purpose of k-NN is to find the following data points, which are at a minimum distance from the present data point and club them into one class.

k is the max value of data point(s) that can be clubbed under a given class.

To find the distance we use employ various algorithms like Euclidean Distance, Hamming Distance, Manhattan Distance, Minkowski Distance, etc.

For example, 1-NN means that we must generate a model that will have classes based on the data point which is at the least distance. Similarly, 2-NN means that we must generate a model that will have classes based on the 2 data points with the least distances.

The algorithm of k-NN or K-Nearest Neighbors is:

- Computes the distance between the new data point with every training example.
- For computing, distance measures such as Euclidean distance, Hamming distance or Manhattan distance will be used.
- The model picks K entries in the database which are closest to the new data point.
- Then it does the majority vote i.e. the most common class/label among those K entries will be the class of the new data point.

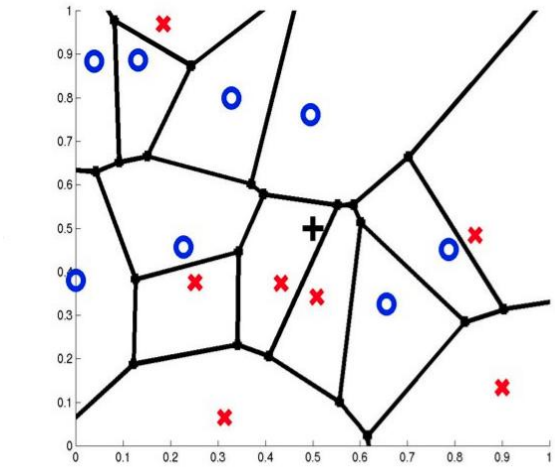
Steps involved in the processing and generating a model

- Decide on your similarity or distance metric.
- Split the original labeled dataset into training and test data.
- Pick an evaluation metric.
- Decide upon the value of k. Here k refers to the number of closest neighbors we will consider while doing the majority voting of target labels.
- Run k-NN a few times, changing k and checking the evaluation measure.
- In each iteration, k neighbors vote, majority vote wins and becomes the ultimate prediction
- Optimize k by picking the one with the best evaluation measure.
- Once you've chosen k, use the same training set and now create a new test set with the people's wages and incomes that you have no labels for, and want to predict.

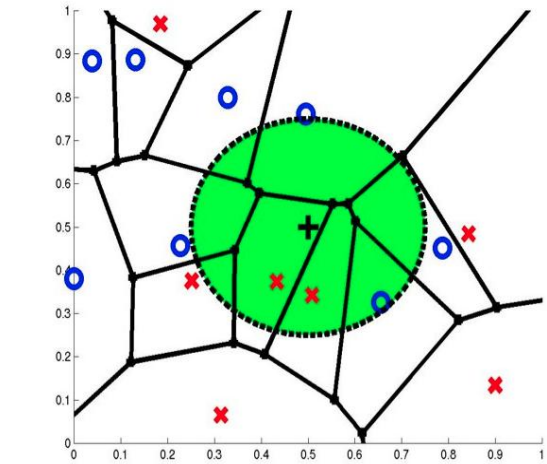
Steps involved in selecting the value for K

- Determined experimentally
- Start with k=1 and use a test set to validate the error rate of the classifier
- Repeat with k=k+2
- Choose the value of k for which the error rate is minimum
  Note: k should be an odd number to avoid ties

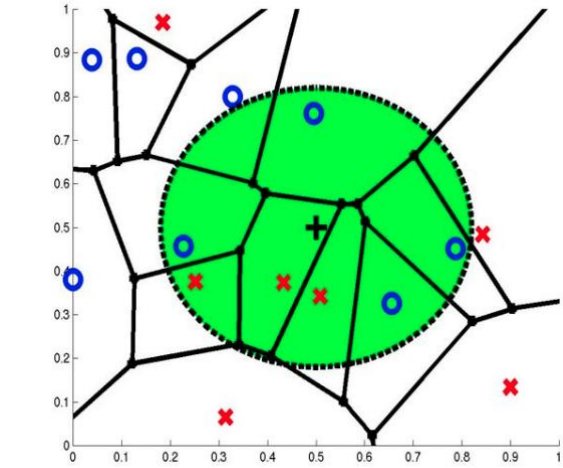
1-NN



3-NN



7-NN



KNN vs K-Means Clustering

	k-NN	k-Means
Types	Supervised	Unsupervised
What is K?	Number of closest neighbors to look at	Number of centroids
Calculation of prediction error possible	Yes	No
Optimization is done using what?	Cross-validation and confusion matrix	Elbow method and the silhouette method
Algorithm convergences	When all observations classified at the desired accuracy	When cluster membership doesn't change anymore
Complexity	Training: O(Dimensions/Features) Test: O (Number of Observations)	O (Number of points * Number of Clusters * Number of iterations * number of attributes)

Curse of Dimensionality

Consider instances with 20 characteristics (attributes) but three are important to the destination function. The curse of dimensionalism implies that when the instant space is wide and occupied by many irrelevant features, the nearest neighbour can easily be fooled.

A solution to this can be

- Stretch j-th axis by weight zj, where z1…, zn chosen to minimize prediction error (weight different features differently)
- Use cross-validation to automatically choose weights z1…, zn
- Note setting zj to zero eliminates this dimension altogether (feature subset selection)
- PCA

Advantages/Features of KNN

- K-NN is intuitive and simple
  The K-NN algorithm is easy to implement and very simple to understand. It reads through the whole dataset to classify the new data point and to find out K nearest neighbors.
- K-NN has fewer assumptions
  K-NN is a non-parametric algorithm which means there are fewer assumptions to be met to implement K-NN. Parametric models like linear regression have lots of assumptions to be met by data before it can be implemented which is not the case with K-NN.
- No Training Step
  K-NN does not explicitly build any model and simply tags the new data entry-based learning from historical data, hence no training step is required.
- It constantly evolves
  The classifier immediately adapts as we collect new training data and respond quickly to changes in the input during real-time use.
- Very easy to implement for the multi-class problem
  K-NN adjusts to multi-class without any extra efforts as compared to other algorithms.
- Can be used both for Classification and Regression
  One of the biggest advantages of K-NN is that K-NN can be used both for classification and regression problems.

- One Hyper Parameter
  K-NN might take some time while selecting the first hyperparameter but after that rest of the parameters are aligned to it.
- KNN stores the entire training dataset which it uses as its representation.
- KNN does not learn any model.
- KNN makes predictions just-in-time by calculating the similarity between an input sample and each training instance.
- A variety of distance criteria to choose from the K-NN algorithm gives the user the flexibility to choose distance while building a K-NN model.
  - Euclidean Distance
  - Hamming Distance
  - Manhattan Distance
  - Minkowski Distance

Disadvantages/Shortcomings of KNN

- K-NN is a slow algorithm
  K-NN might be very easy to implement except for the fact that as the dataset grows efficiency/speed of algorithm declines very fast.
- Curse of Dimensionality
  KNN predicting efficiency decreases as the numbers of variables grow works.
- K-NN needs homogeneous features
  If you decide to build k-NN using a common distance, like Euclidean or Manhattan distances, it is completely necessary that features have the same scale, since absolute differences in features weight the same, i.e., a given distance in feature 1 must mean the same for feature 2.
- An optimal number of neighbors
  One of the biggest and most crucial issues with K-NN is to choose the optimal number of neighbors to be considered while classifying the new data entry.
- Imbalanced data causes problems
  k-NN doesn't perform well on imbalanced data.
- Outlier sensitivity
  K-NN algorithm is very sensitive to outliers as it simply chose the neighbors based on distance criteria.
- Missing Value treatment
  K-NN inherently has no capability of dealing with missing value problems.

Assumptions of KNN

- k-NN performs much better if all the data have the same scale
- k-NN works well with a small number of input variables (p) but struggles when the number of inputs is very large
- k-NN makes no assumptions about the functional form of the problem is solved

Ways to calculate the distance

Euclidean distance

$$dist\,(A,B) = \sqrt{\frac{\sum_{i=1}^{m}(x_i - y_i)^2}{m}}$$

Manhattan Distance

$$distance = \sum_{i=0}^{n-1} |(x[i] - y[i])|$$

Chi-Square Distance

$$dist_Chi\text{-}square\,(A,B) = \sum_{i=1}^{m} \frac{1}{sum_i} \left(\frac{x_i}{size_Q} - \frac{y_i}{size_I} \right)^2$$

Correlation Distance

$$dist_correlation\,(A,B) = \frac{\sum_{i=1}^{m} (x_i - \mu_i)(y_i - \mu_i)}{\sqrt{\sum_{i=1}^{m} (x_i - \mu_i)^2 \sum_{i=1}^{m} (y_i - \mu_i)^2}}$$

Hamming Distance

$$D_H = \sum_{i=1}^{k} |x_i - y_i|$$

Minkowsky Distance

$$dist_Minkowsky\,(A,B) = \left(\sum_{i=1}^{m} |x_i - y_i|^r \right)^{1/r}$$

KNN using an example

Let us understand how k-NN really works. For that let's take a dummy dataset.

```
data = [
      [65.75, 112.99],
      [71.52, 136.49],
      [69.40, 153.03],
      [68.22, 142.34],
      [67.79, 144.30],
      [68.70, 123.30],
      [69.80, 141.49],
      [70.01, 136.46],
      [67.90, 112.37],
      [66.49, 127.45],
   ]
```

In the above example, the data is of the form [feature, target].

Assuming the value of k to be 3 i.e. 3-NN, let's try to find the prediction for feature value "60".

So, the 3 nearest neighbors would be

a. 65.75 with a distance value 5.75
b. 66.49 with a distance value 6.49
c. 67.79 with a distance value 7.79

And hence the predicted value will be 128.25.

Similarly, let's take another dataset

```
data = [
    [22, 1],
    [23, 1],
    [21, 1],
    [18, 1],
    [19, 1],
    [25, 0],
    [27, 0],
    [29, 0],
    [31, 0],
    [45, 0],
]
```

In the above example, the data is of the form [feature, target].

Assuming the value of k to be 3 i.e. 3-NN, let's try to find the prediction for feature value "33".

So, the 3 nearest neighbors would be

a. 31 with a distance value 2.0
b. 29 with a distance value 4.0
c. 27 with a distance value 6.0

And hence the predicted value will be 0.

Python Implementation of Decision Tree

Let's take the example of the MNIST dataset, you can directly import it from the sklearn dataset repository or download it from the chapter. Feel free to use any dataset, there are some very good datasets available on Kaggle and with Google Colab.

From Scratch

Here, I will be using some dummy data. You can use and practice on any kind of data

```
from collections import Counter
import math
```

In the above code, we are exporting the required libraries.

```
def knn(data, query, k, distance_fn, choice_fn):
    neighbor_distances_and_indices = []

    # 3. For each example in the data
    for index, example in enumerate(data):
        # 3.1 Calculate the distance between the query example and the
current
        # example from the data.
        distance = distance_fn(example[:-1], query)

        # 3.2 Add the distance and the index of the example to an order
ed collection
        neighbor_distances_and_indices.append((distance, index))

    # 4. Sort the ordered collection of distances and indices from
```

```
    # smallest to largest (in ascending order) by the distances
    sorted_neighbor_distances_and_indices = sorted(neighbor_distances_a
nd_indices)

    # 5. Pick the first K entries from the sorted collection
    k_nearest_distances_and_indices = sorted_neighbor_distances_and_ind
ices[:k]

    # 6. Get the labels of the selected K entries
    k_nearest_labels = [data[i][1] for distance, i in k_nearest_distanc
es_and_indices]

    # 7. If regression (choice_fn = mean), return the average of the K
labels
    # 8. If classification (choice_fn = mode), return the mode of the K
 labels
    return k_nearest_distances_and_indices, choice_fn(k_nearest_labels)
```

In the above code, we have created a function that will perform all the required tasks to result out a k-NN based model

```
#function to calculate the mean used in case of regression
def mean(labels):
    return sum(labels) / len(labels)

#function to calculate the mode used in case of classification
def mode(labels):
    return Counter(labels).most_common(1)[0][0]

#function to calculate the distance between two data points
def euclidean_distance(point1, point2):
    sum_squared_distance = 0
    for i in range(len(point1)):
        sum_squared_distance += math.pow(point1[i] - point2[i], 2)
    return math.sqrt(sum_squared_distance)
```

The above code lists down all the auxiliary functions used during the program

```
# Regression Data
    #
    # Column 0: height (inches)
    # Column 1: weight (pounds)
    '''
    reg_data = [
       [65.75, 112.99],
       [71.52, 136.49],
       [69.40, 153.03],
       [68.22, 142.34],
       [67.79, 144.30],
```

```
        [68.70, 123.30],
        [69.80, 141.49],
        [70.01, 136.46],
        [67.90, 112.37],
        [66.49, 127.45],
    ]
```

The above code, declares the dummy regression data, in the above data column 0 signifies height and column 1 signifies weight

```
reg_query = [60]
reg_k_nearest_neighbors, reg_prediction = knn(
reg_data, reg_query, k=3, distance_fn=euclidean_distance, choice_fn=mea
n
)
print(reg_k_nearest_neighbors, reg_prediction)
```

In the above code, we will generate a model, and then based on that model will predict the weight for a person with a height of 60

```
# Classification Data
#
# Column 0: age
# Column 1: likes pineapple
'''
clf_data = [
    [22, 1],
    [23, 1],
    [21, 1],
    [18, 1],
    [19, 1],
    [25, 0],
    [27, 0],
    [29, 0],
    [31, 0],
    [45, 0],
]
```

The above code, declares the dummy data for classification-based problem, where column 0 signifies age and column 1 signifies if a person like pineapple or not

```
clf_query = [33]
clf_k_nearest_neighbors, clf_prediction = knn(
              clf_data, clf_query, k=3, distance_fn=euclidean_distance,
choice_fn=mode)
print(clf_k_nearest_neighbors, clf_prediction)
```

In the above code, we first create a model and based on that model, we predict if a 33 years old person likes pineapple or not.

scratch_knn.py

```
from collections import Counter
import math

def knn(data, query, k, distance_fn, choice_fn):
    neighbor_distances_and_indices = []

    # 3. For each example in the data
    for index, example in enumerate(data):
        # 3.1 Calculate the distance between the query example and the
current
        # example from the data.
        distance = distance_fn(example[:-1], query)

        # 3.2 Add the distance and the index of the example to an order
ed collection
        neighbor_distances_and_indices.append((distance, index))

    # 4. Sort the ordered collection of distances and indices from
    # smallest to largest (in ascending order) by the distances
    sorted_neighbor_distances_and_indices = sorted(neighbor_distances_a
nd_indices)

    # 5. Pick the first K entries from the sorted collection
    k_nearest_distances_and_indices = sorted_neighbor_distances_and_ind
ices[:k]

    # 6. Get the labels of the selected K entries
    k_nearest_labels = [data[i][1] for distance, i in k_nearest_distanc
es_and_indices]

    # 7. If regression (choice_fn = mean), return the average of the K
labels
    # 8. If classification (choice_fn = mode), return the mode of the K
 labels
    return k_nearest_distances_and_indices , choice_fn(k_nearest_labels
)

def mean(labels):
    return sum(labels) / len(labels)

def mode(labels):
    return Counter(labels).most_common(1)[0][0]

def euclidean_distance(point1, point2):
    sum_squared_distance = 0
    for i in range(len(point1)):
        sum_squared_distance += math.pow(point1[i] - point2[i], 2)
    return math.sqrt(sum_squared_distance)
```

```
def main():
    '''''
    # Regression Data
    #
    # Column 0: height (inches)
    # Column 1: weight (pounds)
    '''
    reg_data = [
       [65.75, 112.99],
       [71.52, 136.49],
       [69.40, 153.03],
       [68.22, 142.34],
       [67.79, 144.30],
       [68.70, 123.30],
       [69.80, 141.49],
       [70.01, 136.46],
       [67.90, 112.37],
       [66.49, 127.45],
    ]

    # Question:
    # Given the data we have, what's the best-
guess at someone's weight if they are 60 inches tall?
    reg_query = [60]
    reg_k_nearest_neighbors, reg_prediction = knn(
        reg_data, reg_query, k=3, distance_fn=euclidean_distance, choic
e_fn=mean
    )
    print(reg_k_nearest_neighbors, reg_prediction)

    '''''
    # Classification Data
    #
    # Column 0: age
    # Column 1: likes pineapple
    '''
    clf_data = [
       [22, 1],
       [23, 1],
       [21, 1],
       [18, 1],
       [19, 1],
       [25, 0],
       [27, 0],
       [29, 0],
       [31, 0],
       [45, 0],
```

```
    ]
    # Question:
    # Given the data we have, does a 33-year-
old like pineapples on their pizza?
    clf_query = [33]
    clf_k_nearest_neighbors, clf_prediction = knn(
        clf_data, clf_query, k=3, distance_fn=euclidean_distance, choic
e_fn=mode
    )
    print(clf_k_nearest_neighbors, clf_prediction)

if __name__ == '__main__':
    main ()
```

The output that I got is:

Regression case:
Neighbors: [(5.75, 0), (6.489999999999995, 9), (7.790000000000006, 4)]
Prediction: 128.24666666666667

Classification case:
Neighbors: [(2.0, 8), (4.0, 7), (6.0, 6)]
Prediction: 0

Using SkLearn

In sklearn, we have KNeighborsClassifier class, used for implementing k-NN

knn_sklearn.py

```
X = [[0], [1], [2], [3]]
y = [0, 0, 1, 1]
from sklearn.neighbors import KNeighborsClassifier
neigh = KNeighborsClassifier(n_neighbors=2)
neigh.fit(X, y)
print(neigh.predict([[4.567]]))
print(neigh.predict_proba([[4.567]]))
```

In the above code, we create a dummy data, which is then passed to the classifier, to get the predicted value and predicting probability corresponding to 4.567

The output that I got is
Predicted value: [1]
Predicting Probability: [[0. 1.]]

Using TensorFlow

Here I will be using the MNSIT cloth dataset.

```
from __future__ import print_function

import numpy as np
import tensorflow as tf
```

In the above code, we are importing all the required libraries.

In the below code, I have commented on Line numbers 41 and 42, you can uncomment those if you want to see the complete output.

knn_tensorflow.py

```
from __future__ import print_function

import numpy as np
import tensorflow as tf

# Import MNIST data
from tensorflow.examples.tutorials.mnist import input_data
mnist = input_data.read_data_sets("/tmp/data/", one_hot=True)

# In this example, we limit mnist data
Xtr, Ytr = mnist.train.next_batch(5000) #5000 for training (nn candidat
es)
Xte, Yte = mnist.test.next_batch(200) #200 for testing

# tf Graph Input
xtr = tf.placeholder("float", [None, 784])
xte = tf.placeholder("float", [784])

# Nearest Neighbor calculation using L1 Distance
# Calculate L1 Distance
distance = tf.reduce_sum(tf.abs(tf.add(xtr, tf.negative(xte))), reducti
on_indices=1)
# Prediction: Get min distance index (Nearest neighbor)
pred = tf.arg_min(distance, 0)

accuracy = 0.

# Initialize the variables (i.e. assign their default value)
init = tf.global_variables_initializer()

# Start training
with tf.Session() as sess:

    # Run the initializer
    sess.run(init)

    # loop over test data
    for i in range(len(Xte)):
        # Get nearest neighbor
        nn_index = sess.run(pred, feed_dict={xtr: Xtr, xte: Xte[i, :]})

        # Get nearest neighbor class label and compare it to its true l
abel

        #print("Test", i, "Prediction:", np.argmax(Ytr[nn_index]), \
```

```
        #   "True Class:", np.argmax(Yte[i]))
        # Calculate accuracy
        if np.argmax(Ytr[nn_index]) == np.argmax(Yte[i]):
            accuracy += 1./len(Xte)
    print("Done!")
    print("Accuracy:", accuracy)
```

The Output that I got is

Accuracy: 0.9450000000000007

Conclusion

In this chapter, we studied what is k-NN used for, what is k-NN, 1NN, 3NN, 7NN, k-NN vs k-means clustering, the curse of dimensionality, advantages of k-NN, disadvantages of k-NN, assumptions of k-NN, euclidean distance, manhattan distance, chi-square, minkowsky distance, correlation distance, hamming distance, k-NN using an example and python implementation of the k-NN algorithm using functions, sklearn, and TensorFlow. Hope you were able to understand each and everything.

10

K-Means Clustering

Overview

In this chapter, we explore k-Means Clustering, focusing on its key concepts, advantages, and applications. We discuss the basics of clustering, the specifics of k-Means, and provide practical examples using Python. Get ready to gain a clear understanding of how k-Means works and its practical uses in data analysis.

What is k-Means Clustering?

k-means algorithm is the iterative algorithm attempting to divide the dataset into ' k ' predefined non-surrounding sub-groups and/or clusters with only one group of each data point. This attempts to make data points as close as possible and at the same time keep the clusters as distinct or as far as possible. It assigns data points to a cluster such that the sum of the squared distance between the data points and the cluster's centroid is at the minimum. The less variance we have in the samples, the more consistent the data points in the same cluster are

Why k-Means Clustering?

- K-means converges in a finite number of iterations. Since the algorithm iterates a function, whose domain is a finite set, the iteration must eventually converge.
- The computational cost of the k-means algorithm is O(k*n*d), where n is the number of data points, k the number of clusters, and d the number of attributes.
- Compared to other clustering methods, the k-means clustering technique is fast and efficient in terms of its computational cost.
- It's difficult to predict the optimal number of clusters or the value of k. To find the number of clusters, we need to run the k-means clustering algorithm for a range of k values and compare the results.

What is meant by Clustering?

Clustering is the task of dividing the population or data points into a number of groups such that data points in the same groups are more similar to other data points in the same group and dissimilar to the data points in other groups.

Types of Clustering

Centroid-based Clustering

In comparison to the hierarchical clusters described below, centroid dependent clustering organizes data into non-hierarchical clusters. The centroid-based clustering algorithm is the most commonly used k-means. Algorithms focused on centroids are efficient but prone to initial conditions and outliers.

Density-based Clustering

Clustering based on density links high example density areas into clusters. This allows arbitrary distributions while connecting dense areas. For data in different densities and large dimensions, these algorithms are complicated. In fact, these algorithms do not distribute cluster boundaries by default.

Distribution-based Clustering

This clustering approach supposes that data are made up of distributors, for example Gaussian. Data are divided into three Gussian distributions by a distribution-based algorithm. As the distance from the middle of the distribution increases, the probability of a point contributing to the distribution reduces. The bands are likely to decrease. If the form of distribution in y is not known.

Hierarchical Clustering

It creates a cluster vine. Not surprisingly, hierarchical clusters are well-suited for hierarchical data, such as taxonomy. Any number of clusters may be chosen by cutting the tree to the right level. It is also advantageous

Difference between Clustering and Regression

Parameter	CLASSIFICATION	CLUSTERING
Type	Supervised Learning	Unsupervised Learning
Description	Process of classifying the input instances based on their corresponding class labels	Grouping the instances based on their similarity without the help of class labels
The necessity of Training and Testing Dataset	It has labels so there is a need for training and testing dataset for verifying the model created	There is no need for training and testing dataset
Complexity	More complex as compared to clustering	Less complex as compared to the classification
Example Algorithms	Logistic regression, Naive Bayes classifier, Support vector machines, etc.	k-means clustering algorithm, Fuzzy c-means clustering algorithm, Gaussian (EM) clustering algorithm, etc.

Why is k-Means clustering nondeterministic?

The basic k-means clustering is based on a non-deterministic algorithm. It implies that the algorithm may result in different outcomes if it is performed on the same data many times, i. e. it may be necessary for a certain class to have a specific core value at different times for the same dataset. For instance, at a given instance we took D1 and D2 to find the centroid and then we took D1 and D5, the value of centroid will change, hence the resulting model will change

How does k-Means Clustering work?

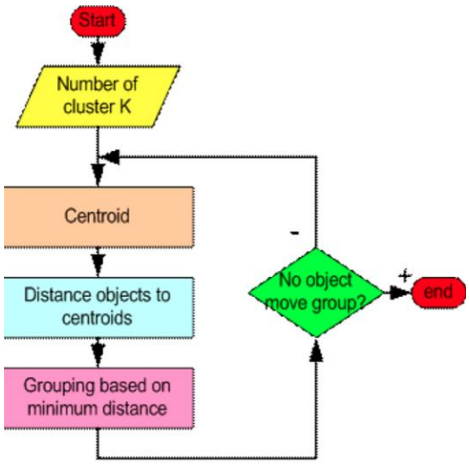
Step 1: Begin with a decision on the value of k = number of clusters.

Step 2: Put any initial partition that classifies the data into k clusters. You may assign the training samples randomly, or systematically as the following:

- Take the first k training sample as single- element clusters
- Assign each of the remaining (N-k) training samples to the cluster with the nearest centroid. After each assignment, recompute the centroid of the gaining cluster.

Step 3: Take any sample in a series and measure the distance from the center of each cluster. If a sample is not in the close-up cluster, assign it to that cluster and change the middle part of the cluster that collects the latest sample and the cluster that lacks the result.

Step 4: Repeat step 3 until convergence is achieved, that is until a pass through the training sample causes no new assignments.



What are Centroids?

The centroid or geometric center of a figure of a plane in mathematics and physics is the mean arithmetic position of all the points of the figure. Informatively, the point at which the cut of the form can be balanced perfectly at the end of a pin.

What are Centroids in k-Means?

The k-means clustering algorithm attempts a partition into a fixed number (k), a group of clusters, of a specified anonymous data collection. The k number of so-called centroids are initially chosen. A centroid is a data point (imaginary or real) at the center of a cluster. Each centroid is an existing data point in the given input data set, picked at random, such that all centroids are unique (that is, for all centroids ci and cj, ci ≠ cj). These centroids are used to train a classifier. The resulting classifier is used to classify the data (using k = 1) and thus produce an initial randomized cluster set. Afterwards, each centroid is set to the arithmetic mean of the cluster it defines. The classification process and centroid adjustment process is repeated until the centroid values stabilize. The final centers are used to generate the final classification and clustering of the input data, essentially transforming a series of data points, each with a class identity, which are originally anonymous.

Advantages/Features of k-Means

- Easy to implement with many variables, K--Means may be computationally faster than hierarchical clustering (if K is small).
- k-Means may produce tighter clusters than hierarchical clustering
- An instance can change cluster (move to another cluster) when the centroids are re--computed

Disadvantages/Shortcomings of k-Means

- Difficult to predict the number of clusters (K-Value)
- Initial seeds have a strong impact on the final results
- The order of the data has an impact on the final results
- Sensitive to scale: rescaling your datasets (normalization or standardization) will completely change results. While this itself is not bad, not realizing that you must spend extra a4ention to scaling your data might be bad.

Applications of k-Means

k-means can be applied to data that has a smaller number of dimensions, is numeric, and is continuous.

- Document Classification
- Delivery Store Optimization
- Identifying Crime Localities
- Customer Segmentation
- Fantasy League Stat Analysis
- Insurance Fraud Detection
- Rideshare Data Analysis
- Cyber-Profiling Criminals
- Call Record Detail Analysis
- Automatic Clustering of IT alerts
- k-Means using an Example

Let's try to understand k-means using a mathematical example. Given the following dummy dataset with two clusters A and B.

Iteration 1

Data Points	X axis	Y axis
D1	2	0
D2	1	3
D3	3	5
D4	2	2
D5	4	6

Let's start by finding the Euclidean Distance given by the formula: $\sqrt{((x1-y1)^2 + (x2-y2)^2}$

Distance between D1 and D2 : $\sqrt{((2-1)^2 + (0-3)^2}$
$= \sqrt{((1)^2 + (3)^2}$
$= \sqrt{(10)}$
$= 3.17$

Distance between D1 and D4 : $\sqrt{((2-2)^2 + (0-2)^2}$
$= \sqrt{((0)^2 + (-2)^2}$
$= \sqrt{(4)}$
$= 2$

Similarly, let us find the distance between other points

Data Points	Distance between D2 and other data points	Distance between D4 and other data points
D1	3.17	2.0
D3	2.83	3.17
D5	4.25	4.48

Seeing the above data, we can form the following two groups
Cluster 1: D1, D4
Cluster 2: D2, D3, D5

Now our next step is to find the mean values and the new centroid values with respect to the corresponding to the centroid values

Data Points	Mean value of data points along the x-axis	Distance between D4 and other data point
D1, D4	2.0	1.0
D2, D3, D5	2.67	4.67

Hence the centroid values are
Cluster 1: (2.0, 1.0)
Cluster 2: (2.67, 4.67)

Iteration 2

Based on the above data, we again calculate the euclidean distance for the new centroids.

Data Points	Distance between D2 and other data points	Distance between D4 and other data points
D1	1.0	4.72
D2	2.24	2.37
D3	4.13	0.47
D4	1	2.76
D5	5.39	1.89

From the above data, we infer the following
Cluster 1: (D1, D2, D4)
Cluster 2: (D3, D5)

Since the clusters have changed so we will now calculate the mean values again and the corresponding centroid values

Data Points	Mean value of data points along the x-axis	Distance between D4 and other data point
D1, D2, D4	1.67	1.67
D3, D5	3.5	5.5

So now the centroid values are:
Cluster 1: (1.67, 1.67)
Cluster 2: (3.5, 5,5)

The above process has to be repeated until we find a constant value for centroids and the latest cluster will be considered as the final cluster solution.

Python Implementation of Decision Tree

I have used Google Colab to run and test my code, you can use any of the IDEs or development environments. Feel free to use any dataset, there are some very good datasets available on Kaggle and with Google Colab.

Using functions

In this, I will be using some dummy data, you can use any data that you want.

```
# Importing the required libraries
import matplotlib.pyplot as plt
from matplotlib import style
import numpy as np

style.use('ggplot')

# Dummy data
X = np.array([[1, 2],
              [1.5, 1.8],
              [5, 8],
              [8, 8],
              [1, 0.6],
              [9, 11],
              [1, 3],
              [8, 9],
              [0, 3],
```

```
                [5, 4],
                [6, 4],
                [1, 2],
                [2, 4],
                [11, 5],
                [2.56, 5.6],
                [2, 2],
                [1.8, 1.8],
                [8, 8],
                [5, 5],
                [0.6, 0.6],
                [11, 11],
                [3, 3],
                [9, 9]])

# Plot of the Dummy Data
plt.scatter(X[:, 0], X[:, 1], s=150)
plt.show()

# Defining the colors that can be used in plots
colors = ["g", "r", "c", "b", "k"]

# Definition of the class K_Means
# Using this class we are implementing all the steps required
# to be performed to generate a k-means model
class K_Means:
    def __init__(self, k=5, tol=0.001, max_iter=300):
        self.k = k
        self.tol = tol
        self.max_iter = max_iter

    # Function to fit the dummy model on to the data
    def fit(self, data):
        self.centroids = {}

        for i in range(self.k):
            self.centroids[i] = data[i]

        for i in range(self.max_iter):
            self.classifications = {}

            for i in range(self.k):
                self.classifications[i] = []

            for featureset in data:
                distances = [np.linalg.norm(featureset -
self.centroids[centroid]) for centroid in self.centroids]
                classification = distances.index(min(distances))
```

```
                self.classifications[classification].append(featureset)

            prev_centroids = dict(self.centroids)

            for classification in self.classifications:
                self.centroids[classification] =
np.average(self.classifications[classification], axis=0)

            optimized = True
            for c in self.centroids:
                original_centroid = prev_centroids[c]
                current_centroid = self.centroids[c]
                if np.sum((current_centroid - original_centroid) /
original_centroid * 100.0) > self.tol:
                    np.sum((current_centroid - original_centroid) /
original_centroid * 100.0)
                    optimized = False
            if optimized:
                break

    # Function to generate prediction result
    def predict(self, data):
        distances = [np.linalg.norm(data - self.centroids[centroid])
for centroid in self.centroids]
        classification = distances.index(min(distances))
        return classification

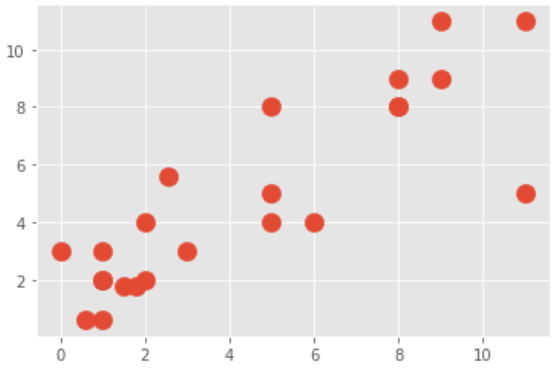
clf = K_Means()
clf.fit(X)

# Putting the centroid values onto the plot
for centroid in clf.centroids:
    plt.scatter(clf.centroids[centroid][0], clf.centroids[centroid][1],
                marker="o", color="k", s=150, linewidths=5)

# Plotting the classified points
for classification in clf.classifications:
    color = colors[classification]
    for featureset in clf.classifications[classification]:
        plt.scatter(featureset[0], featureset[1], marker="x",
color=color, s=150, linewidths=5)

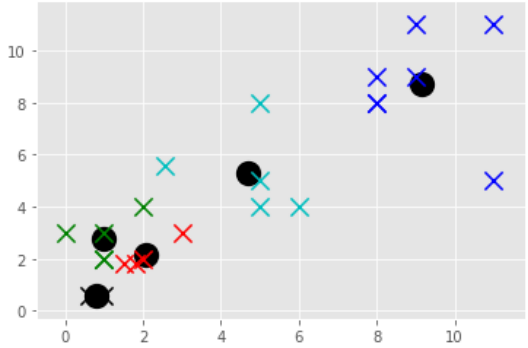
# Printing the centroid values
for centroid in clf.centroids:
    print(clf.centroids[centroid][0], clf.centroids[centroid][1])

plt.show()
```



The centroid values I am getting are:

1.0	2.8
2.075	2.15
4.712	5.32
9.142857142857142	8.714285714285714
0.8	0.6



Using Sklearn

Here I am using dummy data; you can use any dataset of your choice.

```
# importing all the required libraries
import matplotlib.pyplot as plt
%matplotlib inline
import numpy as np
from sklearn.cluster import KMeans
# Dummy data
X = np.array([[1, 2],
              [1.5, 1.8],
              [5, 8],
              [8, 8],
              [1, 0.6],
              [9, 11],
              [1, 3],
              [8, 9],
              [0, 3],
              [5, 4],
              [6, 4],
              [1, 2],
              [2, 4],
```

```
                [11, 5],
                [2.56, 5.6],
                [2, 2],
                [1.8, 1.8],
                [8, 8],
                [5, 5],
                [0.6, 0.6],
                [11, 11],
                [3, 3],
                [9, 9]])

# Plotting the dummy data
plt.scatter(X[:, 0], X[:, 1], label='True Position')
plt.show()

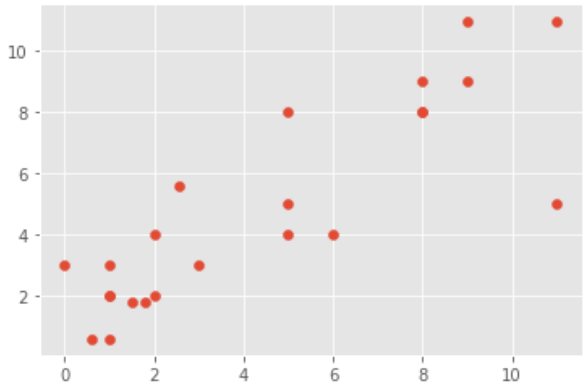
# Creating an object of sklearn.cluster.KMeans class
# Passed the number of clusters value to be 3, i.e., we are making a 3-
Means clustering model
kmeans = KMeans(n_clusters=3)

# Passing the data to the model
kmeans.fit(X)

# Printing the centroid values
print(kmeans.cluster_centers_)
print(kmeans.labels_)

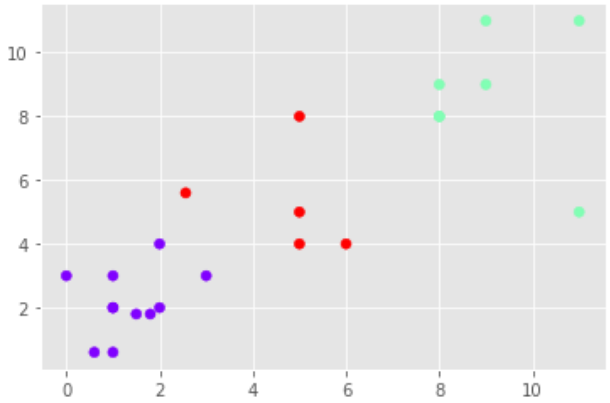
# Plotting the data based upon their predicted cluster
plt.scatter(X[:, 0], X[:, 1], c=kmeans.labels_, cmap='rainbow')
plt.show()
```

The output that I got is



Centroids:
[[1.35454545 2.16363636]
[9.14285714 8.71428571]
[4.712 5.32]]

Points classified into classes:
[0 0 2 1 0 1 0 1 0 2 2 0 0 1 2 0 0 1 2 0 1 0 1]



Using TensorFlow

Here I am using dummy data; you can use any dataset that you wish to use.

```
# importing the required libraries
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf

points_n = 200  # number of data points
clusters_n = 3  # number of classes in which points need to be
classified
iteration_n = 100  # epoch value

# initializing the placeholders
points = tf.constant(np.random.uniform(0, 10, (points_n, 2)))
centroids = tf.Variable(tf.slice(tf.random.shuffle(points), [0, 0],
[clusters_n, -1]))
points_expanded = tf.expand_dims(points, 0)
centroids_expanded = tf.expand_dims(centroids, 1)

# Creation of tensor to hold the euclidean distance formula
distances = tf.reduce_sum(tf.square(tf.subtract(points_expanded,
centroids_expanded)), 2)
assignments = tf.argmin(distances, 0)

# logic to generate the model
means = []
for c in range(clusters_n):
    means.append(tf.reduce_mean(
      tf.gather(points,
                tf.reshape(
                  tf.where(
                    tf.equal(assignments, c)
                  ),[1,-1])
               ),reduction_indices=[1]))

new_centroids = tf.concat(means, 0)
```

```
update_centroids = tf.assign(centroids, new_centroids)
init = tf.global_variables_initializer()

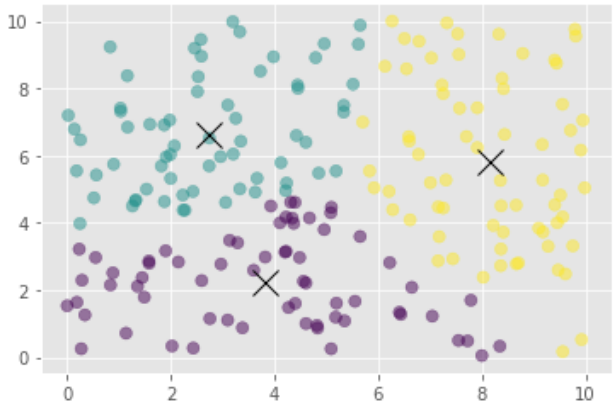
# session to execute the logic and generate the model
with tf.Session() as sess:
  sess.run(init)
  for step in range(iteration_n):
    [_, centroid_values, points_values, assignment_values] =
sess.run([update_centroids, centroids, points, assignments])

  # to print the centroid values
  print("centroids", centroid_values)

# plotting of the dummy data with centroid values
plt.scatter(points_values[:, 0], points_values[:, 1],
c=assignment_values, s=50, alpha=0.5)
plt.plot(centroid_values[:, 0], centroid_values[:, 1], 'kx',
markersize=15)
plt.show()
```

The output that I am getting is:

Centroids:
[[3.81697836 2.2322892]
[2.72036385 6.65968922]
[8.123787 5.82232576]]



Conclusion

In this chapter, we studied what k-means clustering is, why we use k-means clustering, what is meant by clustering, types of clustering, the difference between clustering and regression, how k-means clustering works, what are centroids, what are centroids in k-means, advantages of k-means, disadvantages of k-means, applications of k-means, k-means using an example and python implementation of the k-NN algorithm using functions, sklearn, and TensorFlow. Hope you were able to understand each and everything.

EPILOGUE

All the code used in the book belongs to the respective GitHub repository owners.
As I have been saying throughout the book, machine learning is the same as what we do with humans, the methods and techniques that we follow in making a human learn, we follow the same to make a machine learn.

At the end, I would like to break some myths,

1. Superintelligence by 2100 is impossible and inevitable

Superintelligence is a hypothetical agent that possesses intelligence far surpassing that of the brightest and most gifted human minds. I would say that it may happen in decades, centuries or never. AI experts disagree & we simply don't know. We are still far away from achieving complete superintelligence, but in some areas, we have been able to achieve superintelligence, for example, super computers like PARAM developed by India or SUMMIT Supercomputers, which at the time of writing the book was the world's fastest supercomputer, with a power efficiency of 14.668 GFlops/watt.

2. AI can turn evil and conscious

We all have seen Terminator, and other movies, where machines turned evil and became a threat to human existence. Machines can only be evil, if and only, if they turn competent with goals misaligned with humans. To make it understand better, let's take an example, in schools we are taught that cutting trees is harming environment, so a machine whose work is to protect the environment can be called evil as its goal is not aligned with those who are using trees to generate papers and other products.

3. AI can't control humans

We humans control others by using our intelligence, be it tigers or fellow humans. So now there are two types of people, one who believes that AI will control humans and others who think that AI can't control. Both the types are right and wrong at the same time, i.e. they are in Quantum state, as if AI becomes intelligent, they can control humans. But at what amount that is questionable.

Thanks for reading through the book. Hope you were able to learn about "What is the real meaning of Machine Learning".

C#Corner

OUR MISSION

Free Education is Our Basic Need! Our mission is to empower millions of developers worldwide by providing the latest unbiased news, advice, and tools for learning, sharing, and career growth. We're passionate about nurturing the next young generation and help them not only to become great programmers, but also exceptional human beings.

ABOUT US

CSharp Inc, headquartered in Philadelphia, PA, is an online global community of software developers. C# Corner served 29.4 million visitors in year 2022. We publish the latest news and articles on cutting-edge software development topics. Developers share their knowledge and connect via content, forums, and chapters. Thousands of members benefit from our monthly events, webinars, and conferences. All conferences are managed under Global Tech Conferences, a CSharp Inc sister company. We also provide tools for career growth such as career advice, resume writing, training, certifications, books and white-papers, and videos. We also connect developers with their potential employers via our Job board. Visit C# Corner

MORE BOOKS

